MASTER STUDY GUIDE · PRINT & COLOR SCIENCE

Computer Graphics for Print Production

Color Science · Prepress · PDF/X · Finishing — The Complete Beginner-to-Advanced Study Guide

Prepared 2026-06-21 · Read the sections in order

Table of Contents

How to Use This Guide

This guide teaches you how a digital design becomes ink on paper — accurately, predictably, and at scale. It is written for a **smart beginner**: someone who is comfortable with technology but new to the print world. The ideal reader is a **software developer building a print or design SaaS** — a tool where customers upload artwork, configure a product, and receive a physically printed result. To build that well, you need real fluency in the print domain, not just a vague mental model. That is exactly what this guide gives you.

Before we go further, let's define two terms we'll use constantly:

Print production

> The whole journey of turning a file into a finished printed object — color setup, file preparation, checking, arranging pages, printing, and finishing (cutting, folding, binding).

Print-ready PDF

> A single file packaged so that a printing company's equipment can reproduce it correctly with no guesswork — the right colors, the right page sizes, all fonts and images embedded.

Who this guide is for

You don't need a background in graphic arts, physics, or color science. You *do* need curiosity and patience, because print has decades of accumulated conventions that look arbitrary until you understand the reason behind them. By the end you'll be able to reason about why a customer's "perfect" file looks dull when printed, why a logo's blue shifts to purple, why a thin line vanishes, and why a file passes one printer's checks but fails another's. You'll be able to talk to print operators in their own language and make sound technical decisions in software.

Key takeaway: The goal of this guide is a working mental model of the entire path — **from light hitting your eye, to a validated print-ready PDF, to ink landing on a sheet** — so you can build and debug print software with confidence.

What you'll learn — the 7 pillars and 17 sections

The 17 sections group into **7 pillars**. Each pillar builds on the one before it. The whole structure flows in one direction: **everything moves toward a clean print-ready PDF that survives preflight, the RIP, and the press.**

A quick definition you'll need to follow that sentence:

Preflight

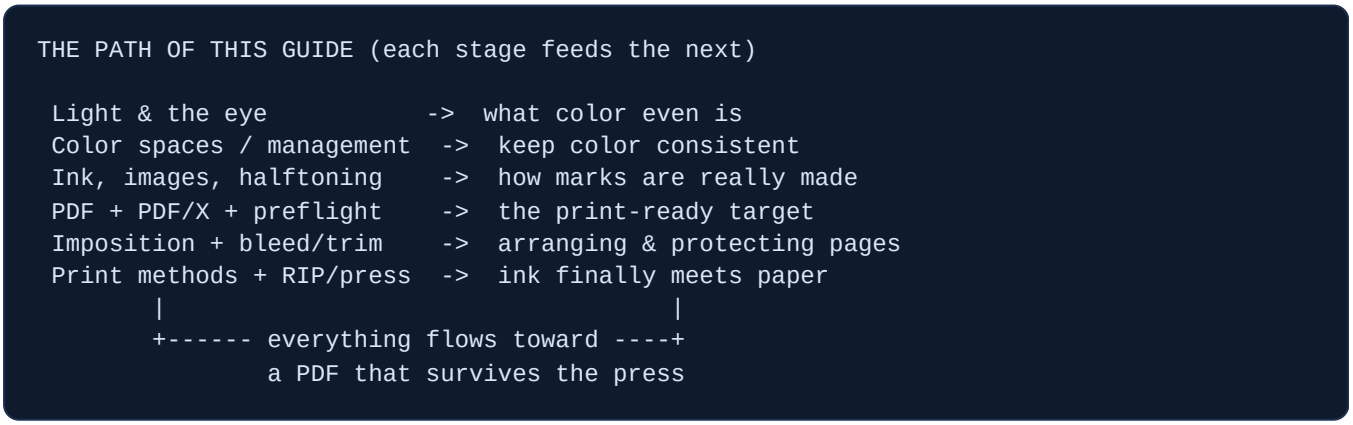
> An automated pre-flight check (the name borrows from a pilot's checklist) that inspects a file for problems before printing — missing fonts, wrong colors, low-resolution images.

RIP (Raster Image Processor)

> The software/hardware that translates your PDF into the exact pattern of tiny dots the printing device actually lays down. It is the bridge between "a file" and "a machine."

Pillar	Sections	What it gives you
1. Color foundations	1–2: How Color Works; Color Spaces & Additive vs Subtractive	Why color exists at all, and the difference between screen color and ink color.
2. Color management	3–4: ICC Profiles & the Pipeline; Rendering Intents & Gamut Mapping	How to keep color consistent as a file moves between devices.
3. Ink & image fundamentals	5–6: Spot Colors, Overprint & Black; Raster vs Vector & Resolution	How real ink behaves and how images are actually stored.
4. From tone to dots	7: Halftoning & Screening	The trick that lets a few inks reproduce smooth photographs.
5. The PDF as the print target	8–10: Inside a PDF; PDF/X, Output Intent & Page Boxes; Preflight	What a print file is made of and how to make it bulletproof.
6. Sheet geometry & finishing	11–12: Imposition & Binding; Bleed, Trim & Safe Area	How pages are arranged on paper and cut without errors.
7. The physical press	13–14: Print Methods & Substrates; RIP, Press & Color Measurement	How ink meets paper and how color is verified in the real world.

Notice the shape of the journey. **Color science is the foundation** — almost every later problem traces back to it. Then we manage that color, understand ink and images, learn the dot-making trick that makes printing possible, package everything into a robust PDF, arrange and protect the geometry of the page, and finally watch it print and get measured.



Analogy: Think of building a house. Color science is the foundation and framing — invisible later, but everything rests on it. The PDF sections are the blueprints handed to the builder. Imposition and finishing are how the rooms are laid out and the edges trimmed. The press is the construction crew. Skip the foundation and the prettiest blueprint still produces a crooked house.

How to read it

Read the sections **in order, at least the first time.** Each one assumes the terms defined earlier. If you jump straight to Preflight (Section 10) without color management (Sections 3–4), the error messages won't mean much. Later, as a reference, you can jump to any section directly.

- **First pass:** read straight through for the mental model — don't try to memorize numbers.
- **Second pass:** revisit with your own product in mind. Ask, "How would my SaaS handle this case?"
- **As a reference:** the comparison tables and summaries are designed to be re-scanned quickly when you hit a real bug.

The box conventions

Throughout the guide, colored boxes flag the most useful moments. Learn to spot them:

Key takeaway

> The one idea from a topic that, if you remember nothing else, you should keep.

Best practice

> What experienced print and prepress people actually do — the safe, professional default.

Common mistake

> A real, recurring error (often the cause of reprints and unhappy customers). Watch for these especially when designing software defaults.

Analogy

> A plain everyday comparison to make an abstract idea concrete.

Example

> A worked, real-world print-shop scenario showing the concept in action.

Best practice: When you read a "Common mistake" box, immediately ask yourself: *"Could my software prevent this for the customer automatically?"* The best print SaaS turns hard-won prepress knowledge into safe defaults and friendly warnings, so non-expert users never hit the mistake at all.

Common mistake: Treating print like the screen. The screen makes color with **light**; print makes color with **ink absorbing light**. They are different physical processes with different limits. Almost every "but it looked fine on my monitor" complaint comes from ignoring this — which is exactly why we start with color science.

What to expect

This is a thorough guide, not a cheat sheet. It favors understanding over shortcuts, because in print the shortcuts are what get reprinted at your expense. The numbers and standards quoted (resolution targets, ink limits, profile names, PDF/X versions) are the well-established industry figures — learn them as reliable defaults, while knowing that a specific printing company may hand you their own exact specifications to follow.

Example: A customer uploads a vivid screen-blue business-card design. It looks brilliant in their browser but prints muted. By Section 4 you'll know *why* (that blue is outside the ink's reachable range, the *gamut*); by Section 10 you'll know how preflight could have flagged it; and by Section 14 you'll know how a press operator measures the printed result to prove it matched the intended color. One small problem, touching nearly every pillar — that's how interconnected print really is.

Section summary:

- This guide takes a smart beginner — especially a developer building print/design software — from the physics of color to a press-ready file and the final printed sheet.
- The 17 sections form 7 pillars that build in order; **color science is the foundation and every pillar flows toward a print-ready PDF that survives preflight, RIP, and press.**
- Read it in order the first time, then use it as a reference; the tables and summaries are made for quick re-scanning.
- Watch for five boxes — **Key takeaway, Best practice, Common mistake, Analogy, Example** — and turn each "Common mistake" into a safeguard in your own product.
- Print is not the screen: color is made by ink absorbing light, not by light emitted — the single most important idea in the whole guide.

1. How Color Works: Light & Human Perception

Before you can manage color, calibrate a monitor, or argue with a client about why their logo "looks wrong," you have to understand a surprising truth: **color is not a thing that lives in the world.** Light carries energy, objects bounce some of it back, and your brain invents the experience we call "color." Everything in print production — proofs, color profiles, spot inks, viewing booths — exists to keep that *invented experience* consistent across machines that work in completely different ways. This section builds that foundation from the ground up.

1.1 The Big Idea: Color Is a Perception, Not a Property

Here is the single most important concept in this entire guide:

Key takeaway: Color is **not** a physical property of objects, and it is not even a property of light. Light only has **wavelength** (a measure of its energy). "Color" is a *perceptual construct* — something your eyes and brain build out of that physical input. A wavelength of 700 nm isn't "red"; redness is what your visual system *makes* of it.

This sounds philosophical, but it has a hard, practical consequence for every print shop:

Key takeaway: There is no "true color" floating inside a design file. A color only becomes a real, seeable thing when three ingredients combine: a **light source**, a **substrate** (the paper or material), and an **observer** (a human eye). Change any one of them and the perceived color can change. This is the entire reason color management exists.

Analogy: Color is like a *taste*, not a *molecule*. Sugar molecules are not "sweet" — sweetness is what your tongue and brain make of them. In the same way, 700 nm light is not "red"; redness is what your visual system makes of it. The same physical input can even produce different experiences depending on context.

1.2 Light and the Visible Spectrum

Let's define our terms before going further.

Wavelength

> The physical "length" of a light wave, measured in **nanometres (nm)**, one-billionth of a metre. Different wavelengths carry different amounts of energy.

Visible spectrum

> The narrow band of wavelengths the human eye can detect, roughly **380–740 nm**. (Different sources cite slightly different edges, but ~380–740 nm is the standard teaching figure.)

White light

> A mixture of *all* visible wavelengths at once — sunlight is the classic example.

Each wavelength, on its own, is perceived as a particular hue. Here is the rough map:

```
WAVELENGTH (nm) → PERCEIVED HUE
380   450   490   560   590   620        740
 |-----|-----|-----|-----|-----|----------|
Violet  Blue  Green Yellow Orange   Red
<380 nm = ULTRAVIOLET (invisible)
>740 nm = INFRARED (invisible)
```

So why does a red apple look red? It is **not** that the apple "contains" redness. White light hits the apple; the apple's surface **absorbs** most of the short and medium wavelengths and **reflects** the long (reddish) ones back to your eye. A prism works on the same principle in reverse — it bends (refracts) each wavelength by a slightly different amount, spreading white light back out into its component colors.

Common mistake: Thinking an object "has" a color. It doesn't — it has a **reflectance curve** (a pattern of which wavelengths it absorbs vs. reflects). Change the light shining on it and the reflected mix changes, which is exactly what causes the metamerism headaches we cover below.

1.3 The Eye: Rods and Cones

At the back of your eye is the **retina**, a light-sensitive layer holding two kinds of **photoreceptors** (light-detecting cells):

Receptor	Count	Works in	Job	Peak sensitivity
Rods	~120 million	Low light (night / "scotopic")	Brightness only — *no* color	~500 nm
Cones	~6 million	Medium/bright light (day / "photopic")	Color + fine detail	see below

This is why everything looks gray at night: in dim light only your rods are active, and rods carry no color information at all. Cones are packed densely into the **fovea**, the tiny central pit of the retina responsible for sharp central vision.

Three cones = trichromatic vision

Humans have **three** cone types, each tuned to a different range of wavelengths by a different light-sensitive pigment (an **opsin**):

S cones (Short / loosely "blue")

> Peak ≈ **420–440 nm**

M cones (Medium / loosely "green")

> Peak ≈ **530–545 nm**

L cones (Long / loosely "red")

> Peak ≈ **560–565 nm**

S/M/L is the modern, accurate naming. The old "blue/green/red" shorthand is misleading — notice the L ("red") cone actually peaks in the *yellow-green*, not red.

Key takeaway: Your brain doesn't read individual wavelengths. It reads the **ratio** of how strongly the L, M, and S cones each fired — a single triplet of numbers, the biological "tristimulus." This 3-signal bottleneck is the reason every 3-primary system works: RGB monitors, the CMYK ink approximations of print, and the XYZ math of color science all only need to *fool three receptors*, not rebuild the full physical spectrum.

Analogy: The three cones are a *3-question survey*. Your eye only ever asks incoming light three questions: "How much do you stimulate S? M? L?" Any two completely different light recipes that give the same three answers will look *identical*. That single fact explains both why RGB screens can fool you with just three primaries *and* why metamerism (below) happens.

1.4 Two-Stage Color Vision: Trichromatic + Opponent

Two classic theories were once seen as rivals; modern science says both are correct and describe different stages of the same system.

1. **Stage 1 — Trichromatic theory (Young–Helmholtz):** three cone types capture light. This explains color *matching* — how two different stimuli can look the same.
2. **Stage 2 — Opponent-process theory (Hering, 1878):** further along the visual pathway, the three cone signals are recombined into three **opponent channels**:
   - Red ↔ Green
   - Blue ↔ Yellow
   - Black ↔ White (luminance / brightness)

Each channel can only lean toward *one* pole at a time. That is precisely why "reddish-green" and "bluish-yellow" are sensations that simply don't exist — the wiring forbids them.

Key takeaway: This opponent structure is the conceptual ancestor of perceptually-organized color spaces you'll meet later, especially **CIELAB (L*a*b*)**, where the `a*` axis runs green↔red and the `b*` axis runs blue↔yellow — the opponent channels turned into measurable numbers.

1.5 Tristimulus and the CIE System: Taming Perception with Math

To trade color reliably between people and machines, science needed device-independent numbers. Enter the CIE (the international color-standards body).

Tristimulus values (X, Y, Z)

> Three numbers describing any color by how strongly it would stimulate a *standard* human observer. Think of XYZ as a color's device-independent "address."

Color-matching functions (CMFs: $\bar{x}$, $\bar{y}$, $\bar{z}$)

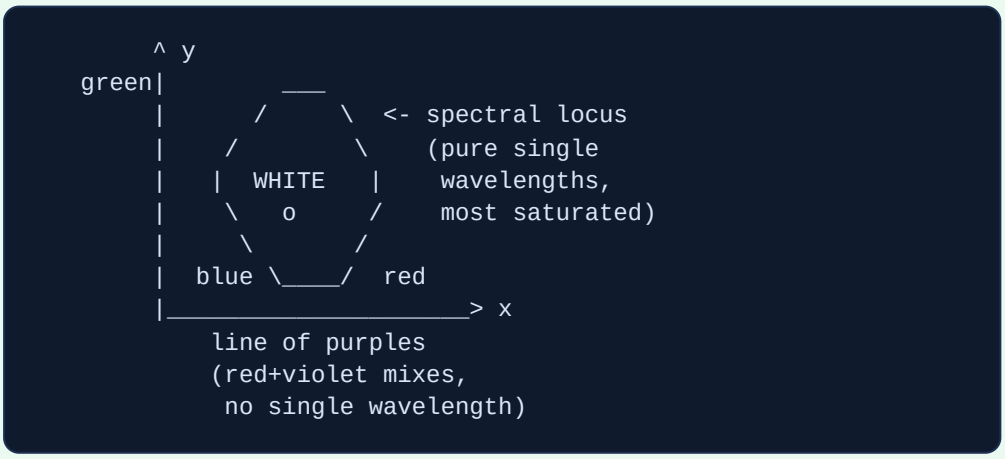
> Weighting curves that encode the average person's cone response. Multiply a light's spectral power against these curves and sum the result — out come X, Y, Z.

CIE XYZ (1931)

> A linear transform of the earlier CIE RGB functions, deliberately chosen so all values stay **positive**. (The RGB functions go negative — meaning some real colors literally cannot be made by *adding* physical primaries.) Importantly, **Y was defined to equal luminance**, i.e. perceived brightness.

The chromaticity diagram (the famous horseshoe)

If you strip brightness out of XYZ by normalizing — `x = X/(X+Y+Z)`, `y = Y/(X+Y+Z)` — you can plot every possible color on a 2D map. The result is the iconic horseshoe (or "tongue") shape:



- **Curved outer edge = spectral locus:** the pure single-wavelength (monochromatic) colors — the most saturated possible.

- **Straight bottom edge = line of purples:** mixtures of red + violet that *no* single wavelength can produce.
- **Center = white / neutral.** Saturation grows as you move outward; hue changes as you travel around the edge.
- **Gamut:** a device's reproducible range plots as a triangle or polygon *inside* the horseshoe. At a glance you can see which real colors a given printer or monitor simply **cannot** reach.

1.6 Whose Eyes Are "Standard"? The Standard Observer

The CMFs above describe an *average* person, called a **Standard Observer**. There are two, and the difference matters in a shop:

Observer	Field of view	Use for	Notes
CIE 1931 2°	~2° (small foveal patch)	Fields ~1–4°: small swatches, patches	Default of most colorimeters & QC tools
CIE 1964 10°	~10° (larger area)	Fields >4°: walls, big solids	Often considered more accurate; recommended for spectrophotometers

> **Best practice:** Pick **one** observer (commonly 2°) and use it across every measurement and spec. Mixing 2° and 10° numbers makes colors that "should match" disagree on paper. Always state the observer alongside any Lab or XYZ value — e.g. "Lab under D50/2°."

1.7 Illuminant, Color Temperature & White Point

Because color depends on the light, you must define *which* light.

Color temperature (Kelvin, K)

> A description of a "white" light's spectral character. Counterintuitively, **lower K = warmer/yellower**, **higher K = cooler/bluer**.

White point / reference white

> The color the system treats as neutral white. Every other color is judged relative to it.

White point	Temperature	Standard for	Appearance
D50	≈ 5000 K (≈5003 K)	**Graphic arts & printing** (mandated by ISO 3664 for viewing prints/proofs)	Slightly yellower
D65	≈ 6500 K	Monitors, web, video, sRGB (average noon daylight)	Cooler / bluer

The print industry standardizes critical color judging with **ISO 3664 viewing booths**: a light source approximating **D50** at roughly **2000 lux** at the viewing surface for critical proof evaluation (a lower ~500 lux condition exists for general viewing). Brands like GTI and JUST-Normlicht build these booths so everyone judges color under identical light.

> **Example:** A proof signed off in the shop under a D50 booth can be rejected by the client back at their store under fluorescent lights — they are literally seeing a different reflected spectrum. The print didn't change; the *light* did.

1.8 Metamerism: The Print Shop's Recurring Nightmare

Metamerism is when two samples that have *different* spectral reflectance curves nonetheless look **identical under one light**, yet visibly **different under another**. The match was an accident of one particular illuminant, not a genuine spectral match.

Metameric match

> The lucky agreement under a specific light.

Metameric failure

> When changing the light (or the observer) breaks that match.

Three flavors to know:

- **Illuminant metamerism:** match under D50, mismatch under store LED/fluorescent. (Most common in print.)
- **Observer metamerism:** two people with slightly different cone sensitivities disagree on whether two samples match.
- **Geometric metamerism:** the match changes with viewing or lighting angle — common on textured, metallic, or special surfaces.

Key takeaway: A true **spectral match** (identical reflectance curves) holds under *every* light and is the gold standard. A metameric match is fragile — it can collapse the moment the lighting changes.

1.9 Putting It to Work: Mistakes and Best Practices

Common mistake: Approving color under the wrong light. Judging a proof under office fluorescents or window daylight instead of a D50 ISO-3664 booth means the client later sees a "color shift" that is really a *lighting* mismatch, not a printing error.

Common mistake: Trusting an uncalibrated monitor at the wrong white point. Soft-proofing on a D65 sRGB screen and expecting it to equal a D50 paper proof — they're built around different whites. Worse, your eye *adapts* so the screen still *looks* white either way, hiding the mismatch.

Common mistake: Matching colors by eye across different inks/substrates and ignoring metamerism. A spot color matched on coated stock under shop light can fail on uncoated stock under the customer's lighting because the reflectance curves differ.

Best practice: Standardize the viewing condition *first* — a D50 / ~2000 lux ISO-3664 booth for all critical sign-off, and have client approvals happen under that same light.

Best practice: Specify color with measured, device-independent numbers (CIELAB or XYZ, with stated illuminant *and* observer), not "match this printout." This removes ambiguity and lets you verify with a spectrophotometer.

Best practice: For brand-critical colors, prefer spectral matches over visual/metameric ones, and check candidate matches under at least two illuminants (e.g. D50 and a store-typical light) to catch metameric failure *before* production.

Section summary:

- **Color is a perception, not a property.** Light has only wavelength; your brain assigns the color. A color is only real when a light source + substrate + observer combine — which is why color management exists.
- **Vision is trichromatic then opponent.** Three cone types (S/M/L) reduce the full spectrum to a 3-number "tristimulus," and downstream opponent channels (red–green, blue–yellow, black–white) shape how we organize color.
- **The CIE system turns perception into math.** XYZ tristimulus values give a color a device-independent address; the xy chromaticity horseshoe lets you see any device's gamut and its limits at a glance.
- **Light defines color.** Print standardizes on the **D50** white point (~5000 K) under ISO 3664 (~2000 lux); monitors use **D65** (~6500 K). Always state the standard observer (commonly 2°) with any measurement.
- **Metamerism is the practical trap:** samples that match under one light can mismatch under another. Standardize viewing light, specify measured Lab/XYZ values, and prefer spectral over metameric matches.

2. Color Spaces & Additive vs Subtractive Color

Before you can control color in print, you need to understand one surprising fact: a **screen and a printed page make color in physically opposite ways**. A glowing phone screen and a printed flyer are not just "the same colors on different surfaces" — they are built on two different laws of physics. This section explains those two laws, then walks through the main **color spaces** (organized systems for describing color with numbers) you will meet in a print shop: RGB, CMYK, Lab, Grayscale, and HSL/HSV.

2.1 The two opposite ways to make color

Let's define the two core ideas in plain words first.

Additive color (RGB)

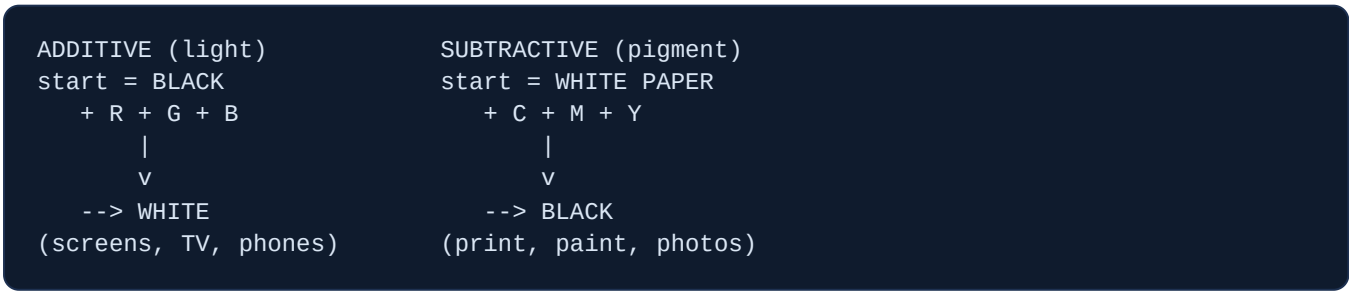
> Making color by **mixing emitted light**. You start from **black** (no light at all) and *add* red, green, and blue light. The more light you add, the brighter it gets, heading toward **white**. This is how every screen works — each pixel emits its own light.

Subtractive color (CMYK)

> Making color by **mixing pigment that absorbs (subtracts) light**. You start from **white paper** (which reflects nearly all light) and *add* ink, which absorbs certain wavelengths and removes them. The more ink you add, the darker it gets, heading toward **black**. You see only the light the pigment did *not* absorb, bounced back off the paper.

Here is the heart of it: **a screen makes its own light; paper only borrows light from the room and throws some of it back.** A screen can fire a pure, intense beam of red straight into your eye. Ink can only *fail to absorb* red light that happens to be in the room. You cannot out-shine a light bulb with a piece of paper. This is the root reason print always looks a little duller than a screen, and why the two can never match perfectly.

Analogy: RGB is like shining three colored flashlights — red, green, and blue — onto a black wall in a dark room. Overlap all three and you get white. CMYK is like stacking colored gel filters over a bright white window: each filter removes some light, and stacking them all blocks nearly everything (black). One builds *up from dark by adding light*; the other works *down from bright by subtracting it*.



The mixing rules are mirror images

When you mix two **additive** primaries, you get a **subtractive** primary — and vice versa. This is not a coincidence; the two systems are exact opposites.

Additive mix (light)	Result	Subtractive mix (ink)	Result
Red + Green	Yellow	Cyan + Magenta	Blue
Green + Blue	Cyan	Magenta + Yellow	Red
Blue + Red	Magenta	Cyan + Yellow	Green
R + G + B	White	C + M + Y	(theoretically) Black

Notice that **Cyan, Magenta, and Yellow are the exact complements (opposites) of Red, Green, and Blue.** Each ink is really a dial that controls one RGB light:

- **Cyan ink** absorbs *red* light, reflecting green + blue.
- **Magenta ink** absorbs *green* light, reflecting red + blue.
- **Yellow ink** absorbs *blue* light, reflecting red + green.

So a CMY ink set is just a way of subtracting RGB lights one at a time. That is exactly why the two systems are mirror images of each other.

> **Key takeaway:** Screens emit light and build color up from black by *adding* RGB; ink absorbs light and builds color down from white paper by *subtracting* with CMY. They are physically opposite, so a print will never glow like a screen.

Why CMYK adds K (black)

In theory, 100% Cyan + 100% Magenta + 100% Yellow should make black. In reality, real pigments are impure, so all three together make a muddy dark brown — never a true, deep black. So printers add a fourth, dedicated **K (Key/black)** ink. The "K" stands for **Key** (the key plate that holds the fine detail and registration), not "blacK."

Benefits of having a separate K ink: crisp deep blacks, razor-sharp black text, **less total ink** used, faster drying, and lower cost (one ink instead of three).

2.2 RGB — additive, for screens

- **Components:** Red, Green, Blue channels. The standard is **8 bits per channel** = 256 levels each (values 0–255).
- **Total colors:** 256 × 256 × 256 = **16,777,216 colors** ("16.7 million," or "True Color"). Because that's 8 bits × 3 channels, it's called **24-bit color**.

- **Hex notation:** six hexadecimal digits, two per channel, each running `00` – `FF` (0–255). For example `#FF0000` = pure red, `#FFFFFF` = white, `#000000` = black. Add a fourth pair for transparency (alpha) and you get 8 hex digits / 32-bit RGBA.
- **Strength:** a huge, bright gamut; native to every digital display; intuitive for anything that glows.
- **Weakness: device-dependent** — the same RGB numbers look different on every monitor unless you use color management. And it is not directly printable.
- **Use when:** designing for web, screen, video, UI, and digital photos — and as your editing space *before* a controlled conversion to CMYK.

RGB working spaces matter too: even within RGB, the size of the color range differs.

RGB space	Size	Best for
sRGB	Smallest — the default for web, browsers, OS, most monitors, social media	Safe universal web export
Adobe RGB (1998)	~35% larger than sRGB; richer cyans/greens	Most pro print work — holds colors CMYK can reach that sRGB would clip
ProPhoto RGB	Massive — covers ~90% of real-world visible colors	Fine-art / high-end editing (use 16-bit to avoid banding); risky for delivery

Best practice: Edit in a wide RGB space (Adobe RGB or ProPhoto, 16-bit) to keep maximum color, then export sRGB for web, Adobe RGB for most print, and let the print lab's ICC profile drive the final CMYK conversion.

2.3 CMYK — subtractive, for ink

- **Components:** Cyan, Magenta, Yellow, Key/black — each expressed as a percentage of ink coverage (0–100%).
- **Strength:** it is the actual model a press prints in; you control real ink coverage, and K gives you clean text and rich black.
- **Weakness:** its gamut is **roughly half the size of RGB**. It simply cannot reproduce bright neons, electric blues, vivid greens and oranges, or glowing reds. It is also dependent on the device, paper, and press.
- **Use when:** preparing final files for offset or digital printing. Always do (and check) the CMYK conversion before sending to press, using that press's ICC profile.

Common mistake: Setting small black body text as "rich black" (C+M+Y+K together) instead of **100% K only**. When the four plates don't align perfectly on press, you get colored fringing and registration halos around the letters. Pros set body text to K-only and reserve a controlled rich-black recipe for large solid black areas.

2.4 Lab / CIELAB — the device-independent reference

Full name: CIE 1976 L*a*b*, standardized as **ISO/CIE 11664-4:2019** and defined by the CIE (the international body for color science). Its purpose is unique: it describes what a color **looks like** to a human, not how a specific screen or ink would make it.

It has three axes:

L* (Lightness)

> 0 = black, 100 = white.

a*

> green (−) ← → red (+).

b*

> blue (−) ← → yellow (+).

A neutral gray sits dead center, where a* = b* = 0. Lab is **device-independent**: it's tied to the CIE "standard observer" (an average of human color-matching data), so it acts as a universal lingua franca. It is also **biologically modeled** — it mirrors human "opponent-process" vision, where the eye feeds a red-vs-green channel (a*), a blue-vs-yellow channel (b*), and a brightness channel (L*). It is designed to be **perceptually uniform**: an equal numeric change roughly equals an equal change in what your eye perceives.

Role in print: Lab is the hub that ICC color-management profiles translate through (RGB → Lab → CMYK). The standard color-difference measure, **Delta E**, is calculated in Lab. And spot colors (Pantone) are specified with Lab values so they can be reproduced anywhere.

> **Analogy:** RGB and CMYK are two "local languages," each tied to a specific device. Lab is the neutral exchange currency every device converts through, so "this exact red" means the same thing on any screen or any press in the world.

2.5 Grayscale — a single channel

Grayscale uses one channel of luminance. At 8-bit it gives **256 shades of gray** (0 = black, 255 = white). In print, a grayscale image can be printed with **K ink only** — cheaper, more consistent, and impossible to misalign — or converted to a neutral CMYK build. Use it for black-and-white photos, single-color (K) jobs, and line art.

2.6 HSL / HSV — human-friendly remaps of RGB

HSL and HSV are **not new gamuts** — they describe the exact same RGB colors using friendlier, more human coordinates (a cylinder instead of three light values). Humans don't naturally think in "how much red + green + blue light," so these models let you pick a color the way a person thinks.

Hue

> An angle, 0–360°, around the color wheel (red → yellow → green → cyan → blue → magenta).

Saturation

> Distance from the gray center axis — the color's purity or intensity.

The difference is the brightness axis:

- **HSL — Lightness:** 0% = black, 100% = white, and **50% = the most vivid color**.
- **HSV/HSB — Value:** 0% = black, **100% = the most vivid color** (Value alone never reaches white).

Use when: HSL is common in CSS, web, and UI design tools (like Figma); HSV is favored in computer vision, image processing, and light/LED controls, and in color pickers.

2.7 Why you can't reproduce every RGB color in CMYK

Because the CMYK gamut is only **about half** the size of the RGB gamut, bright neons, electric blues, vivid greens and oranges, and glowing reds have **no ink combination** that matches a backlit screen pixel. They are "out of gamut." When you convert, these colors get **remapped** into the printable range — coming out duller, muted, or shifted in hue. This is the number-one cause of "it looked great on screen but printed flat."

The conversion is steered by **rendering intents**:

- **Perceptual** — gently compresses the whole gamut to keep color relationships smooth. Best for photos.
- **Relative Colorimetric** — keeps in-gamut colors exact and clips only the out-of-gamut ones to the nearest printable color. Best for logos and most print — pair it with **Black Point Compensation** so blacks stay rich instead of turning dark gray.
- (Saturation and Absolute Colorimetric exist for specialized cases.)

> **Example:** A client designs a flyer with a glowing electric blue (boosted `#0000FF`). On press it prints as a flat navy-purple, because that blue sits far outside CMYK's gamut. Soft-proofing in Lab/CMYK first would have shown the dull result and let them choose a printable blue — or specify a Pantone spot ink.

Delta E (ΔE) — measuring color difference

Delta E is the standard number for "how different are these two colors," measured in Lab. The modern formula is **ΔE2000 (ΔE00)**. A change of **ΔE ≈ 1.0 is the Just Noticeable Difference** — the smallest gap a human eye can detect.

ΔE2000 value	What it means
< 1	Imperceptible
1 – 2	Barely noticeable
2 – 3.5	Noticeable
> 3.5	Clearly different

The **print industry target is ΔE2000 < 2** (acceptable). Consumer goods tolerate around 3.0–4.0; a well-calibrated display should average ΔE under 1 with a max under 3.

Example: A brand's exact corporate red is specified as a Pantone spot color with Lab values. A press in Tokyo and a press in London both hit it within ΔE < 2 — even though their underlying CMYK builds differ — because Lab is device-independent.

Common mistake: Designing in RGB, trusting bright neon screen colors, and sending the RGB file straight to a CMYK press. The RIP converts it blindly, often badly, and your vivid colors come out muted. Convert and soft-proof yourself with the correct ICC profile and rendering intent — and don't forget Black Point Compensation, or your blacks turn dark gray.

Best practice: Use K-only for black text and neutral grays, reserve rich black / CMY builds for large solids (staying under the press's total ink limit), and specify critical brand colors as Pantone/spot with Lab values — then verify with ΔE2000 (< 2) so color is measured, not eyeballed.

Section summary:

- **Additive (RGB)** mixes emitted light up from black toward white; **subtractive (CMYK)** mixes pigment down from white paper toward black — they are physically opposite, so print never glows like a screen.
- RGB is 8-bit-per-channel, 16.7 million colors, hex-coded, device-dependent, and for screens; CMYK is ink percentages with a separate K (Key) plate for clean blacks and is for press.
- **Lab/CIELAB** (L* lightness, a* green–red, b* blue–yellow) is the device-independent, perceptually uniform reference that color management and Delta E rely on.
- CMYK's gamut is ~half of RGB's, so bright/neon colors are out of gamut and print duller; rendering intents and soft-proofing control how that loss happens.
- Edit wide in RGB, convert to the press's CMYK profile at the end, use K-only for text, and verify brand colors with Pantone + Lab and ΔE2000 < 2.

3. Color Management: ICC Profiles & the Pipeline

In the last section you saw that a screen and a printing press do not speak the same color language: a glowing screen mixes light (RGB), a press lays down ink (CMYK), and each device can only reach a limited **gamut** — its range of reproducible colors. So how does a vivid photo on your monitor become the same-looking color on paper? The answer is **color management**: a system of files, math, and standards that translates color faithfully between devices. This section unpacks the whole machine.

> **Key takeaway:** Color management exists because the same color *numbers* mean different things on different devices. Its job is to keep the *appearance* of a color consistent as it travels from camera, to screen, to press.

3.1 What an ICC Profile Is

Start with the single most important fact in all of color management:

> **Key takeaway:** RGB and CMYK numbers are *meaningless on their own*. "R255 G0 B0" just means "max red channel." It does not say *which* red. The profile is what tells you which red.

An **ICC profile** is a small standardized data file that *describes* how one specific device reproduces or responds to color. "ICC" stands for the **International Color Consortium**, the industry group that defined the file format (the original v2 spec dates to 1995; the current version is v4). A profile is not a setting you flip — it is a **translation dictionary** that records two things about a device:

- **The device's gamut boundary** — the outer edge of every color it can capture, show, or print.
- **The math and lookup tables** needed to convert that device's color numbers to and from a neutral, device-independent reference.

Profiles use the file extension `.icc` (or `.icm` on Windows). They get **embedded inside image files** (PSD, TIFF, JPEG, PDF) so the color numbers travel with their meaning, or they get installed at the operating-system level for a monitor.

> **Analogy:** Think of a color number as the amount written on a banknote — "100." On its own, "100" is ambiguous: 100 US dollars and 100 Japanese yen are very different values. The profile is the currency label stamped on the note (USD vs JPY) that finally tells you what the number is worth.

> **Example:** Open the exact same file — say a logo at R0 G128 B0 — tag it with sRGB and it looks like one green; re-tag the very same pixels as Adobe RGB and the green visibly shifts. Same numbers, different profile, different actual color.

3.2 The Profile Connection Space (PCS): the universal hub

If every device only knew how to talk to every other device directly, the system would explode in complexity. Color management avoids this with a shared middle language called the **Profile Connection Space (PCS)** — a device-*independent* hub that every conversion passes through.

The PCS is always one of two standard color spaces defined by the CIE (the international body for color science):

CIELAB (Lab)

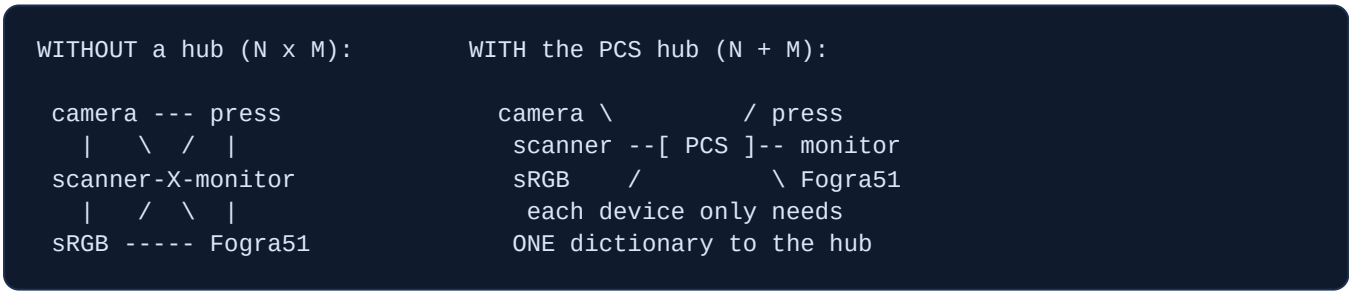
> A perceptual color space modeled on human vision, where distances roughly match how different colors *look* to us. Often preferred for the PCS because this perceptual uniformity makes the interpolation inside color lookup tables more accurate.

CIEXYZ (XYZ)

> A colorimetric space tied directly to how the eye's three cone types respond. Convenient for displays, whose RGB↔XYZ relationship is close to simple linear math.

Lab and XYZ convert to each other by a fixed, well-defined formula, so the two are interchangeable. Importantly, the ICC PCS is anchored to a fixed reference white called **D50** (a standard daylight white near 5000 K) — note that this is D50, *not* the D65 used for general screen work.

Why does a hub matter so much? Math. With **N** source spaces and **M** output devices, a hub means you only need **N + M** profiles to connect anything to anything — each profile just maps its own device to and from the PCS. Without a hub you would need **N × M** direct conversions, and every new device would multiply the work.



Analogy: The PCS is the universal interpreter at the UN. Rather than teaching every delegate every other delegate's language, everyone translates to and from one shared language. Each delegate needs only one dictionary, not dozens.

Key takeaway: The PCS decouples input, display, and output profiles so each can be built independently — that is the entire reason a camera profile from one vendor and a press profile from another "just work" together.

3.3 The full pipeline: Source → PCS → Destination

Now we can trace a color's actual journey. The engine that performs the math is the **CMM** — the **Color Management Module** (also called the Color *Matching* Module). The CMM is the translator that reads the dictionaries and does the conversion.

1. You start with **source values** — say an sRGB photo — tagged with its **source profile**.
2. The CMM uses the source profile to convert those numbers **into the PCS** (Lab/XYZ).
3. Inside the PCS, any needed Lab↔XYZ conversion and **gamut mapping / rendering-intent** decisions happen (covered in §3.8).
4. The CMM then uses the **destination profile** — e.g. a CMYK press profile — to convert **from the PCS into the destination device's native numbers**.

The result: the *same perceived color*, now expressed in the destination's own values.

```
sRGB photo        CMM (Adobe ACE,        CMYK press
+ source   --->   ColorSync, LittleCMS)  + destination
profile           |   converts via   |   profile
                  v                  v
       [ source->PCS ]  [ Lab/XYZ ]  [ PCS->destination ]
       same APPEARANCE preserved, NEW device numbers out
```

Common CMMs include **Adobe ACE** (used in Photoshop, InDesign, Illustrator), **Apple ColorSync**, Microsoft ICM/WCS, and the open-source **LittleCMS**. Different CMMs can produce tiny differences, but profiles themselves are portable across them.

Key takeaway: Remember the trio — **profile = the dictionary, PCS = the shared language, CMM = the translator.**

3.4 The profile classes (by device role)

Profiles come in types matching what role a device plays:

Input profiles

> Characterize capture devices — scanners and cameras. They map device RGB → PCS.

Display profiles

> Characterize monitors and projectors. The OS uses them so your screen shows correct color. Built by calibrating and profiling the monitor.

Output profiles

> Characterize printers, proofers, and presses — the CMYK profiles like SWOP, GRACoL, and Fogra. The class print shops talk about most.

Working-space (color-space) profiles

> Abstract editing containers (sRGB, Adobe RGB, ProPhoto) that are *not* tied to any physical device. Covered next.

DeviceLink profiles

> Bake a specific source+destination pair (and one rendering intent) into a single direct transform. Used in production RIPs because they preserve channel structure — e.g. keeping pure-black text as **K-only** instead of letting it become rich black — and give higher fidelity since both gamuts are known at build time.

Abstract profiles

> Apply a subjective creative edit purely within the PCS. Rare in everyday work.

3.5 Working spaces: your editing containers

Working space	Gamut size	Best used for	Watch out for
sRGB	Smallest common gamut	Web, social media, most monitors, consumer/office printing	Clips vivid cyans/greens a press could actually print
Adobe RGB (1998)	~35% larger than sRGB	Print-bound photography; reaches CMYK cyans/greens sRGB clips	Looks dull if delivered to an sRGB-only system without converting
ProPhoto RGB	Enormous (~90% of visible color; ~13% of its coordinates are "imaginary" colors outside human vision)	High-bit-depth editing container that avoids clipping	**Must be 16-bit** (8-bit causes banding); too wide for any real printer/monitor; convert down before delivery

Best practice: Edit in a space at least as big as your output, at 16-bit, then *convert down* to the output space only at the very end. Never hand ProPhoto or Adobe RGB to a system expecting sRGB — colors go dull and wrong.

3.6 Calibration vs Characterization — two separate steps, in order

People mix these up constantly, but they are different jobs done in sequence:

Calibration

> Physically adjusting a device into a known, stable, repeatable state — e.g. setting a monitor's brightness, white point, and gamma, or setting a press to its target ink densities and dot gain. Calibration **changes the device's behavior**, stored as device settings or correction curves (for monitors, often loaded into the graphics card's LUT).

Characterization (= profiling)

> Measuring how the device reproduces color *in its calibrated state* and recording that as an ICC profile. Profiling does **not** change the device — it only describes it.

Common mistake: Profiling before calibrating, or recalibrating and keeping the old profile. A profile is only valid for the exact calibrated state it was made in — recalibrating *invalidates* the old profile. Always **calibrate first, then profile.**

The tools: a **colorimeter or spectrophotometer** (X-Rite i1Display/i1Pro, Datacolor Spyder, Calibrite) reads color patches; software (DisplayCAL, i1Profiler) builds the profile from those readings.

Canonical display calibration targets

- **White point: D65 (~6500 K)** — the common default for general/screen work. (Some print houses use D50/5000 K to match a print viewing booth, but D65 is the default.)
- **Gamma: 2.2** — the standard SDR tone response (sRGB/Rec.709 territory).
- **Luminance: ~120 cd/m² typical** — use **80–120 cd/m²** for print matching (dimmer, to match reflective paper under booth light) and **120–160 cd/m²** for screen-only work. A too-bright screen makes prints look "too dark" — the #1 soft-proof complaint.
- **Recalibrate every 2–4 weeks** — displays drift over time.

Hardware calibration (which adjusts the monitor's *internal* LUT, on displays like EIZO ColorEdge or NEC) preserves tonal precision better than software/GPU-LUT calibration, which can introduce banding by clipping levels.

3.7 Standard output (CMYK) profiles — the print-shop vocabulary

Every major CMYK profile implements **ISO 12647-2**, the offset-printing process standard. Think of ISO 12647 as the law, and the named profiles below as regional "rulebooks" that ship characterization data plus an ICC profile. **TAC** below means *Total Area Coverage* — the maximum combined ink of all four channels allowed.

Profile	Region / use	TAC	Measurement	Notes
US Web Coated (SWOP) v2	North American web-offset (magazines, coated)	~300%	M0	Older standard; still a common default in legacy files
GRACoL 2006 / 2013 (CGATS21 / CRPC6)	North American premium *sheet-fed* coated	300%	GRACoL2013 = M1	The U.S. sheet-fed go-to
Fogra39 (= ISO Coated v2, ECI)	European coated	~330% (300% variant exists)	M0	2006, ISO 12647-2:2004; long-time European default
Fogra51 (= PSO Coated v3)	European coated	~300–330%	M1	2015, ISO 12647-2:2013; current European recommendation, replaces Fogra39
Fogra47 / Fogra52 (PSO Uncoated v3)	European *uncoated* paper	~300% or less	M0 / M1	Uncoated absorbs more ink → lower TAC, more dot gain

PSO = "Process Standard Offset," the ISO 12647-compliant European workflow from ECI/Fogra. The **measurement condition** matters more than beginners expect:

- **M0** — legacy illumination that ignores UV content.
- **M1** — D50 with defined UV content, so it correctly predicts color on papers with **optical brighteners (OBAs)**, the fluorescent whiteners in most modern stock. Modern profiles (GRACoL2013, SWOP2013, Fogra51) use M1.

Best practice: Don't guess the profile — ask the shop which one they want. Rule of thumb: US sheet-fed → GRACoL; US web/magazine → SWOP; Europe → Fogra51 / PSO Coated v3 (Fogra39 only for legacy).

Common mistake: Using a mismatched/legacy profile — e.g. judging an M0 SWOP prediction for a job printed on a bright, OBA-loaded sheet measured under M1 — so the predicted color never matches the real print.

3.8 Soft proofing: simulating the print on a calibrated screen

Soft proofing means using the destination (printer/press) profile to *preview on your calibrated monitor* how a file will actually print — before wasting paper and ink. In Photoshop: *View ▸ Proof Setup* (pick the destination profile) then *Proof Colors* (Ctrl/Cmd+Y).

> **Common mistake:** Soft proofing on an uncalibrated (or far too bright) monitor. Everything downstream is then built on a lie — this is why §3.6 comes first.

A good soft proof *honestly shows gamut compression*: vivid RGB colors CMYK can't hit appear duller on-proof. That's correct, not a bug. **Gamut Warning** (Cmd+Shift+Y) highlights out-of-gamut colors, and *Simulate Paper Color / Black Ink* dims the white and lifts the black to mimic dull paper and weak CMYK black — a realistic, often shocking preview.

Rendering intents — how out-of-gamut colors get handled

Perceptual

> Compresses the *whole* gamut proportionally so color relationships are preserved; everything shifts slightly. Best for photos packed with out-of-gamut colors (saturated landscapes).

Relative Colorimetric

> Keeps in-gamut colors *exact*, clips out-of-gamut to the nearest reproducible color, and remaps source white to paper white. The common print default; best for portraits/skin tones. Usually paired with **Black Point Compensation (BPC)** so deepest shadows map to the printer's darkest black instead of blocking up.

Absolute Colorimetric

> Like relative but does *not* remap white — reproduces paper white literally. Used to simulate one paper stock on another (e.g. proofing newsprint on a brighter proofer).

Saturation

> Maximizes vividness over accuracy; for charts and business graphics, not photos.

> **Key takeaway:** A soft proof is a *prediction*, not a guarantee. For critical jobs, a physical **hard proof / contract proof** (printed on a calibrated proofer to the press standard) is the legally binding color reference between designer and printer.

3.9 The two operations that trip everyone up: Assign vs Convert

> **Analogy:** Back to the banknote. **Convert** = exchanging 100 USD into the equivalent yen — the number changes, but the *value/appearance* stays the same. **Assign** = just stamping "JPY" onto a 100 USD note — same number, but suddenly it means something completely different.

- **Assign Profile** — relabels the file; pixel numbers stay, appearance changes. Correct only for an *untagged* file that needs its true profile attached.
- **Convert to Profile** — recalculates the pixel numbers so the appearance is preserved across spaces. The correct move when sending to a different output space.

Example (intake): A customer's "Save for Web" sRGB business card has a vivid green logo. The shop soft-proofs against GRACoL2013; Gamut Warning lights up the green. They warn the client it will print muted — or switch to a Pantone spot ink. Without color management they'd only discover the dull green after printing 5,000 cards.

Example ("too dark"): A photographer edits on a 300 cd/m² laptop; prints look murky. Cause: screen too bright and never calibrated. Fix: calibrate to ~100–120 cd/m², D65, gamma 2.2, then soft-proof with the lab's paper profile — screen and print finally agree.

Best practice: Edit at 16-bit in an appropriately wide space, *always embed* the profile on save ("Embed Color Profile" checked), and **Convert** (never Assign) to the correct output profile only at delivery — and always on a *copy*, never the master.

Common mistake: Stripping or forgetting to embed the profile on save. The recipient's app then *guesses* the wrong space, and your color is corrupted before anyone even opens it.

Section summary:

- Color numbers are meaningless without an **ICC profile** — the dictionary that records a device's gamut and how it renders color.
- Every conversion routes through the device-independent **PCS** (Lab/XYZ at D50), turning an N×M problem into N+M; the **CMM** is the engine that does *source → PCS → destination*.
- **Calibrate first, then profile**; for monitors aim D65, gamma 2.2, ~120 cd/m², and recalibrate every 2–4 weeks.
- Edit wide and at 16-bit, but **Convert** (not Assign) down to the printer's specified CMYK profile — GRACoL (US sheet-fed), SWOP (US web), or Fogra51/PSO Coated v3 (Europe) — and embed it.
- **Soft-proof** against that profile (with Simulate Paper/Black and the right rendering intent) before printing; for critical work, confirm with a contract hard proof.

4. Rendering Intents & Gamut Mapping

In the last section you learned how color gets translated from one space to another. This section answers the harder question: **what happens when the destination simply cannot make a color the source asked for?** A bright screen orange or an electric-blue logo may have no ink recipe at all. The press *must* substitute something printable — the only choice is *how* to choose the substitute. That choice is called the **rendering intent**, and the family of techniques behind it is **gamut mapping**.

4.1 Gamut and out-of-gamut colors

Gamut

> The full range of colors a device or color space can actually reproduce. Every device has its own gamut "shape" sitting inside 3D color space (like LAB).

Out-of-gamut (OOG) color

> A color the source contains but the destination (the press and its paper) physically cannot make. It **must** be changed to some printable color.

Gamuts come in very different sizes. From largest to smallest:

```
human vision  >  ProPhoto RGB  >  Adobe RGB (1998)  >  sRGB  >  CMYK press
  (biggest)                                                      (smallest)
```

CMYK is the bottleneck. Its gamut is notably smaller than the RGB spaces designers work in, especially for saturated cyans, vivid greens, oranges, deep blues and purples, and bright reds. This is rooted in physics: **a screen emits light (additive RGB), while paper reflects light (subtractive CMYK)**. Neon screen colors — bright orange, electric blue, lime green — have no ink equivalent, so they will always shift on press. This is unavoidable, not a defect.

Analogy: A gamut is like the set of notes an instrument can play. A piano (RGB) reaches notes a kazoo (CMYK) simply cannot. When you arrange a piano piece for kazoo, the impossible notes must be swapped for the nearest playable ones — the art is in choosing those swaps so the tune still sounds right.

Common mistake: Treating Photoshop's gamut warning as an "error." A color being out-of-gamut does *not* mean it's wrong — the warning just flags areas to *inspect*. If the remapped color still looks fine, leave it alone.

4.2 The two underlying strategies: clipping vs. compression

Before the four named intents, understand the two raw mechanics they're built on:

- **Clipping** — In-gamut colors are left untouched. Only OOG colors get "snapped" to the nearest color on the gamut boundary. Maximum accuracy for printable colors, but the most saturated tones can merge together and lose detail.
- **Compression** — The *whole* source gamut is squeezed inward so that OOG colors fit *and* the spacing between all colors is preserved. Costs some overall saturation, but keeps gradations and detail intact.

Analogy (clipping vs. compression): Moving people into a low-ceiling room. **Clipping** = anyone too tall gets their head squashed flat against the ceiling (OOG colors crush to the boundary; everyone short enough stands normally). **Compression** = you shrink *everyone* proportionally so the tallest just fits — nobody is crushed, but everybody is a little smaller.

Key takeaway: The one-line rule of thumb — **few OOG colors → clip (relative colorimetric); many OOG colors → compress (perceptual)**.

4.3 The four rendering intents

An ICC profile can store a separate lookup table for each intent. The intent tells the **CMM** (Color Management Module — the engine that performs the conversion) which strategy to apply.

1. Perceptual (a.k.a. "Photographic" / "Images")

What it does: Compresses the entire source gamut to fit the destination. *All* colors shift — even in-gamut ones — in order to preserve the *relationships* between colors. It maintains smooth gradations and the visual "sense" of the image, sacrificing the absolute accuracy of any individual color. Colors may desaturate or darken slightly, and hues may shift a touch to keep transitions smooth.

Why it exists: If you only clipped OOG colors and left everything else alone, similar saturated tones would collapse together — causing posterization/banding in skies, sunsets, and deep shadows. Compression keeps that detail and gradation.

Use for: photographs, especially images with many saturated/OOG colors (sunsets, vivid flowers, landscapes). It's the safe default for photographic content that throws a lot of gamut warning. Note: **perceptual quality varies by profile** — it depends on the profile maker's compression algorithm, so two profiles can give different perceptual results.

2. Relative Colorimetric (the common general default)

What it does: Clips. It leaves all **in-gamut colors exactly unchanged** and maps only the **out-of-gamut** colors to the **nearest reproducible color** on the gamut boundary.

White-point handling: It maps source white to **destination/paper white**, so paper white prints as "no ink" — the bare substrate. This is the key difference from Absolute.

Risk: several distinct OOG colors can collapse onto the *same* boundary color, losing detail in the most saturated highlights and shadows. Pair it with **Black Point Compensation** (below) to protect shadow detail.

Use for: logos, brand/spot colors, illustrations, photos with few OOG colors, and any job where in-gamut accuracy matters most. It's often the best all-rounder — many use it as their standard photo intent too, with BPC on.

3. Absolute Colorimetric (proofing only)

What it does: Identical to Relative Colorimetric *except* for white-point handling. It does **not** remap source white to paper white — it preserves the source's white/paper color. If the source paper is whiter than the proofing paper, it lays down a tint of ink to *simulate* the target stock's white.

Use ONLY for: hard/cross-rendering **proofing** — simulating one output condition (e.g., an offset litho press meeting an ISO 12647 reference) on a different proofer. **Never for normal photo printing** — it will cast a tint over your whites. ISO 12647-7 proofing ties to this: the proof substrate should match the production stock's white, and absolute intent simulates that production white when stocks differ.

> **Analogy (white-point handling):** Relative colorimetric "rebases" white onto the new paper — *"treat this paper as white."* Absolute keeps the original paper's color and actually prints the difference as a tint — *"show me exactly how that other paper looked, dinginess and all."*

4. Saturation

What it does: Maps saturated source colors to saturated destination colors, maximizing vividness and punch at the expense of hue and lightness accuracy.

Use for: business graphics, charts, infographics, pie graphs, and technical diagrams — where "make it pop and keep colors distinct" beats accuracy. **Rarely used for photos**: skin tones and neutrals go wrong.

4.4 Black Point Compensation (BPC)

What it does: BPC is the black-point analog of the white-point mapping in relative colorimetric. It scales and aligns the source's darkest black to the destination's darkest printable black, so the **full tonal range** of the source maps into the full range of the destination.

Effect: it prevents blocked-up, crushed shadows — the situation where everything darker than the print's max black becomes one flat black blob. It preserves shadow detail.

- BPC applies to the **colorimetric** intents (Relative and Absolute) — it appears there as a checkbox.
- For **Perceptual**, BPC behavior is effectively always built in; the checkbox has no effect.

Best practice: Keep **BPC ON** for relative colorimetric print conversions. Color-management literature describes BPC as the "best possible compromise" for tonal range. Without it, deep shadows clip and lose detail.

4.5 Soft proofing & the gamut warning (Photoshop)

A **soft proof** is an on-screen preview of how a file will look when printed on a specific device.

```
View > Proof Setup > Custom   pick destination ICC profile
                              + rendering intent
                              + (toggle) Simulate Paper Color / Black Ink
View > Proof Colors (Ctrl/Cmd+Y)         preview the print on that device
View > Gamut Warning (Shift+Ctrl/Cmd+Y)  overlay flat color on OOG pixels
```

The **Gamut Warning** overlays a flat color (default gray, configurable in Preferences > Transparency & Gamut) over every pixel that falls *outside* the proof profile's gamut. It is **diagnostic, not corrective** — it shows *where* significant remapping will happen so you can decide whether to hand-edit (for example, desaturate a hot color) before printing. The **"Simulate Paper Color"** toggle previews using absolute-colorimetric white handling, so you see the dull/tinted paper white — often shocking, but realistic.

Common mistake: Over-editing every flagged pixel to chase the gamut warning away → you end up globally desaturating the image for no reason. The warning means "inspect," not "fix."

4.6 Practical chooser — photos vs. logos vs. proofing

Content	Recommended intent	Why
Photos, few OOG colors	**Relative Colorimetric + BPC**	Keeps in-gamut accuracy; nudges only the few OOG colors
Photos, many OOG colors (sunsets, vivid scenes)	**Perceptual**	Compression preserves gradation/detail; avoids posterization
Logos / brand colors / spot colors	**Relative Colorimetric**	Exact in-gamut match to the brand color
Charts, infographics, business graphics	**Saturation**	Maximizes vividness & color separation
Press / hard-proof simulation	**Absolute Colorimetric**	Simulates target paper white + exact press color

Example (vivid sunset → CMYK): The saturated oranges and magentas are far OOG. Relative colorimetric clips them, banding the sky into flat patches. Switch to Perceptual and the whole image desaturates slightly, but the sky keeps its smooth gradient. **Perceptual wins.**

Example (corporate "Pantone-blue" logo): The electric blue is OOG for the press. Perceptual would desaturate the *whole* logo and any in-gamut text. Relative colorimetric leaves everything else pixel-accurate and only nudges the blue to the nearest printable blue. **Relative colorimetric wins.**

Example (newspaper ad proof on a bright inkjet proofer): The client needs to see how dull and yellowish the ad will look on newsprint. Absolute colorimetric lays a faint yellow tint into the "white" areas to simulate the newsprint stock (ISO 12647-7) — accurate expectation-setting.

Best practice: Don't guess by habit. Soft-proof with the *real* destination profile and toggle between Perceptual and Relative Colorimetric on the actual image — whichever looks better on *that* image wins.

Common mistake: Using Absolute Colorimetric for normal printing. It simulates someone else's paper and casts an ugly tint over all your "white" areas. Reserve it strictly for hard-proof press simulation.

Section summary:

- **Gamut** is the range a device can reproduce; CMYK is the bottleneck, so saturated screen colors are often out-of-gamut and *must* change on press — the rendering intent decides how.
- Two mechanics underlie everything: **clipping** (snap OOG colors to the boundary, keep in-gamut colors exact) and **compression** (shrink the whole gamut, preserve relationships). Few OOG → clip; many OOG → compress.
- The four intents: **Perceptual** (photos with lots of OOG), **Relative Colorimetric** (logos, spot colors, accurate all-rounder), **Absolute Colorimetric** (proofing only — preserves source paper white), and **Saturation** (charts and business graphics).
- Keep **Black Point Compensation ON** for colorimetric print conversions to protect shadow detail; it's built into Perceptual already.
- **Soft-proof with the real profile** and treat the gamut warning as "inspect, not fix"; compare Perceptual vs. Relative on the actual image before sending to press.

5. Gamut & Out-of-Gamut Handling (Deep Dive)

You pick a gorgeous electric-blue logo on your screen. It glows. You order 500 business cards. They arrive looking like a tired, dusty navy. Nobody made a mistake at the print shop — the color you chose was simply *impossible to print*. This section explains why that happens, what the industry does about it, and how a print platform like Print-Flow-360 can warn a customer **before** they hit "Order" instead of after.

5.1 What is a gamut?

Gamut

> The complete range of colors that a particular device or color space can capture, display, or reproduce. Think of it as a "menu" of colors — anything not on the menu can't be served.

Color space

> A defined system for describing colors with numbers (for example, RGB or CMYK). Each color space has its own gamut.

Device-independent reference space

> A neutral "map of all colors a human can see," used so we can compare different devices fairly. The two common ones are **CIE 1931 xy** (the famous horseshoe-shaped diagram) and **CIELAB / CIE L*a*b*** (a 3D space).

A gamut is best pictured as a 3D *solid* sitting inside that reference space. The outer horseshoe boundary represents every color the human eye can perceive. Every real device — your phone, your monitor, a printing press — can only reach *part* of that horseshoe. Its gamut is a smaller blob inside.

Analogy — The human-visible horseshoe is the whole box of 120 crayons. Your monitor got a 64-crayon box. The printing press got a 40-crayon box — and a few of *its* crayons aren't even in the monitor's 64. A color is "out of gamut" simply when the crayon you want isn't in the box you're using right now.

Key idea — "Out of gamut" (OOG) is never absolute. A color is out of gamut *for a specific target* — this monitor, or this printer + this paper + this ink. Always ask "out of gamut for *what*?"

5.2 Why CMYK can't print everything your screen shows

This comes down to physics — two completely different ways of making color.

RGB (additive color)

> Used by screens. You start with a black screen and **add** red, green, and blue *light*. All three at full = white. Color comes from *emitted light*, so it can be intensely bright and saturated.

CMYK (subtractive color)

> Used by printing. You start with *white paper* and lay down Cyan, Magenta, Yellow, and blacK ink. Ink **absorbs (subtracts)** some wavelengths and reflects the rest. More ink = darker. Color comes from *reflected light*.

Because ink can only *absorb* light and never *emit* it, two hard limits apply: the brightest a printed color can ever be is limited by the paper's whiteness, and the most saturated it can be is limited by the pigment. Glowing neon greens, vivid cyans, electric blues, hot oranges, and saturated purples are exactly the colors pigments struggle with. So the CMYK solid is a *smaller* blob that mostly nests *inside* RGB.

Common mistake — Believing "CMYK is always smaller than RGB everywhere." Mostly true, but not strictly. Offset CMYK profiles like **FOGRA39** or **GRACoL/G7** are smaller than Adobe RGB overall, yet slightly *exceed* plain **sRGB** in a few spots (very saturated cyans and yellow-oranges). A handful of printable colors actually live outside sRGB.

Some rough, illustrative figures: people often cite RGB as ~16 million addressable shades versus CMYK reproducing maybe ~16,000 — treat those as ballpark, not gospel. On wider RGB spaces: **Adobe RGB** covers about **86.2%** of Pointer's gamut (the range of real-world surface colors) and was deliberately

designed to include more cyan-green for cleaner CMYK conversion; **ProPhoto RGB** reaches ~90% of real-world colors; **sRGB** is similar to Adobe RGB in overall coverage but weaker in the cyan-green region.

5.3 What happens to an out-of-gamut color? Clipping vs. compression

When a color can't be printed, software must decide where to *put* it instead. That decision is controlled by the **rendering intent**.

Rendering intent

> The rule for mapping out-of-gamut colors into the destination gamut. The ICC standard defines four: **Perceptual**, **Relative Colorimetric**, **Absolute Colorimetric**, and **Saturation**.

They fall into two big strategies:

Strategy	Intents	How it works	Risk / best for
Clipping	Relative & Absolute Colorimetric	In-gamut colors stay *exact*; only OOG colors get snapped to the nearest printable boundary color.	Many different OOG colors can collapse onto the same edge color → lost detail (posterization, blocked saturated areas, hue shifts). Best when few colors are OOG (logos, brand colors you want kept exact).
Compression	Perceptual	Squeezes the *entire* source gamut to fit the destination, preserving the *relationships* between colors.	Even in-gamut colors lose a little saturation, but gradients stay smooth and nothing snaps abruptly. Best for photos with *many* OOG colors.

The fourth intent, **Saturation**, maximizes vividness over accuracy — good for charts and business graphics, bad for photos.

Analogy — Clipping is shoving everyone who's too tall against the back wall of a photo — they all line up at the same spot and you lose who was taller. Compression is asking the *whole crowd* to take one small step back so everyone still fits in frame and you can still tell them apart.

Black Point Compensation (BPC)

> An option that maps the source's darkest black to the destination's darkest black so shadows don't get crushed or muddy. Keep it **ON** for most work.

Common mistakes — (1) Using **Absolute** when you wanted **Relative** — Absolute even simulates the source *paper white* (it's for proofing one printer on another), giving an odd tint. (2) Forgetting BPC and getting blocked, muddy shadows. Sensible default for most graphics: **Relative Colorimetric + BPC on**.

Key idea — Conversion by clipping is *irreversible*. Once two distinct colors are snapped to the same boundary color, the difference is gone forever. Never treat RGB→CMYK as something you can undo.

5.4 Seeing the problem early: soft proof & gamut warning

Soft proof

> An on-screen *simulation* of how the print will actually look, viewed through a specific printer + paper ICC profile, before any ink is used. In Photoshop: `View > Proof Setup > Custom`, then toggle **Proof Colors** with `Ctrl/Cmd+Y`.

Gamut warning

> An overlay (Photoshop: `Shift+Ctrl/Cmd+Y`) that paints a flat gray mask over the exact pixels that fall *outside* the destination gamut and will be remapped significantly.

ICC profile

> A standardized file describing a specific device's gamut, so color can be translated accurately between devices.

The difference matters: **Soft Proof shows you the simulated final look** (more useful for judging the real outcome), while **Gamut Warning just flags which pixels are risky**. The Color Picker also shows a small triangle "gamut alarm" with a swatch of the nearest in-gamut substitute you can click to accept.

Common mistakes — Soft-proofing on an *uncalibrated* monitor or with the wrong/generic profile — garbage in, garbage out. And panicking at every gamut warning: a flagged color that still looks fine in the soft proof needs no edit. A warning means "inspect," not "broken."

5.5 Handling it gracefully: suggesting the nearest printable color

The kind move is to warn early, non-blockingly, and offer the closest color that *will* print. "Closest" is measured with a perceptual distance called **Delta E (ΔE)**.

Delta E (ΔE)

> A number for how different two colors look to the human eye. Smaller = closer. A **just-noticeable difference (JND)** is roughly ΔE 1–2; most tools treat ΔE < 2 as "close enough to be indistinguishable."

The ΔE formulas, oldest to best: **CIE76** (plain Euclidean distance in Lab — crude), **CIE94**, **CIEDE2000** (the industry standard, perceptually tuned), **ΔEOK** (Euclidean in Oklab), and **ΔEITP** (for HDR).

The modern web approach — the CSS Color Module 4 default, used by libraries like `colorjs.io` — is smart: instead of naively clipping channels, it works in **OkLCh** (a perceptual hue/chroma/lightness space). It *holds the hue and lightness fixed* and does a binary search reducing **chroma** (saturation) until the color fits, comparing each guess to a channel-clipped version with ΔEOK and stopping when ΔE < 2. (If lightness ≤ 0 → black; ≥ 1 → white.) Chroma-reduction keeps the color recognizably "the same hue, just calmer," which beats naive clipping for most hues; yellows are a known weak spot.

5.6 Bringing it to Print-Flow-360

Here's the honest state of the stack: PF360 has **no documented RGB→CMYK conversion and no preflight step**. The Fabric.js **design studio** (in `designer/`) lets customers pick any RGB/hex color, the storefront brand-color theming is RGB, and the Node **PDF service** (`pdf-service`, port 4000, using `sharp` for images and `PDFKit` for PDFs) outputs RGB PDFs. So a neon hex chosen in the studio sails straight through to a PDF with *no* warning — the customer discovers the dull result only on the doorstep. That's a textbook **silent-lie** bug: the screen promised something the print can't keep.

Worked example — a graceful PF360 gamut warning.

1. Customer picks `#00FF88` (electric mint) for a flyer headline in the Fabric.js studio.
2. On color-pick, check it against a target print profile (a FOGRA-like CMYK gamut for the product's print process). ΔE to its nearest printable color is large → it's OOG.
3. Show an *inline, non-blocking* message in **plain language**: "This bright color may print noticeably duller than it looks on screen." (Per PF360 UX rules: reserve the layout space so nothing jumps, never a raw error, never block.)
4. Render a **live swatch of the nearest printable color** (computed via CIEDE2000 or the OkLCh chroma-reduction method, stopping at ΔE < 2) with a one-click **"Use printable color"** button.
5. **Suggest, never substitute.** If the customer keeps their color, honor it. Silently swapping it would be its own silent-lie bug.
6. If no color is out of gamut, behavior is byte-for-byte unchanged — no warning, no surprises.

Pro tip — The gamut check, the ΔE math, and a true RGB→CMYK transform all belong on the **Node PDF service**, not bolted into the browser studio. It's the heavy-file, server-side layer that already owns rendering — the natural home for a future preflight step that flags OOG colors, low-res images, and missing bleed, strengthening the platform's weak order-to-production spine.

Key takeaways —

- **Gamut** = the colors a device can reproduce; OOG is always relative to a specific target (printer + paper + ink), never absolute.
- CMYK (subtractive ink) is mostly smaller than RGB (additive light) because pigment can't emit — neons, bright greens/blues/oranges are the usual casualties — though a few CMYK cyans/yellow-oranges poke outside sRGB.
- **Clipping** (Relative/Absolute Colorimetric) keeps in-gamut colors exact but snaps OOG colors to the edge, losing detail; **compression** (Perceptual) squeezes everything to keep relationships smooth — best for photo-heavy art. Conversion is irreversible.
- Default to **Relative Colorimetric + Black Point Compensation on** for most graphics; Perceptual for photos with lots of OOG color.
- **Soft proof** simulates the real printed look; **gamut warning** just flags risky pixels. Both need a calibrated monitor and the correct profile, or they lie.
- Suggest the nearest printable color by minimizing **ΔE** (CIEDE2000, or OkLCh chroma-reduction stopping at $\Delta E < 2$ — one JND).
- PF360 today has no RGB→CMYK conversion or preflight. The right fix: a friendly, non-blocking gamut warning in the studio plus a real CMYK/preflight step in the Node PDF service — warn and suggest, never silently substitute or block.

6. Ink on the Page: Spot Colors, Overprint & Black Generation

So far we've talked about *color* in the abstract. This section is about the **physical ink that lands on paper**: where it comes from, how layers of it interact, and why a black box on screen can come out looking gray, soak through the sheet, or smear onto the back of the next page. These are the everyday decisions that separate a clean print job from a costly reprint.

5.1 Spot Colors & Pantone (PMS)

First, two ways a printer can put color on a page.

Process color (CMYK)

> Full-color images are built from tiny **halftone dots** of Cyan, Magenta, Yellow and Key/black. The dots overlap in little flower-like clusters called **rosettes**, and your eye blends them into a continuous color. Zoom in with a loupe and you literally see the dots.

Spot color

> A single, **premixed ink** applied through its own dedicated plate or press unit, laying down one solid, continuous film of ink. The color you see is the real physical ink — not an optical trick of overlapping dots.

Analogy: A spot color is opening a can of *pre-mixed paint* and rolling it on — one exact color, solid and even. Process color is an Impressionist painting seen up close: countless tiny C/M/Y/K dots that only *read* as a color from a distance.

The Pantone Matching System (PMS)

Pantone publishes a standardized library of premixed inks. Each has a number and a published mixing formula (parts of base inks), e.g. **"PMS 185 C"**. The suffix tells you the substrate/finish:

- **C = coated** stock **U = uncoated** stock **M = matte**

The *same* PMS number prints differently on coated vs uncoated paper because the paper drinks ink differently — so you must **always specify the suffix**. Pantone also sells physical swatch guides, so a designer's spec and the printer's output match regardless of who, where, or which press.

Why use spot colors?

1. **Brand consistency.** A logo's exact color must be repeatable across runs, presses, vendors and countries. CMYK drifts a little run-to-run; a premixed ink does not. (Coca-Cola red, Cadbury purple, Tiffany blue 1837 are spot-color brand assets.)
2. **Out-of-gamut colors.** CMYK's color range is smaller than the spot range. Roughly **30%+ of Pantone solid colors cannot be reproduced in CMYK** — vivid oranges, certain greens, deep

blues, soft pastels. A spot ink hits them directly.

3. **Special effects CMYK simply can't do: metallics** (silver/gold), **fluorescents/neons**, and clean pastels. CMYK can't fake the reflective or glowing quality.
4. **Fewer plates on limited-color work.** A 1- or 2-color job (say black + 1 PMS for letterhead) needs only 1–2 plates instead of 4 — cheaper on short, few-color runs.
5. **Cleaner solids.** A big flat area of color is smoother as one solid ink than as overlapping halftone screens.

Example: A retailer's PMS 286 logo is sent as a 4-color CMYK build on a catalog cover and comes out dull and grayish — 286 sits near the edge of the CMYK gamut. Switching it to a true spot plate makes the blue snap and match every other piece of collateral.

Kept as a separation vs simulated in CMYK

- **Kept as a separation** ("named color"): the spot stays on its own plate/channel and the printer loads the real ink. Required for true brand/metallic/fluoro fidelity. Each extra spot = an extra plate, an extra press unit, and extra cost.
- **Simulated / converted to CMYK** ("process simulation"): the spot is approximated with a CMYK build. Cheaper (no extra plate), but for that ~30% of colors it's a visible mismatch — the classic "my logo printed wrong" complaint.
- **Extended-gamut (ECG / fixed-palette) printing** trades extra spots for wider reach with one fixed ink set: **CMYKOG (Hexachrome)** adds orange + green; **7-color CMYKOGV** (+violet) can match **~90% of Pantone solids** without swapping inks per job.
- A **fifth (or 6th/7th) unit** on a press or digital device (e.g. Kodak NexPress 5th imaging unit) carries an extra station for one spot, white, metallic, or clear printed alongside CMYK.

Key takeaway: Keep brand, metallic, fluorescent, and out-of-gamut colors as named **spot separations**; convert purely decorative spots to CMYK to save plates — but always confirm the conversion *visually*, never assume the build matches.

5.2 Overprint vs Knockout

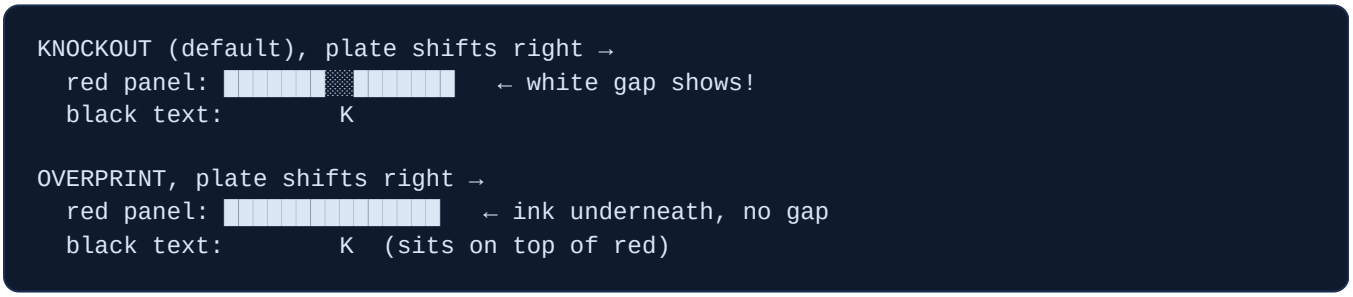
When two colored objects overlap, the software must decide what happens to the ink underneath. There are two behaviors.

Behavior	What happens	Effect on color
Knockout (default for most colors)	The top object "punches a hole" in everything beneath it; the underlying inks are removed so the top color prints on bare paper.	Colors stay pure — no mixing.
Overprint	The top object prints *on top of* the inks below; where they overlap, the inks physically mix/multiply.	Colors blend (e.g. yellow over cyan → green).

Analogy: Overprint is a transparent highlighter laid over existing ink — the colors blend through. Knockout is slapping down an opaque sticker that hides what's underneath — but if the sticker shifts even slightly, you see a gap of bare paper at the edge.

The registration problem overprint solves

Each ink is a separate plate, and presses can't align plates perfectly. Tiny shifts are called **misregistration**. With knockout, a misregistered top object exposes a sliver of **bare white paper at the edge** — an ugly white halo or gap. If you set black (or thin elements) to **overprint**, there's ink underneath at the edges, so a small shift never shows white. That's why **black text and thin rules are conventionally set to overprint**.



Example — the wedding-invite white halos: Fine black serif type was knocked out of a deep red panel; a 0.3 mm press shift left white hairlines around every letter. Fixed by setting the K100 text to **overprint** — now no white gap is possible.

Overprint black — the standard rule, and its limit

- **100% K is opaque enough on small areas** (text under ~60 pt, lines under ~2 pt) that the underlying color won't show through — so overprint it and skip trapping entirely. This is the #1 everyday use of overprint.
- **But on large solid black areas**, 100% K overprinting a colored background is *not* fully opaque — the background "ghosts" through, and plain-K solid over a photo looks weak and washed out. Large solids need **knockout + rich black + trapping**, not naive overprint.

Common mistake: Leaving accidental overprint in the file. A *white* object set to overprint **vanishes on press** (white "ink" is just bare paper, and overprint means "don't knock out"). Always check Output Preview / Separations before sending.

5.3 Rich Black vs Plain Black

Plain / "true" black

> K only (**C0 M0 Y0 K100**). Use for small body text and thin rules. One ink → no registration risk, crisp edges. Looks slightly gray/weak over large areas.

Rich black

> K plus CMY underneath for a deeper, denser black. Use for large fills, backgrounds, and hero panels.

- **Standard neutral rich black recipe: C60 M40 Y40 K100** (= 240% total ink, safely under limits; leans neither warm nor cool).
- A common simple build is **C40 K100** (a slightly cool lean). Variants: **cool/blue black** C60 M0 Y0 K100; **warm black** C0 M60 Y30 K100.

Common mistake: Using **Registration black (C100 M100 Y100 K100 = 400%)** as a design fill. Registration color is *only* for crop and registration marks. As a fill it floods ink → won't dry, sets off, cracks at folds, soaks the sheet.

Common mistake: Rich black (4-color) on small text or fine lines. Four plates must register perfectly, so any shift gives colored fringes around the letters. Small type must be **plain K100, overprinted**.

Example — the ghosting black box: A designer fills a full-bleed background with K100 set to overprint over a photo. On press the photo ghosts through and the marks show — because K100 isn't opaque on large areas. Fixed by switching to rich black (C60 M40 Y40 K100) with knockout.

5.4 Black Generation: GCR & UCR

For any neutral or dark color, the CMYK conversion engine must decide how much to build from C/M/Y and how much from K. Two strategies control this.

UCR — Under Color Removal

> Reduces C, M, Y **only in the dark/neutral shadow areas** (three-quarter tones up to solids) and substitutes black ("short black") for the removed gray. Goal: cut total ink in the heaviest areas to fix drying, trapping, and set-off. Midtones and highlights are untouched.

GCR — Gray Component Replacement

> More general and aggressive: wherever C, M, Y overlap to form a **gray (neutral) component**, that gray is replaced with black ink — **across the whole tonal range**, not just shadows.

Example: Instead of C40 M30 Y70 (which hides a neutral gray component inside it), GCR prints **C10 M0 Y40 K30** for the same visual color — the shared gray was swapped for a single K ink.

Why GCR helps

- **Cheaper:** less colored ink used (K is the cheapest ink).
- **More stable/neutral on press:** grays are controlled by one ink instead of a delicate 3-ink balance, so when ink density drifts you get less color cast.
- **Lower total ink → better drying.**

The full relationship: **GCR = UCR + Black Generation (BG) + UCA.** BG sets how much K to add. **UCA (Under Color Addition)** adds a little CMY *back* into the deepest shadows to restore density that heavy GCR can strip out. GCR is usually offered in levels — **Light / Medium / Heavy / Maximum**: heavy GCR for stability and ink savings (packaging, gravure); lighter GCR keeps richer shadows for photographic work.

5.5 Total Area Coverage (TAC / TIC / Total Ink Limit)

TAC (also called TIC) is simply the **sum of the CMYK dot percentages at any one spot**. 100C + 100M + 100Y + 100K = 400%, the theoretical maximum.

Why limit it? Too much wet ink stacked in one place **won't dry**, causes **set-off** (wet ink transferring onto the back of the next sheet in the stack), smudging, poor wet-on-wet trapping, and on web presses even web breaks.

Analogy: TAC is a stack of wet paint coats. Pile four wet coats in one spot and it never dries and smears the next sheet. UCR/GCR swaps some colored coats for one cheaper black coat so the stack stays thin enough to dry.

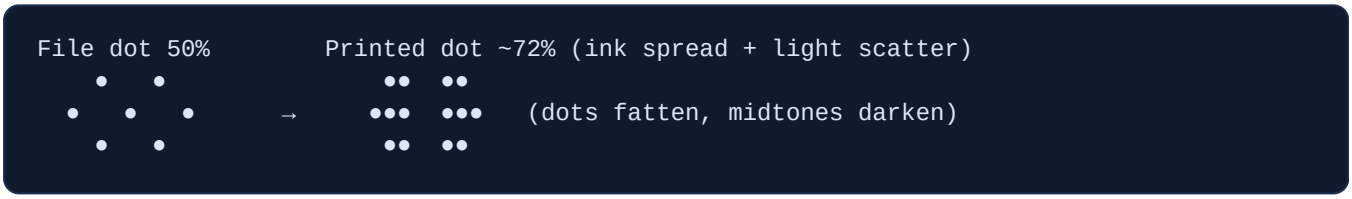
Condition / spec	TAC limit
Newsprint / non-heatset web, uncoated	**240–260%**
SWOP (US web offset publications)	**300%**
Heatset web offset (magazines)	**300–320%**
GRACoL — sheetfed offset, coated (commercial)	**320–340%**
Print-on-demand rule of thumb (IngramSpark etc.)	**≤ 240–300%** (many POD cap at 240%)

The limit is enforced during CMYK conversion (the **Total Ink Limit** in the ICC profile / Photoshop separation settings) and verified in **preflight** (Acrobat Output Preview ink-coverage readout, FlightCheck, etc.). UCR and GCR are the tools that bring TAC down under the ceiling.

5.6 Dot Gain / Tonal Value Increase (TVI)

Dot gain (a.k.a. **TVI, Tone Value Increase**) is the fact that halftone dots print **larger** than specified, so tones come out **darker** than in the file. It has two causes: **mechanical** (ink spreading and absorbing into paper) and **optical** (light scattering at the dot edges).

- It's measured at the **midtone (50%)**, where gain peaks. **TVI = printed % − file %.** File says 50%, prints as 70% → 22% dot gain.
- **Standard ISO 12647-2 values:** about **16% TVI on premium coated (#1) stock**; about **22% on uncoated**. Uncoated and newsprint absorb more, so they gain more; coated stock resists. Flexo and screen printing typically gain still more.
- **Effect if ignored:** midtones plug up, shadows fill in, images look muddy and dark, contrast is lost.



Compensation: apply a **dot-gain / TVI compensation curve** in the output profile (ICC) or RIP. It makes the *plate* lighter by exactly the expected gain so the *printed* result lands on target. Always profile and compensate for the specific press + ink + paper combination.

Best practice: Fix dot gain in **prepress curves**, not by eyeballing and darkening/lightening the file. Set the right TAC and GCR for the substrate in the conversion profile, apply the press's TVI compensation curve, and verify TAC and overprints in preflight before sending.

Best practice: Match the black to the job — plain **K100 overprinted** for small text and lines; **rich black** (e.g. C60 M40 Y40 K100, knockout, trapped) for large solids; **never registration black** as a fill.

Section summary:

- **Spot vs process:** spot = one premixed solid ink (brand/metallic/fluoro/out-of-gamut fidelity, fewer plates); process = CMYK halftone dots. ~30%+ of Pantone solids can't be matched in CMYK; extended gamut (CMYKOGV) reaches ~90%.
- **Overprint vs knockout:** knockout punches a hole (pure color but risks white gaps on misregistration); overprint lets inks mix and hides registration shifts — which is why small K100 text is overprinted.
- **Black choice:** plain K100 (overprint) for small text/lines; rich black like C60 M40 Y40 K100 (knockout) for large solids; never 400% registration black as a fill.
- **Ink control:** UCR/GCR swap a color gray for cheaper, more stable black and keep **TAC** under the substrate's ceiling (~240% uncoated/POD up to ~340% coated sheetfed) so ink dries and doesn't set off.
- **Dot gain (TVI ~16% coated / ~22% uncoated)** darkens midtones; compensate with a profile/RIP curve tuned to the exact press, ink, and paper.

7. Raster vs Vector, Resolution & Image Quality

Every image that goes to a printing press is, at heart, one of two things: a grid of colored dots, or a set of math instructions. Knowing which is which — and how each behaves when you scale, edit, and output it — is the single biggest skill that separates print files that come out crisp from files that come out blurry. This section starts from "what is a pixel" and builds all the way to the layer math your designer tool quietly performs before it hands a file to the printer.

6.1 Two ways to describe a picture: raster and vector

Before any definitions, the core split:

Raster (also called bitmap)

> A picture stored as a **grid of pixels**. A *pixel* ("picture element") is one tiny square that holds a single color value. The whole image is just rows and columns of these squares. Because the number of pixels is fixed the moment the image is created, raster is **resolution-dependent** — the detail is baked in and can never be increased later. Common formats: JPEG, PNG, TIFF, PSD, GIF, BMP, RAW. Made by cameras, scanners, and Photoshop. Best for **photographs, continuous-tone images, and complex gradients** — anything with subtle color change from one pixel to the next.

Vector

> A picture stored as **math** — points, lines, curves, and fills — not pixels. There are no squares; there are instructions like "draw a circle of radius r, fill it blue." Because it is math, it is **resolution-independent**: at output time it is re-drawn (rasterized) at whatever resolution the device needs, so it stays razor-sharp at any size. Common formats: AI, EPS, SVG, PDF (vector content), CDR. Made by Illustrator, CorelDRAW, Inkscape — and the shapes, text, and paths inside a Fabric.js designer are vectors. Best for **logos, icons, type, line art, and packaging die-lines** — anything needing crisp edges at any scale.

Analogy: Raster vs vector is a photo mosaic vs a recipe. A mosaic has a fixed number of tiles — zoom in and you see the tiles, and you can't add detail that was never laid. A recipe ("draw a circle radius r") can be re-baked at any size and is always perfectly smooth.

Property	Raster (pixels)	Vector (math)
Made of	A fixed grid of colored pixels	Points, lines, Bézier curves, fills
Scaling	Resolution-dependent — blurs/blocks when enlarged	Resolution-independent — sharp at any size
Best for	Photos, continuous tone, gradients	Logos, type, icons, line art, die-lines
Formats	JPEG, PNG, TIFF, PSD, RAW	AI, EPS, SVG, PDF (vector), CDR
Output behaviour	Stuck at the pixels it has	Rasterized at the device's full resolution

The plain rule for print: use **vector for logos, type, and line art** (it scales infinitely and prints at the device's full resolution), and **raster for photos** (a photo can't be "made vector" without losing realism). The dangerous middle is a logo left as a low-res JPEG — it has thrown away its scalability forever.

The handoff: rasterizing (RIP) at output

Vectors don't print directly as math — at the final stage they are **rasterized** (turned into dots) by the printer's *RIP* (Raster Image Processor, the software that converts a page into the dot pattern the press lays down). The key point: a vector logo sent to a 2400-dpi imagesetter gets **2400-dpi-sharp edges**. The exact same logo flattened earlier to a 150-ppi JPEG is stuck at 150 ppi forever, no matter how good the press is.

Key takeaway: Raster bakes its detail in at creation and can only lose quality from there; vector carries instructions and is rendered fresh at the printer's full resolution. Match the type to the element: photos = raster, logos/type/line-art = vector.

Example: A customer emails a 400×400-px, 72-ppi JPEG logo pulled off their website to go on a 3-foot banner. Blown up that big it's a blurry mess. The vector .AI/.EPS/.SVG of the same logo prints knife-sharp at any size because it is re-rasterized at the printer's resolution.

6.2 Bézier curves & vector geometry — the math under a Fabric.js designer

If vectors are "math instructions," the most important instruction is the curve. Almost every smooth vector shape is built from **Bézier curves**. Here are the building blocks in plain words:

Anchor point (node)

> An **on-curve** point where a curve segment starts or ends. The line actually passes through it.

Control handle (control point / direction handle)

> An **off-curve** point that pulls the curve like a magnet. It sets the curve's *direction* and *tension* but the line does **not** pass through it.

The standard curve in PostScript, PDF, SVG, Illustrator, and Fabric is the **cubic Bézier**: four points `P0, P1, P2, P3`. The curve runs from `P0` to `P3` (the two anchors), while `P1` and `P2` are the off-curve control handles. As a parameter `t` sweeps from 0 to 1, the point on the curve is:

```
B(t) = (1-t)^3 P0 + 3(1-t)^2 t P1 + 3(1-t) t^2 P2 + t^3 P3

   P1 *----.            The 4 weights are the Bernstein
       \    `--.        polynomials:  (1-t)^3, 3t(1-t)^2,
        \      curve     3t^2(1-t), t^3  -- they ALWAYS
   P0 *--\----.   \      sum to 1.
          \    `-._ *  P3   Curve leaves P0 toward P1,
           `------* P2      arrives at P3 coming from P2.
```

The **geometric meaning** is friendlier than the formula: the curve leaves `P0` heading straight toward `P1` (so the line `P0→P1` is the starting tangent) and arrives at `P3` coming from `P2` (line `P2→P3` is the ending tangent). **Handle length** controls how strongly the curve bows; **handle angle** sets the tangent direction. The curve always stays inside the *convex hull* (the rubber-band outline) of its four points.

A **quadratic Bézier** uses only 3 points (one control): `B(t) = (1-t)²P0 + 2(1-t)t·P1 + t²P2`. TrueType fonts use quadratics; PostScript and OpenType-CFF fonts use cubics.

How it renders: the software steps `t` in small increments, computes `(x,y)` at each step, and joins them with short straight segments — this is called *flattening*. Finer steps look smoother on screen, but at print time the device rasterizes the true math regardless of your screen preview.

Analogy: Bézier handles are magnets pulling a wire. The two anchors pin the wire's ends; the control handles are magnets off to the side. The farther/stronger the magnet, the more the wire bows toward it — but the magnet itself never touches the wire.

Best practice: When drawing or editing nodes, place anchors at the curve's **extrema** (its top, bottom, left, and right points) and keep the handles horizontal/vertical there. You get fewer, cleaner nodes that are far easier to smooth and edit.

6.3 Resolution: DPI vs PPI vs LPI

These three abbreviations get used interchangeably in shops, but they are three different layers of the print process. Mixing them up is the root of most "why is it blurry?" confusion.

PPI — Pixels Per Inch

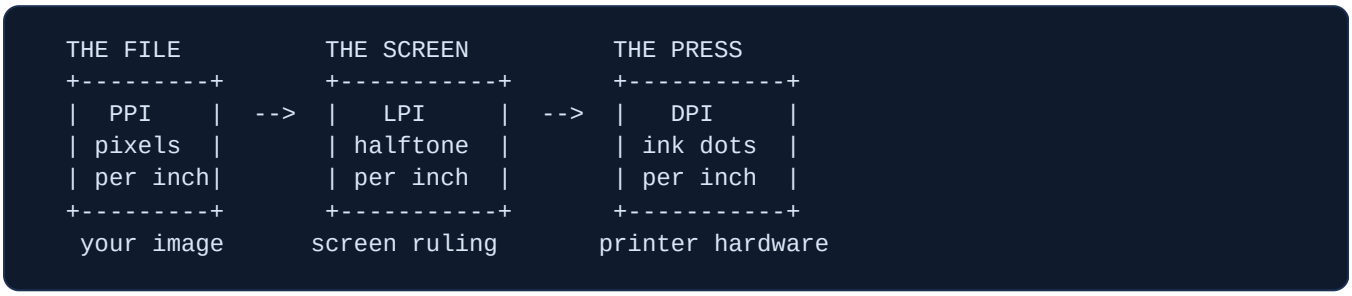
> The pixel density of a **digital image (file)**. This is the number that actually governs raster print sharpness. When someone says "300 dpi file," they almost always mean **300 PPI** — the industry uses "dpi" loosely.

DPI — Dots Per Inch

> The physical **ink dots a printer lays on paper**. Much higher than PPI (inkjet 1200–2880 dpi, laser 600–1200) because many tiny ink dots build a single image pixel or halftone cell.

LPI — Lines Per Inch

> The halftone **screen frequency** — rows of halftone dots per inch in offset/litho printing. Typical values: newspaper 85 lpi; general commercial & magazines 133–150 lpi (150 lpi is the common standard); high-end art books 175–200 lpi.



Key takeaway: PPI is your file, LPI is the halftone screen, DPI is the printer's ink. Only PPI is something you control in your artwork — get that right and the press handles the rest.

6.4 The 300 dpi rule and the 2× Quality Factor

"Use 300 dpi for print" is the most repeated rule in the business — and it's not magic. It comes from the **Quality Factor (QF)** rule: the image PPI you need is roughly **1.5× to 2× the LPI** of the press, with **2:1 being the Adobe/industry standard**.

So at the common commercial screen of 150 lpi: **150 lpi × 2 = 300 ppi**. That's the entire origin of "300 dpi for print." Because it's a ratio, it scales with the screen ruling:

Press screen (LPI)	Use case	Required image PPI (QF=2)
85 lpi	Newspaper	~170 ppi
150 lpi	General commercial / magazines	300 ppi
175 lpi	High-end work	350 ppi
200 lpi	Fine art books	400 ppi

A QF **below ~1.5** risks halftone artifacts and softness; **above ~2** just wastes data — the RIP throws away the extra pixels, giving you bigger files with no visible gain. The big exception is **large-format** (banners, billboards): they're viewed from far away, so 100–150 ppi (or far less for billboards) is fine because the eye can't resolve the dots at distance.

Common mistake: Treating "300 dpi" as a universal law. Newspaper photos legitimately need only ~170 ppi, and a billboard far less — while a fine-art book at 200 lpi actually needs 400 ppi. Match the PPI to the press's LPI, not to a memorized number.

6.5 Effective resolution after scaling

Here's the trap that bites even experienced designers. When you place a raster image in a layout and resize it, **the actual pixels don't change** — but the **effective (output) resolution** does. The formula:

```
Effective PPI = Original PPI / Scale Factor

ENLARGE  -> effective resolution DROPS (danger)
  300 ppi @ 200%  = 150 ppi   (too low)
  300 ppi @ 120%  = 250 ppi   (still good)

REDUCE   -> effective resolution RISES (always safe)
   72 ppi @  50%  = 144 ppi
   72 ppi @  25%  = 288 ppi
  300 ppi @  50%  = 600 ppi
```

Rule of thumb: enlarging a placed image up to about **120%** keeps very good quality; beyond that the effective resolution falls below target. Reducing is always safe — you're packing more source pixels into each printed inch.

Common mistake — the "drag-to-fit" trap: A designer places a 300-ppi photo in InDesign, then drags the frame corner to fill the page, scaling it to 220%. Preflight flags the effective resolution at ~136 ppi — it will print soft. The fix is a bigger source file or a smaller placement, never just dragging it bigger.

Best practice: Always check **effective resolution (after scaling)**, not just the file's stored PPI. This is why a good preflight tool catches "images dragged larger than they can support" rather than offering a "set to 300 dpi" button — bumping the PPI number does not add real detail.

6.6 Image resampling — bicubic, Lanczos, nearest neighbor

Resampling means changing an image's pixel count; the *algorithm* decides how the new (or removed) pixels are computed.

Algorithm	How it works	Quality / use
Nearest neighbor	Copies the single closest pixel	Fastest; blocky/jagged. Only for pixel art or hard-edge images you don't want smoothed
Bilinear	Averages the 4 (2×2) nearest pixels	Smoother than nearest, softer and cheaper than bicubic
Bicubic	Uses 16 (4×4) neighbors, non-linear	Smooth, detail-preserving; Photoshop's default ("Smoother" for enlarging, "Sharper" for reducing)
Lanczos	Windowed-sinc over a larger neighborhood	Sharpest, best detail retention; heaviest to compute. Preferred for downscaling (sharp/ImageMagick offer it)

Why upscaling degrades: there is no real information between the original pixels, so interpolation can only *guess* or blur. Detail is invented, never recovered. **Why downscaling is safe:** you're discarding surplus data, and a good filter averages many source pixels into each new one. (Downscaling still needs an anti-alias/low-pass step — nearest-neighbor downscaling just drops pixels and aliases badly, causing moiré.)

Analogy: Upscaling vs downscaling is like books and summaries. You can always summarize a long book into a paragraph (downscale: throw away detail cleanly). You cannot expand a paragraph back into the original book (upscale: you can only make stuff up).

Common mistake: Using a 72-ppi web image for print, then "fixing" it by upsampling the PPI number in software. This adds zero real detail — it only invents and blurs pixels. The only real fixes are a higher-resolution source or printing the image smaller.

6.7 Bit depth & channels

Bit depth is the number of bits used per channel — which sets how many tonal steps each color channel can hold.

8-bit per channel

> 256 levels per channel → about **16.7 million colors** in RGB. The standard delivery depth; perfectly fine for final output.

16-bit per channel

> 65,536 levels per channel → trillions of colors. Vastly more **headroom** for editing.

Channels

> The separate color components: RGB = 3 channels, CMYK = 4 (the print color space), grayscale = 1. Total image bits = depth × channels — so 8-bit RGB = 24-bit color, and 16-bit RGB = 48-bit.

Alpha channel

> An extra channel storing **per-pixel opacity** (0 = transparent, 255 = fully opaque at 8-bit). Supported by PNG, TIFF, and PSD — but **not by JPEG**.

Why edit in 16-bit: heavy edits (curves, large gradients, dodging/burning) on an 8-bit image cause **banding** — visible stepping in smooth gradients like skies — because there simply aren't enough levels to redistribute. 16-bit keeps gradients smooth all the way through manipulation.

Example: A sunset edited heavily in 8-bit shows visible step-bands in the gradient when it hits the press. Redoing the same edit in 16-bit and exporting flat removes the banding entirely.

Best practice: Edit and retouch your master in 16-bit, convert to 8-bit only on final export. For print transparency, deliver TIFF-with-alpha or PNG (16-bit if smooth edges matter), never JPEG.

6.8 Alpha compositing & blend modes — layer math before flattening

When layers overlap, the software performs real math to combine them. Understanding it explains why some exports get ugly halos or "white boxes."

Alpha compositing

Alpha compositing is combining a layer over a background using its alpha (opacity) to fake partial transparency. The standard "over" operator (from Porter–Duff) is:

```
out = src * alpha + dst * (1 - alpha)
      \_______/      \____________/
      top layer        background showing through
```

There are two ways alpha can be stored, and mixing them up causes edge halos:

Straight (unassociated) alpha

> The RGB color is stored independently of the opacity value.

Premultiplied (associated) alpha

> The RGB is already multiplied by alpha — faster to composite and avoids dark/light fringe halos at edges. **Mixing the two wrongly produces edge halos**, which matters when exporting from a canvas/designer for print.

Blend modes

Blend modes are per-channel math formulas that combine a source layer with the base layer below it:

- **Multiply:** `result = base × top ÷ 255` → always darkens. It mimics ink layering / CMYK absorption (each ink layer absorbs more light) — the print-realistic darkening mode.
- **Screen:** the inverse of multiply → always lightens.

- **Overlay / Soft Light:** contrast modes (multiply in the shadows, screen in the highlights).

Flattening

Flattening bakes all the layers, blend modes, and transparency down into one opaque raster. Once flattened it is **permanent** — the blend math is resolved into final pixel values and the layers are gone.

Common mistake — bad transparency flattening: Many RIPs and older PostScript can't render live transparency or blend modes, so prepress flattens them. Done badly, this causes "white box" artifacts, color shifts, hairline stitching lines, or soft rasterized-text edges wherever transparent objects overlap.

Best practice: Keep transparency **live** by exporting a **PDF/X-4** file (it supports live transparency) instead of **PDF/X-1a** (which forces flattening). If you must flatten, flatten at high resolution. And always outline type to curves or embed fonts in the PDF/X — otherwise the RIP can't find the font, substitutes a wrong one, and text reflows or breaks.

Section summary:

- **Raster = pixels, vector = math.** Use raster for photos, vector for logos/type/line-art; vector re-rasterizes at the printer's full resolution while raster is stuck at the pixels it was born with.
- **Bézier curves** (cubic: P0–P3 with two off-curve handles) are the geometry under every vector tool — anchors pin the ends, handle length/angle bow the curve, and the math is rendered fresh at output.
- **PPI (file) → LPI (screen) → DPI (ink)** are three different layers. The "300 dpi rule" is just Quality Factor 2 at 150 lpi; scale it with the press (350 ppi at 175 lpi, ~170 ppi for newspaper).
- **Effective PPI = original PPI ÷ scale factor.** Enlarging drops resolution (keep under ~120%); reducing is always safe. Upscaling invents detail and never recovers it — downscale, never upscale.
- **Edit in 16-bit** to avoid banding, keep an alpha/layered master, and export **PDF/X-4** to preserve live transparency — flatten and convert to 8-bit only at the final output step.

8. Halftoning & Screening: Turning Tone into Dots

Look at a photo on your phone and you see a smooth, continuous slide from light to dark — millions of shades. Now look at that same photo printed in a newspaper through a magnifying glass, and the smoothness vanishes: it is just a field of tiny black dots, big in the dark areas, small in the light areas, sitting on white paper. This section explains the trick that bridges those two worlds. It is one of the oldest and cleverest ideas in all of printing.

7.1 Why a press cannot print "continuous tone"

Before we define anything, let's name two terms.

Continuous tone (contone)

> An image where tone changes smoothly with no visible steps — like a photographic gradient or what you see on a glowing screen.

Halftoning (also called screening)

> The process of converting a continuous-tone image into a pattern of dots that, viewed from a normal distance, the eye reads back as smooth tone.

Here is the core problem. A printing press is **fundamentally binary**: at any single spot on the paper, ink is either there or it isn't. A cyan ink film prints at **one fixed density** — it cannot lay down "50% cyan" as a thinner, paler film the way a screen simply glows dimmer. The ink is full strength or absent.

So how do you show a pale sky or a soft shadow with an ink that only knows "on" or "off"? You break the image into a grid of tiny dots and vary **how much of the white paper those dots cover** — the **area coverage**, or tint percentage. Tiny dots floating on white paper read as a light tone; dots so fat they nearly touch read as a dark tone. 0% coverage = paper white; 100% = solid ink.

The magic is in your eyeball. At normal viewing distance your eye **cannot resolve the individual dots**, so it averages — it blends the black dots and the white gaps into a single gray. This is called **optical mixing** or spatial integration. (The technique was first patented by **William Henry Fox Talbot in 1852** and refined into the glass cross-line screen later in the 1800s.)

Analogy: A halftone is **pointillism**, or a stadium card-stunt where thousands of fans each hold up one solid-color card. Up close it's just colored squares; from across the stadium your eye fuses them into a smooth picture of a flag or a face.

Key takeaway: A press can only print "ink" or "no ink" at full strength. Halftoning fakes every shade in between by varying how much paper the dots cover, and your eye does the blending.

7.2 Two ways to vary the dots: AM vs FM screening

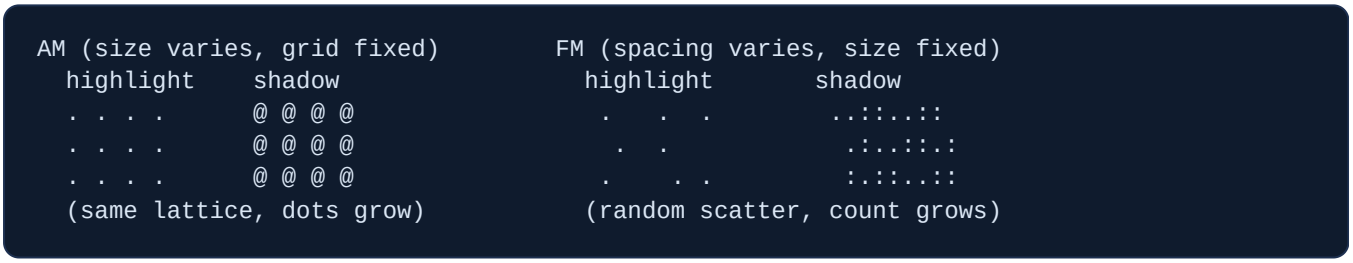
There are two fundamentally different strategies for arranging the dots.

AM — Amplitude-Modulated (conventional) screening

The dots sit on a **fixed, regular grid at fixed spacing**; only the **dot size varies** to change tone. ("Amplitude" means size.) Highlights get tiny dots, shadows get big dots, but every dot lives on the same evenly-spaced lattice. This is the traditional screen and still the default for most offset work — it gives very smooth, predictable, stable midtones. Its one weakness: that regular grid is exactly what causes **moiré** when several colors overlap (more on that below), which is why the plates must be angled.

FM — Frequency-Modulated (stochastic) screening

Here the **dot size is fixed and tiny**; only the **spacing and number of dots varies** (the "frequency"). Highlights get a few scattered microdots; shadows get a dense crowd of them. Placement is **pseudo-random**. The microdots are roughly **10–30 µm (microns)** across — often cited around 20–30 µm. Because the pattern is random, there is **no regular grid, so no LPI and no screen angle** — and therefore **moiré and rosettes essentially disappear**. That makes FM excellent for tricky subjects (fabric weaves, fine detail, sharp diagonal lines) and for 6- or 7-color extended-gamut printing. The cost: the tiny dots are fragile, harder to hold on press, more prone to dot gain, and can look slightly grainy.



Hybrid / XM (cross-modulated) screening

Modern CTP (computer-to-plate) RIPs often blend the two: **AM in the midtones** for stability, switching to **FM in the extreme highlights and shadows** where AM dots would be too small to hold or too merged to separate. Best of both worlds.

Aspect	AM (conventional)	FM (stochastic)
What varies	Dot size	Dot spacing / count
Dot placement	Fixed regular grid	Pseudo-random
Has LPI / screen angle?	Yes	No (none needed)
Moiré / rosette risk	Yes — needs angled plates	Essentially none
Detail / sharpness	Good	Excellent
Press stability	High, predictable	Harder; more dot gain

Key takeaway: AM keeps dots on a grid and grows them; FM keeps dots tiny and scatters more of them. AM is stable but needs angled plates to dodge moiré; FM sidesteps moiré entirely but is fussier on press.

7.3 Screen frequency — LPI (AM only)

LPI (lines per inch) is the number of rows of halftone dots packed into one inch — also called the ruling, line screen, or screen frequency. Higher LPI = finer, smaller, more numerous dots = more detail, *but only if the paper and press can hold those tiny dots without smearing them together*.

Application	Typical LPI	Why
Newspaper / newsprint	85–110 (≈85)	Rough absorbent paper can't hold fine dots
Magazines / general commercial	133–175 (150 is the workhorse)	Coated stock holds finer dots
Coated commercial sheetfed	150–200	Smooth surface, controlled press
Fine-art / high-end / specialty	200–300	Best paper, best plates

Coarser, more absorbent paper forces a *lower* LPI (otherwise the dots "plug" and merge into mud); smoother coated stock permits higher LPI.

The relationships everyone confuses: LPI vs PPI vs DPI

These three look alike and trip up beginners constantly. Keep them straight:

PPI (pixels per inch)

> The resolution of your *image file* at its final printed size.

LPI (lines per inch)

> The fineness of the *halftone screen* the press uses.

DPI (dots per inch)

> The resolution of the *output device* (the platesetter/imagesetter) — how many tiny laser spots per inch it can place.

Image resolution rule (the "quality factor"): supply images at **1.5× to 2× the LPI** in PPI at final size. So for a 150-LPI job, use **300 PPI** images (2× — this is the origin of the famous "300 dpi for print" rule of thumb). 1.5× (~225 ppi) is the practical minimum; 2× gives a safety margin.

Why platesetters run at 2400+ DPI while the screen is only 150 LPI: each halftone dot is itself built from a little grid of device spots, and you need many spots per dot to vary the dot's size in fine steps. The number of reproducible gray levels is:

```
gray levels = (device DPI / LPI)^2 + 1

  2400 dpi / 150 lpi = 16
  16^2 + 1 = 257 gray levels   (>= 256 wanted for smooth tone)
```

That formula is *why* a 2400-dpi platesetter is needed to drive a mere 150-lpi screen smoothly. **Never confuse the device's DPI with the screen's LPI — they are different things.**

Common mistake: Supplying a 72-ppi web image for a 150-lpi print (you need ~300 ppi). It prints soft and pixelated. People do this because they confuse screen resolution with print resolution.

7.4 Dot shape and the "50% problem"

Halftone dots come in several shapes: **round, square, elliptical (oval / "chain dot"), and diamond**. A dot keeps its chosen shape until it grows big enough to touch its neighbors — and *when* it touches is where the drama happens.

As dots grow toward dark tones, at around **50% coverage** adjacent dots can suddenly connect. The moment they join changes the geometry abruptly, producing a visible **jump or step in midtone gradients (banding)** — the "dot-gain cliff."

- **Round dots:** all four neighbors connect almost simultaneously near 50% → the most abrupt, most violent midtone jump (and a visible "chain" look in midtones).
- **Square dots:** corners touch first; very sharp, snappy detail, but still a hard transition through 50%. Past 50% the shape inverts to negative white dots.
- **Elliptical / chain dots:** the oval connects along its *short axis first*, then later along its *long axis* — two gentle transitions instead of one harsh one → the **smoothest midtones**. This is the most popular choice for general commercial work and for skin tones.

```
Tone growth (highlight -> shadow):
  o   o   o        round dots, separate (highlight)
  oo  oo  oo       growing
  ((  ((  ((       ellipse joins SHORT axis  (~40%)
  ##  ##  ##       midtone
  joins LONG axis later (~60%)  -> inverts -> solid
```

Best practice: For photographs and skin tones, elliptical/chain dots give the gentlest midtone transition and avoid the sudden tonal jump that round dots show right at 50%.

7.5 Screen angles — why every plate is rotated

Now the heart of color halftoning. If two screens with the *same* LPI overlap at the *same* angle, their grids beat against each other and create a large, ugly, wavy interference pattern called **moiré** (think of a

faint plaid shimmer). Even a hair of misregistration makes it worse.

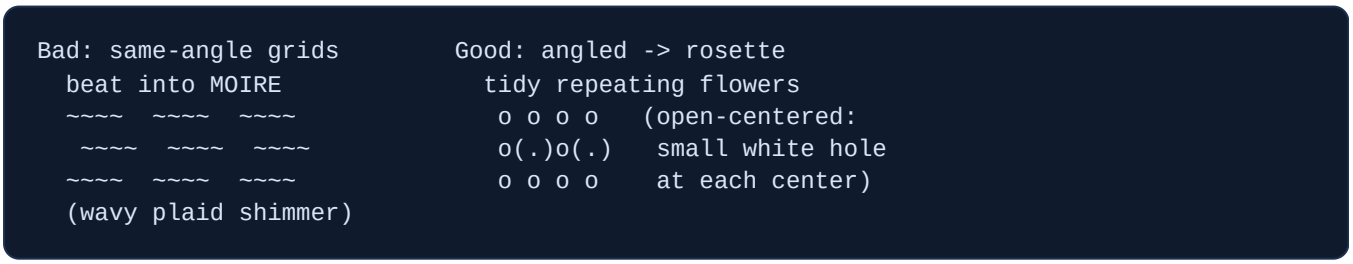
The fix is to **rotate each color's screen to a different angle** so the grids interleave instead of clashing. The standard US CMYK set is:

Ink	Screen angle	Reason
Cyan	15°	One of the three strong inks; kept 30° from the others
Black (K)	45°	Most dominant ink; the eye is *least* sensitive to diagonal patterns, so 45° hides it best
Magenta	75°	The third strong ink; 30° from both cyan and black
Yellow	0° (90°)	The faintest ink, so it "draws the short straw" at only 15° from cyan/magenta — its moiré is nearly invisible anyway

The key idea: the three **strong, dark inks (C, K, M) are spaced 30° apart** (15 / 45 / 75). A 30° separation is the "magic number" that shrinks any leftover moiré to its smallest, least-visible scale. **Black gets 45°** because it's the most visible ink and the diagonal orientation is where our eyes least notice a regular pattern. **Yellow gets 0°** because it's so pale that even though it sits only 15° from cyan and magenta (breaking the ideal 30° rule), the resulting moiré is essentially invisible.

Rosettes — the "good" pattern we are aiming for

When all four correctly-angled screens print over each other, the dots arrange into tiny, repeating, **flower-like clusters called rosettes**. This is the *intended, healthy* result — it's what your eye blends into smooth color. There are two kinds: **open-centered** (a small white hole in the middle — generally preferred and more stable) and **closed / dot-centered**. **Moiré is bad** (large and distracting); **a rosette is good** (small and designed). The entire angle system exists to guarantee a clean rosette instead of moiré.



Analogy: Screen angles are like **stacking window-screen mesh**. Two meshes laid at the same angle show no pattern; nudge one slightly and an ugly shimmer (moiré) appears; rotate them to deliberate angles and the interference shrinks into a tidy little flower (the rosette).

Common mistake: Re-screening an already-screened image — e.g. placing a halftoned scan or a customer-made dot pattern, then letting the RIP screen it again. The two grids clash and you get moiré with the press screen. **Always send continuous-tone art and let the RIP screen it exactly once.**

Example: A green landscape (cyan + yellow) is the classic place moiré escapes, because yellow sits only 15° from cyan. A low-res or badly-angled job shows a faint plaid shimmer in the grass — which is exactly why yellow is handed the weak 0° angle in the first place.

7.6 Trapping — overlaps that hide misregistration

Two more terms first:

Registration

> The perfect alignment of all four color plates on top of each other.

Misregistration

> When plates print slightly off (from paper stretch, web tension, plate/blanket movement, or press mechanics). Without protection, this leaves **slivers of bare white paper** peeking between two colored areas.

Trapping is the prepress fix: *deliberately overlapping adjacent colors by a tiny amount* so that even when registration drifts, no white gap (or wrong-color fringe) shows. There are two moves:

- **Spread ("fattie"):** make the *lighter* foreground object slightly **bigger** so it bleeds into the darker background.
- **Choke ("skinny"):** make the *lighter* background opening slightly **smaller** so the darker object overlaps it.

Golden rule: always spread the lighter color into the darker color. The light ink contaminates the dark ink the least, so the overlap is the least noticeable. Trap width is tiny — typically about **0.25 pt (~0.003–0.004 in, or 0.08–0.1 mm)**.

Two related terms: a **knockout** removes the background where the top object sits (no ink mixing, but it *requires* a trap); an **overprint** lets the top object print right over the background (e.g. black text overprints colored areas so no trap is needed at all). Modern RIPs perform trapping automatically.

Analogy: Trapping is **sewing a slightly oversized patch** — cut the patch a hair bigger than the hole, so even if your stitching wanders a little, no bare fabric shows through the gap.

Example: Red text on a blue box, printed with no trap. The press drifts 0.2 mm and a thin white halo appears around every red letter. Add a 0.25 pt spread on the red and the gap is swallowed — nobody ever notices.

7.7 Dot gain (TVI) — and how it ties everything together

Dot gain means the printed halftone dot ends up **larger than it was in your file**, so the print looks darker and muddier than intended. ISO calls it **TVI (Tone Value Increase)**. It has two causes: **mechanical** gain (ink physically squashed wider under press and blanket pressure, plus absorption into the sheet) and **optical** gain (light scatters sideways under the ink edge inside the paper, making each dot *look* bigger than it physically is).

Dot gain is **greatest in the midtones (~50%)**, because that's where dots have the most total *perimeter* — the most edge available to grow from. It's classically measured at the 40% and 80% tints, with **50% as the headline figure**.

Process / paper	Screen	Typical 50% dot gain
Coated sheetfed offset	150 lpi	~15% (ISO 12647-2 gloss coated ≈ 15–17%)
Uncoated sheetfed	133 lpi	~20%
Coated web offset	133 lpi	~22%
Newsprint web	100 lpi	~30% (rough absorbent paper = highest gain)

Notice how this section's earlier ideas all connect to dot gain:

- **Higher LPI = more dot gain** (smaller dots have more perimeter relative to area, so they fatten proportionally more) — another reason coarse papers are forced to lower LPI.
- **FM/stochastic = more dot gain** (microdots are almost all perimeter) — so FM must be characterized carefully.
- **Dot shape spikes gain at 50%** (the touch-point is the tonal-jump cliff from §7.4).

The fix — compensation: the RIP applies a **dot-gain compensation curve** that deliberately *shrinks* the dots in the file so that, after the press fattens them, they land at the right size. The correct curve is determined by printing to a standard — **ISO 12647-2, G7, or GRACoL** — and measuring a control strip with a densitometer or spectrophotometer.

Common mistake: Designing to on-screen color and ignoring dot gain. Midtones then print 15–30% darker and muddier than your monitor showed, and shadows plug to flat black. Soft-proof with the correct ICC / dot-gain profile so the screen tells you the truth.

Best practice: Match LPI to the paper and image PPI to the LPI (the ~2× rule: 150 lpi → 300 ppi). Let the RIP own screening, angles, and trapping — supply clean contone CMYK plus the correct ICC profile and use the standard 15/75/0/45 angles. Print to a standard (ISO 12647 / GRACoL / G7) with dot-gain compensation and verify with a control strip and spectrophotometer so results repeat job to job.

Section summary:

- A press only prints ink-on or ink-off at full strength, so **halftoning** fakes every shade by varying how much paper the dots cover; your eye blends them optically.
- **AM** screening grows fixed-grid dots (stable, but needs angled plates); **FM/stochastic** scatters tiny fixed-size dots (no LPI, no angles, no moiré, but fussier and grainier); hybrid/XM mixes both.
- **LPI** is screen fineness, **PPI** is image resolution (~2× the LPI → 300 ppi for 150 lpi), and **DPI** is device resolution; $(DPI/LPI)^2+1$ gray levels is why platesetters run at 2400+ dpi.
- Plates are rotated to **C 15° / M 75° / Y 0° / K 45°** — the strong inks 30° apart, black at the hard-to-see 45°, yellow at the weak 0° — to turn ugly **moiré** into the intended tiny **rosette**.
- **Trapping** spreads the lighter color into the darker (~0.25 pt) to hide misregistration, and **dot gain (TVI)** — worst at 50% — must be compensated via standards (ISO 12647 / G7 / GRACoL) so prints don't come out dark and muddy.

9. Trapping (Deep Dive)

When a printing press lays down color, it does not paint all the colors at once. Each ink — say cyan, then magenta, then a special blue — is applied by its own separate *plate* (a metal or rubber carrier that holds one ink) in its own separate pass or unit. The paper, and those plates, never line up perfectly. They shift by a fraction of a hair from one ink to the next. **Trapping** is the prepress trick that hides those tiny shifts so your final print does not show ugly white slivers where two colors meet.

Trapping

> Deliberately making two adjacent colors overlap by a tiny amount, so that if the press misaligns the inks slightly, no bare paper shows at the seam between them.

Misregistration

> When the ink layers do not line up exactly — the plates or paper shifted between passes. "Registration" means the inks are in perfect alignment; *mis*registration means they are not.

Plate / ink unit

> The part of the press that applies one single color. Four-color print uses four plates (cyan, magenta, yellow, black); a job with spot colors adds one plate per spot color.

Why trapping exists: the white-gap problem

Imagine two flat blocks of color sitting edge to edge — a blue shape next to a yellow shape. To print this, the press normally **knocks out** the yellow: it cuts a yellow-shaped hole in the blue so the two inks do not mix and muddy each other. The blue plate prints around the hole; the yellow plate fills the hole. If both plates land in exactly the right spot, the edges meet perfectly. But they never do. The paper stretches, a plate is a thread off, a web press whips the paper through at speed. The result is a thin gap where neither ink printed — and the bare white paper peeks through.

Analogy — Think of a child coloring a cartoon. They draw the outline of a balloon and fill it in, but leave a thin unpainted ring just inside the outline. White paper shows through that ring. Trapping is like telling the child: "bleed your fill a tiny bit *past* the outline." Now even if your hand wobbles, no bare paper ever shows. Or picture hanging wallpaper: you overlap the seams slightly so the wall behind never peeks out, no matter how the paper settles.

```
NO TRAP — press shifts, white gap appears:

   BLUE |  | YELLOW       ( the "|  |" = bare paper )

WITH TRAP — colors overlap, no gap can show:

   BLUE [##] YELLOW        ( "[##]" = shared overlap zone )
```

Misregistration is worst on **offset**, **flexo** (flexible-plate printing, common for packaging and labels), and **screen** printing, and on stretchy stock like newsprint or thin film. It is least on single-pass digital/toner presses, because those lay all colors in one go with no plate-to-plate shift.

Spread vs. choke: two ways to make the overlap

There are only two ways to create that overlap zone, and they differ only in *which color moves*.

Spread ("fattie")

> The foreground object is grown slightly so it bleeds *outward* over the background. The object gets fatter.

Choke ("skinny")

> The foreground object stays put, but the background grows slightly *inward* over it. The object effectively shrinks.

Both produce the identical result: a thin shared band where the two inks overlap. The only question is which color grows into the other.

The decision rule: the lighter color always spreads into the darker one

This is the single most important rule in trapping, so memorize it:

Key rule — The **lighter** color always spreads into the **darker** color. The decision is based on *neutral density* (how light or dark a color is to the human eye). The lighter color is the one that grows.

Why? Because the **darker color defines the edge your eye sees**. Your eye reads the dark color as "the shape." If you move (fatten) the lighter color, the shape's perceived outline stays crisp and the overlap hides invisibly under the dark edge. If you moved the dark color instead, you would visibly distort and fatten the shape.

Worked example — Light-yellow text on a dark-blue background. The blue defines the letters' outline to your eye. So you *spread the yellow outward*. Even if the press shifts, the yellow now reaches under where the blue edge sits — no white gap, and the letters still look sharp. If you instead choked (spread the blue inward into the yellow), you would visibly fatten and smudge the letterforms.

How wide is a trap?

A trap is tiny — measured in fractions of a point. At 150 lpi (lines per inch, a common print resolution):

Measure	Typical trap width
Inches	1/300" to 1/150"
Points	0.24 pt to 0.48 pt
Millimeters	0.08 mm to 0.16 mm

A handy rule of thumb is "about half a dot" — roughly 1/300" (0.08 mm) at 150 lpi. **Black and dark traps can be 1.5× to 2× wider**, because dark ink masks any misalignment well and a wider trap simply won't be noticed. Trap width should scale with press quality: a tight, well-registered press needs a smaller trap; a sloppy press or stretchy stock (flexo, newsprint) needs a larger one. Too wide and the trap becomes a visible dark or colored outline; too narrow and gaps still slip through.

When you need trapping — and when you must NOT

Trap it	Do NOT trap it
Spot colors (Pantone/PMS) adjacent to anything — each is its own plate with zero shared ink, so any shift = white gap. Always trap.	**Continuous-tone photographs** (CMYK images) — soft edges and shared CMY inks already overlap. Trapping adds visible color fringes and degrades the image.
Sharp, hard edges between two solid flat colors — logos, vector art, text on a colored background, packaging. The classic trapping cases.	**Two process colors that share enough common ink** (e.g. both contain a lot of cyan) — the shared plate already bridges the gap. Little or no trap needed.
Spot color touching process color.	**High-quality presses with tight registration**, and most digital/toner/inkjet — trap artifacts can be worse than the rare misalignment.

Pro tip — Counter-intuitively, 4-color (process) art is often *easier* on trapping than 2-spot-color art, because the four process inks frequently share ink across the seam and bridge gaps on their own. Pure spot-to-spot is the hardest because the two inks share nothing.

The simpler alternative for black: overprint vs. knockout

Knockout (the default)

> The top color punches a hole in the layer beneath so the inks don't mix. This is exactly where gaps appear — so knockouts are what need trapping.

Overprint

> The top color prints directly *on top of* the background with no hole cut beneath it. No knockout, so no gap is possible — and no spread/choke math is needed.

Black text and thin black lines are routinely set to **overprint**. Black ink is opaque enough to cover whatever sits under it, so a shift can never reveal white. This is the everyday solution for small black type.

```
KNOCKOUT edge (gap-prone):

   ___YELLOW___|GAP|___BLUE___

OVERPRINT black (no gap possible):

   ___YELLOW___[ BLACK on top ]___BLUE___
```

Common mistake — Overprinting *light* colors causes color shifts (the background bleeds through and changes them). Worst of all, **overprinting white makes the white disappear entirely** — white "ink" with overprint on means nothing prints there. A classic, expensive disaster. Overprint is safe for black, dangerous for everything light. Note: "rich black" (black plus a CMY underlay for a deeper black) follows normal black *trapping* rules, not the simple overprint shortcut.

Automatic / in-RIP trapping

Nobody traps by hand anymore on real jobs. Trapping is computed automatically by software, either in the prepress/PDF workflow or — most commonly — at the **RIP**.

RIP (Raster Image Processor)

> The engine that converts your page into the dot pattern the press actually prints. "In-RIP trapping" means the traps are calculated at this final raster stage.

Trap preset

> A named bundle of trap settings (default trap width, black width, image-trapping on/off, thresholds) you can reuse across jobs.

The **Adobe In-RIP Trapping** engine detects contrasting color edges, reads the neutral density of the two colors, and automatically expands the lighter into the darker — even trapping one object against several different backgrounds at once. To use it you need a PostScript Level 2+ device with a RIP that supports Adobe In-RIP Trapping; in the print dialog you set Color to "In-RIP Separations" and Trapping to "Adobe In-RIP." Always confirm your print provider's RIP supports it.

Big gotcha — In Acrobat Pro, the "Trap Presets" feature does *not actually create traps in the PDF*. It only stores instructions that a compatible RIP applies later. If your printer's RIP ignores them, nothing is trapped. And if the designer manually traps *and* the RIP traps, you get a **double trap** — visibly fat, ugly seams. Coordinate with your printer on who does the trapping; do it once.

Where this lands in Print-Flow-360

Trapping is a press-side, separated-color concern — and Print-Flow-360 today works in **RGB only**, which keeps trapping out of the platform's direct scope but makes it a documented gap you should understand:

- The **Fabric.js design studio** (`designer/`) produces RGB vector and raster artwork. It has no concept of spot colors, overprint, knockout, or trap zones — a customer's two flat shapes are just RGB fills with no trapping intent attached.
- The **Node PDF service** (`pdf-service/` , port 4000) uses `sharp` for images and PDFKit for PDFs. These generate RGB output; PDFKit does not author trap presets or color separations. So any trapping must happen *downstream*, at the print provider's RIP — not here.
- The platform has **no RGB→CMYK conversion and no preflight** step. That means there is no stage that would separate colors into plates or flag "these two adjacent spot colors will gap." Combined with the **weak order-to-production spine**, trapping responsibility falls entirely on whichever press/RIP fulfills the order.

Actionable for PF-360 — Until CMYK separation and preflight exist, the safe, honest position is: artwork leaves the platform as RGB, and *trapping is the print provider's RIP responsibility*. If you later build a production handoff, the right place to add trap-preset metadata is at the PDF export/handoff layer (so the RIP knows what to do) — never hand-bake traps into the designer's RGB canvas, where they'd be wrong for any other output device.

Key takeaways —

- Trapping = a deliberate tiny overlap between adjacent colors so press misregistration never shows a white paper gap.
- **Spread** grows the foreground outward; **choke** grows the background inward — same overlap, different color moves.
- **The lighter color always spreads into the darker one**, because the dark color defines the edge your eye sees.
- Trap width is tiny: ~0.24–0.48 pt (0.08–0.16 mm) at 150 lpi; black traps run 1.5–2× wider.
- **Trap:** adjacent spot colors, and sharp edges between solid flat colors (logos, text, packaging). **Don't trap:** photographs, process colors sharing ink, tight digital presses.
- **Overprint black** instead of trapping it; never overprint white (it vanishes).
- Modern shops trap automatically **in the RIP** — coordinate so you don't double-trap.
- In Print-Flow-360, the RGB-only designer and PDF service can't trap; with no CMYK/preflight, trapping is the print provider's RIP job downstream.

10. Inside a PDF: Structure, Graphics & Fonts

You send PDFs to the print shop every day, but what is actually inside one? Understanding the anatomy of a PDF is the single best way to stop the most common print disasters: wrong fonts, muddy colour, and pixelated images. This section opens the file up and shows you the gears turning inside — in plain English, no programming required.

8.1 What a PDF really is

Most people imagine a PDF as a "picture of a page." It is closer to a **small database of objects plus a map telling the reader where each object lives**. It is *not* a top-to-bottom stream of text like a web page. PDF is defined by the open ISO standard **ISO 32000** (PDF 1.7 = ISO 32000-1, 2008; PDF 2.0 = ISO 32000-2, 2020). It began as Adobe's format and is now a published international standard.

Analogy: Think of a PDF as a shipping container with a manifest taped to the back door. The contents are boxes (objects). The manifest lists the exact shelf (byte position) of every box. A sticker on the door says "the manifest is on the last page, and the master inventory is box #1." You read the door sticker first, then jump straight to any box — you never unpack the whole container.

The four physical parts

Header

> The first line, e.g. `%PDF-1.7`, declaring the spec version. It is usually followed by a line of high-byte binary characters so email/FTP tools treat the file as binary, not text.

Body

> The actual content — every object that makes up the document.

Cross-reference (xref) table

> An index giving the **exact byte offset** of every object, so the reader can jump straight to any object without scanning the whole file. This is what makes a PDF *random-access*.

Trailer

> A tiny dictionary at the very end. It tells the reader two critical things: where the xref table starts (`startxref`) and which object is the document `/Root` (the Catalog). It also carries `/Size` (object count) and `/ID`.

Key takeaway: A reader opens a PDF by reading the **end first** (trailer → xref), then jumps directly to the objects it needs — the opposite of how you read a book.

8.2 Objects — the eight building blocks

Everything in a PDF body is made of exactly **eight object types**: Boolean, Numeric (integer or real), String, Name, Array, Dictionary, Stream, and Null. Four of them do most of the work:

Name

> A token starting with `/`, e.g. `/Type`, `/Font`, `/MediaBox`. Used as keys and identifiers — not the same thing as a text string.

Array

> An ordered list in square brackets, e.g. `[0 0 612 792]` (a page box measured in points).

Dictionary

> Key→value pairs wrapped in `<< >>`, e.g. `<< /Type /Page /MediaBox [0 0 612 792] >>`. This is the workhorse object.

Stream

> A dictionary *followed by raw bytes* between `stream … endstream`. Used for anything large or binary: page content, embedded fonts, images. The dictionary carries `/Length` and a `/Filter` (compression, usually `/FlateDecode` = zlib/deflate).

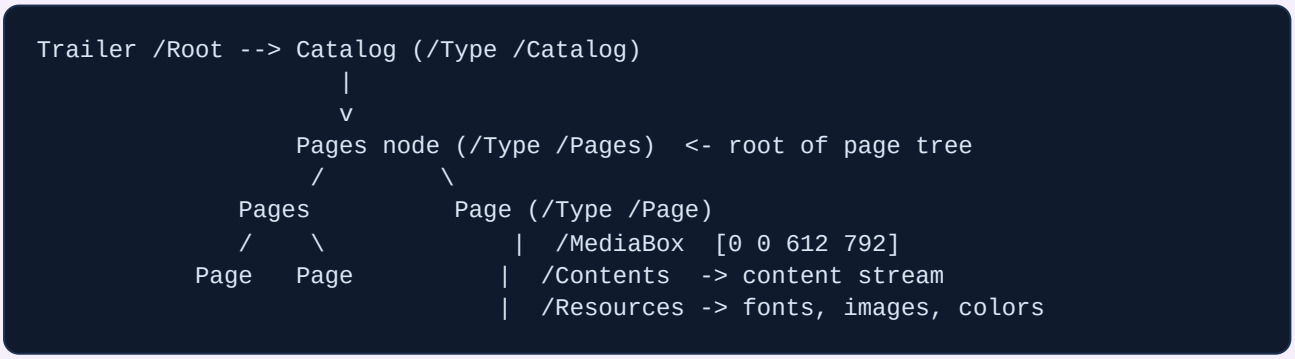
Any object can be made an **indirect object** by giving it an ID, written as `12 0 obj … endobj` (object number 12, generation 0). Other objects point at it with `12 0 R` — the "R" means *reference*. The xref table is simply the index of where every numbered object lives.

How the xref table works

Each classic xref entry is **exactly 20 bytes** so the table is itself randomly addressable: a 10-digit byte offset (zero-padded) + space + a 5-digit generation number + space + a one-character flag `n` (in-use) or `f` (free), then a 2-byte end-of-line.

> **Common mistake:** Assuming "deleting" content in a PDF removes it. PDFs use **incremental update** — edits are *appended* to the end with a new xref section and trailer chained to the old one via `/Prev`; the original bytes stay. This is why edited PDFs grow, and why "removed" text can sometimes be recovered — a real privacy gotcha. PDF 1.5+ can compress the index into a *cross-reference stream* and pack objects into *object streams*, giving smaller files that are harder to read in a hex editor.

8.3 The document hierarchy



The trailer's `/Root` points to the **Catalog**, the top of the tree. The Catalog points to the **Pages** node — the root of the **page tree**. The page tree is a (usually balanced) tree of branch nodes (`/Pages`) and leaf nodes (`/Page`). Balancing it means a 5,000-page document can find page 4,000 quickly without walking a flat list.

Each **Page** dictionary holds `/MediaBox` (the physical sheet size), `/Contents` (the stream that draws the page), `/Resources` (the fonts, images and colour spaces it uses), and optionally the print boxes `/CropBox`, `/BleedBox`, `/TrimBox` and `/ArtBox` — where **TrimBox = the final cut size** and **BleedBox = trim + bleed**.

> **Key takeaway:** PDF measures everything in **PostScript points: 1 pt = 1/72 inch**. US Letter = `[0 0 612 792]` (8.5×11 in), A4 ≈ `[0 0 595 842]`. The origin is the **bottom-left** corner and Y increases upward — the maths convention, not the screen convention.

8.4 Content streams — how the page is "drawn"

A page's appearance comes from its **content stream**, which is a tiny **postfix (Reverse-Polish) program**: the operands come first, then the operator. For example `1 0 0 RG` sets the stroke colour to red, then `100 100 m` moves the pen. It is an *imperative paint program* — there is no "this is a paragraph," only "put this glyph at this X/Y, draw this line, fill this path." Meaning and structure have to be reconstructed afterwards, which is exactly why text extraction and reflow are hard.

Graphics-state operators

- `q` / `Q` — **save / restore** the graphics state on a stack. Everything between them runs in isolation (colour, line width, clipping, transform). Always paired.
- `cm` — concatenate a **transformation matrix** (the CTM): scale, rotate, skew or move everything drawn afterward (six numbers).
- `w` line width, `J` / `j` caps/joins, `d` dash pattern, `gs` apply an extended graphics state (transparency, blend mode).

Vector paths (resolution-independent)

Construct with `m` (moveto), `l` (lineto), `c` (cubic **Bézier curve**), `re` (rectangle) and `h` (closepath). **Paint** with `S` (stroke), `f` (fill, *nonzero winding rule*), `f*` (fill, *even-odd rule*), `B` (fill and stroke), and `W` / `W*` (use the path as a **clipping** mask). The two winding rules decide which side of overlapping shapes counts as "inside" — even-odd is what lets a donut have a real hole.

Images

A raster image is an **XObject** of subtype Image — a stream listed in the page's `/Resources`, carrying `/Width`, `/Height`, `/BitsPerComponent`, `/ColorSpace` and a `/Filter` (`DCTDecode` = JPEG, `JPXDecode` = JPEG2000, `FlateDecode` = lossless, `CCITTFaxDecode` / `JBIG2Decode` = bilevel scans). It is drawn by setting a CTM that maps a **1×1 unit square** to the desired size and position, then calling `Do`.

> **Common mistake:** Scaling a placed raster up via the matrix. Because the image's scale lives in the CTM, not the pixels, enlarging that 1×1 square too far silently drops the **effective resolution below 300 dpi**. It looks crisp on screen yet prints pixelated.

Text

Text lives inside a text object between `BT` and `ET`. `Tf` sets the font and size (`/F1 12 Tf`), `Td` / `Tm` position it, `Tj` shows a string of glyphs, and `TJ` shows them with per-glyph kerning. Crucially, the string in `Tj` contains **glyph codes**, decoded to real characters through the font's **Encoding** and, for searchable text, a **ToUnicode** map.

> **Common mistake:** Forgetting `/ToUnicode`. The page prints perfectly but copies out as garbage and is unreadable to search and accessibility tools.

8.5 Fonts — the number-one cause of print disasters

Font types PDF supports

Type	What it is	Print note
Type 1	Classic Adobe PostScript outline font	Legacy only — Adobe ended authoring support in Jan 2023
TrueType	Apple/MS quadratic-outline font	Very common, well supported
OpenType	Modern wrapper holding either TrueType (glyf) or PostScript (CFF) outlines	PDF 1.6+; advanced typography
Type 0 / Composite (CID-keyed)	"Big alphabet" wrapper (CIDFontType0 or CIDFontType2)	Required for >256 glyphs: CJK, large Unicode, complex scripts
Type 3	Glyphs drawn as arbitrary PDF graphics	Rare; often non-scalable — usually undesirable for print

Embedding vs. subsetting

Embedding

> The font program is stored *inside* the PDF (a FontFile stream in the font descriptor). It guarantees identical output on any viewer or RIP.

Subsetting

> Embed *only the glyphs actually used*. Shrinks the file dramatically and is the prepress default. Subset fonts get a name prefixed with **6 uppercase letters + a "+"**, e.g. `ABCDEF+Helvetica`. Spotting `XXXXXX+Name` in the font list confirms it is embedded and subset.

> **Analogy:** Embedding a font is like packing the typewriter *with* the letter. If you ship only the text and assume the recipient owns the same typewriter, they'll retype it on whatever machine they have (Courier) — different key widths, so the whole letter re-flows and no longer fits the page.

Why missing or unembedded fonts ruin print

If a font is not embedded, the viewer or RIP must **substitute**. Acrobat fakes a match using Multiple-Master substitutes (Adobe Serif MM / Adobe Sans MM); a print RIP often just drops to **Courier or Helvetica**. Substitutes have different glyph shapes and different **advance widths**, so text **reflows**: words overlap, lines re-wrap, paragraphs push off the page, and special characters or logos turn into wrong glyphs or empty boxes (□). **Print RIPs are stricter than screen viewers**, so a file can look fine on screen and still substitute on the press.

The **Standard 14 ("Base-14") fonts** (Helvetica, Times, Courier — four styles each — plus Symbol and ZapfDingbats) were historically never embedded because every PostScript device had them. For modern print you should **embed them anyway**.

> **Example — the Courier surprise:** A designer sends a brochure in a licensed display font but forgets to embed it. On their Mac it's perfect; on the shop's RIP every headline renders in Courier, lines rewrap, a two-line headline becomes three, and it collides with the photo. It previewed fine only because the designer's machine had the font installed.

> **Best practice:** Export to **PDF/X-4** (or X-1a for legacy CMYK-flat work). PDF/X and PDF/A both *require every font to be embedded* — a non-embedded font is a preflight failure, not a warning. Then preflight (Acrobat Pro, Enfocus PitStop, callas pdfToolbox) and confirm the `XXXXXX+FontName` subset prefixes are present before plating.

8.6 Colour spaces in PDF

Colour space	What it is	Print suitability
DeviceGray	One 0–1 channel (`g` / `G`)	Fine for grayscale / black-only
DeviceRGB	Red/Green/Blue (`rg` / `RG`), screen-oriented, **no defined gamut**	Poor as a final print space — looks different on every device
DeviceCMYK	Cyan/Magenta/Yellow/Key ink % (`k` / `K`), press-oriented, **no press/paper profile**	Colour undefined until you know the printing condition
ICCBased	RGB/CMYK/Gray with an **embedded ICC profile** pinning an exact appearance	The industry standard for colour-managed work
CIE-based (CalGray, CalRGB, Lab)	Device-independent reference spaces; Lab is the absolute reference	Used internally for conversions
Separation	A single named **spot** colorant (Pantone, varnish, white) + alternate space + tint transform	Real press uses a dedicated plate; others approximate
DeviceN	Several named colorants at once (multi-spot, duotones)	Same idea as Separation, N inks
Indexed	A palette into a base space	Small fixed colour sets, paletted images

For PDF/X, a single **OutputIntent** declares the intended printing condition (e.g. GRACoL2013, FOGRA39/FOGRA51, US Web Coated SWOP) — the reference everything in the file is meant to be reproduced against.

Analogy: DeviceCMYK vs. ICCBased is "add salt" vs. "add 5 g of salt." Device numbers are vague proportions that taste different in every kitchen (press); an ICC profile or OutputIntent pins the recipe to a known kitchen so the dish comes out the same everywhere.

Example — RGB black turns muddy: A logo built in DeviceRGB pure black (0,0,0) gets converted at the press into rich black (high C, M, Y and K), so crisp 100% K text would have been fuzzy and registration-sensitive instead. Fixing the colour space before plates is the prepress job.

Example — Pantone prints as process: Packaging uses a Separation colour "PANTONE 286 C," expecting a fifth plate. The job is set CMYK-only, so the RIP falls back to the alternate tint transform and the brand blue prints as a dull process approximation — the classic spot-vs-process mismatch.

Common mistake: Delivering DeviceRGB, or untagged DeviceCMYK, for print. With no defined gamut or printing condition, colour is unpredictable. Final artwork should be ICCBased or carry a PDF/X OutputIntent.

Best practice: Colour-manage explicitly — tag artwork with ICC profiles, keep spot colours as named Separation channels *only* when you genuinely want extra plates, and convert to the press's output condition (FOGRA/GRACoL) on purpose, never by accident.

Section summary:

- A PDF is a random-access **database of objects** indexed by an xref table; readers open it from the end (trailer → xref → Catalog → page tree).
- Pages are painted by a postfix **content stream** of operators — vectors (`m` / `l` / `c` / `f`), images (1×1-square + `Do`), and text (`BT…ET` , glyph codes decoded via Encoding/ToUnicode); measured in points (1 pt = 1/72 in) from a bottom-left origin.
- **Fonts are the top print risk:** always **embed and subset** (look for the `XXXXXX+Name` prefix); unembedded fonts substitute to Courier/Helvetica and reflow text — RIPs are stricter than screen viewers.
- For colour, prefer **ICCBased** over device spaces, keep **Separation** for true spot plates, and declare an **OutputIntent** so the press condition is unambiguous.
- Ship **PDF/X-4** and **preflight** before plating — it enforces font embedding and surfaces colour, transparency and resolution problems before they cost a print run.

11. Typography & Text Rendering

Text is the part of a design people actually *read*. A logo can be vague and still work; a price, a phone number, or a paragraph of legal fine print cannot. This section teaches you how text really works inside a computer and on paper — from the letters you type, to the shapes that get drawn, to the pixels and ink that finally appear. We start with the most basic and most-confused idea: the difference between a character and a glyph.

1. Characters vs glyphs — the foundation

Character

> The abstract *meaning* of a letter or symbol — the thing you type and store. The computer records it as a **Unicode code point**, a universal ID number. The capital letter A is `U+0041` everywhere on Earth.

Glyph

> The actual *drawn shape* on screen or paper. One character can be drawn many different ways depending on the font.

The relationship between characters and glyphs is **not one-to-one**, which trips up almost every beginner:

- **One character → many glyphs:** the letter "a" can be drawn as a normal "a", a small-cap "A", or a fancy swash alternate.
- **Many characters → one glyph:** the two characters "f" and "i" often join into a single *ligature* shape "fi".
- **One character → zero glyphs:** a combining accent mark may have no standalone shape; it just modifies its neighbour.

Inside the font, a table called the **cmap** ("character map") translates each Unicode code point into a default glyph. Then the font's layout features can substitute or reposition glyphs from there.

Common Mistake — Assuming "the font has the letter" is enough. A code point can be mapped but the font may have no glyph for it — you then see `.notdef`, the "tofu" box □. A storefront brand that uses an emoji or a rare currency symbol in a font that lacks it will print a box, not the symbol.

2. Font formats — what those file extensions really mean

A font is a file full of glyph outlines plus metric tables. Over 40 years the industry settled on a family of formats:

Format	Outline math	Notes
PostScript Type 1 (.pfb/.pfm)	Cubic Bézier	Adobe, 1984. The deprecated ancestor — **end-of-life in Adobe apps since Jan 2023**; these files no longer render. You'll only meet them in old archives.
TrueType (.ttf)	Quadratic Bézier (B-spline)	Apple + Microsoft, late 1980s. Famous for powerful "bytecode" hinting.
OpenType (.otf or .ttf)	Either type	Adobe + Microsoft, 1996. The modern **umbrella** format.
WOFF / WOFF2	Either	Compressed OpenType for the web. WOFF2 uses Brotli compression, ~30% smaller than WOFF.

Bézier curve

> A smooth curve defined by a few "control points" — the math used to describe every letter outline. *Cubic* curves use more control points (smoother, heavier); *quadratic* use fewer (lighter, faster).

The key teaching point about OpenType: OpenType is a *container*, not a competitor to TrueType. An OpenType font wraps *either* outline type: TrueType/quadratic outlines (stored in a `glyf` table, usually saved as `.ttf`) *or* PostScript/CFF cubic outlines (Compact Font Format, usually saved as `.otf`). So:

> **Key Takeaway —** `.ttf` vs `.otf` tells you the *outline type inside*, not "TrueType vs OpenType." Both files *are* OpenType. OpenType added Unicode (65,536+ glyphs, vs the old 256-glyph limit), one cross-platform file for Mac and Windows, and the advanced layout engine described next.

Variable fonts — one file, infinite weights

Added to OpenType in v1.8 (September 2016), a **variable font** stores a whole continuous "design space" in a single file. Instead of shipping 12 separate files (Light, Regular, Bold, …), one file interpolates between master designs along named **axes** (lowercase 4-character tags):

- `wght` — Weight (1–1000)
- `wdth` — Width (percentage)
- `ital` — Italic (0 or 1)
- `slnt` — Slant (degrees, typically −90 to 90)
- `opsz` — Optical Size (in points)

Custom axes use UPPERCASE tags. Adoption climbed from ~11% of web pages in 2020 to ~33–34% in 2024 because one small file replaces dozens. For PF360's storefront, a variable font lets a theme nudge heading weight without downloading another file — fewer requests, faster pages.

3. The OpenType layout engine — how features work

Two tables do the clever typography, both keyed by 4-character *feature tags*:

GSUB (Glyph Substitution)

> Decides *which* glyphs to use — swap "f"+"i" for the "fi" ligature, swap digits for small caps, etc.

GPOS (Glyph Positioning)

> Decides *where* glyphs sit — kerning pairs, stacking accents onto letters.

The text shaper runs **GSUB first, then GPOS** (pick the glyphs, then position them). Useful tags:

- **GSUB:** `liga` standard ligatures (fi, fl — on by default), `dlig` discretionary ligatures (st, ct — off by default), `smcp` small caps, `onum` / `lnum` oldstyle vs lining figures, `frac` fractions, `salt` / `ssNN` stylistic alternates/sets, `swsh` swashes.
- **GPOS:** `kern` kerning, `mark` accent placement. Modern kerning lives in GPOS, not the old legacy `kern` table.

4. From outlines to pixels — the rendering pipeline

```
[glyph outline]  ->  scale to size  ->  apply HINTS
  (Bézier pts)        (ppem grid)        (snap to grid)
       |                                      |
       v                                      v
  RASTERIZE  ->  anti-alias / subpixel  ->  composite
(fill to pixels)   (ClearType RGB ~3x)      (on screen)
```

em / UPM (units per em)

> The font's internal design grid. All measurements are in these units — typically 1000 (PostScript/CFF) or 2048 (TrueType) — then scaled by your point size.

ppem (pixels per em)

> How many pixels tall the em box becomes at the chosen size. Small ppem = few pixels = hard to render cleanly.

Rasterize

> Convert the vector outline into a grid of filled pixels.

Hinting

Hinting means instructions baked into the font that gently distort outlines to snap onto the pixel grid at small sizes / low resolution, so stems stay even and letters stay readable. TrueType hinting is a stack-based assembly *bytecode* run by an interpreter (a tiny virtual machine), stored in `glyf` / `prep` / `fpgm` tables — fonts like Verdana have thousands of hand-tuned lines. PostScript/CFF hinting is lighter and more automatic.

Common Mistake — Worrying about hinting for *print*. Hinting matters below ~24px on screens and low-DPI displays. At print resolution the rasterizer has so many pixels that hinting is nearly irrelevant. PF360's `pdf-service` (PDFKit + sharp at port 4000) renders at high resolution — don't burn time tuning hints there.

5. Typographic geometry — the text-tool vocabulary

```
ascender ---  b   d       l    <- tops above x-height
cap-height-   B   D   H        <- top of capitals
x-height ---  x   o   n  e     <- top of lowercase
baseline ===  x o n e B D b d l <- glyphs sit here
descender --      g   p   y    <- tails below baseline
```

Baseline

> The invisible line glyphs rest on.

x-height

> Height of a lowercase "x". A tall x-height reads better at small sizes.

Cap-height

> Top of capital letters (usually a touch below the ascender height).

Ascender / Descender

> Parts rising above x-height (b, d, h, k, l) / dropping below baseline (g, j, p, q, y).

Counter, bowl, overshoot

> The enclosed space in "o"; the round stroke of "b"; the tiny amount round glyphs exceed the lines so they look optically equal to flat ones.

The classic spacing trio (clear this up once and for all)

Leading (line-spacing)

> **Vertical** distance from one baseline to the next. Named after the lead metal strips printers slid between lines. Rule of thumb: point size + 2–4 pt, or 120–150% of size (12 pt body → ~14.4–18 pt). Longer lines need *more* leading.

Tracking (letter-spacing)

> **Uniform** horizontal space across a whole run/word/line. Tighten big display headlines; add some to CAPS and small caps; never heavily track lowercase body text.

Kerning

> Space between one *specific pair* of letters — fixing optical gaps in "AV", "To", "WA", "Ty", "Yo", "LT". *Metric* kerning uses the font's GPOS pairs; *optical* kerning is computed by the app.

Workflow order: leading → tracking → kerning (macro to micro).

Common Mistake — Calling tracking "kerning." Kerning is *pairwise only*. A control that spaces the whole line evenly is tracking, no matter what it's labelled.

6. Print legibility — the numbers that matter

- **Body text:** 9–12 pt (10–11 pt is common). Around 8 pt is the floor where reading speed drops.
- **Line length (measure):** 50–75 characters per line, 66 ideal. Too long and the eye loses the line on return.
- **Leading:** 120–150% of size. Size, measure, and leading are coupled — change one, retune the others.
- **Black ink:** body/small text should print as **100% K only** (single-channel black), not rich 4-color black — registration misalignment fuzzes small 4-color text. Reserve rich black for large display type.
- Avoid thin serifs and hairlines in reverse (white on dark) — ink gain fills the counters. Keep legal fine print to ~5–6 pt minimum.
- **Embed or outline all fonts** in the print PDF (PDF/X) so the print engine (RIP) never substitutes a wrong font. Ligatures and kerning must survive flattening.

Real-world case — A PF360 customer types a phone number in 7 pt black using a default rich-black fill. On the proof it looks crisp (screen is RGB). At the press, the cyan/magenta/yellow/black plates are a hair out of register and the number prints fuzzy and ghosted. The fix: 100% K only, bump to 9 pt. Because the platform currently has **no RGB→CMYK conversion and no preflight**, nothing catches this automatically — it must be caught by design guidance and review.

7. PF360 designer text tool — the controls users see

The design studio (`designer/`) is built on Fabric.js. The text controls live in `designer/src/views/Editor/CanvasRight/Components/ElementText.vue` (toolbar) and `ElementStylePanel/TextboxStylePanel.vue` (style panel), with config lists in `src/configs/texts.ts` (`FontSizeLibs`, `LineHeightLibs`, `CharSpaceLibs`). Visible controls: font color picker; Font size +/−; Bold / Italic / Underline; horizontal align Left/Center/Right; vertical align Top/Middle/Bottom; and a **"Spacing"** row exposing line height and character spacing.

UI label / control	Typographic term	Implementation note
Spacing → line height	Leading	Fabric `lineHeight` is a *unitless multiplier* (default 1.16) — relative leading, not absolute points.
Spacing → char spacing	Tracking	Fabric `charSpacing` is measured in **1/1000 em**, not px or pt; PF360 steps it ±10. Per-character overrides are cleared (`clearPerCharCharSpacing`) so the global value applies uniformly.
(none)	Kerning	No true pairwise kerning UI — Fabric kerns by font metrics only. The "Spacing" control is *tracking*.

Common Mistake / UX flag — Per the project's plain-language rule, a single "Spacing" control hides two different concepts (leading and tracking) and risks being mistaken for kerning, which PF360 doesn't expose at all. Clearer labels would be "Line spacing" and "Letter spacing" — never "Kerning," since no pairwise control exists.

Best Practice — When preparing print-bound text in the designer: keep body text 9 pt+, set line spacing around 1.3–1.5, use letter spacing sparingly (tighten headlines, loosen CAPS), and choose a font with a generous x-height for small labels. Because there's no automated preflight or font-embedding guarantee in the order-to-production spine yet, treat these as manual checks before a job is sent to print.

Key takeaways —

- **Characters are meaning, glyphs are shapes** — the mapping is not 1:1; a missing glyph shows a □ tofu box.
- **OpenType is the umbrella format.** `.ttf` vs `.otf` = which outline math is inside (quadratic/glyf vs cubic/CFF), not "TrueType vs OpenType." PostScript Type 1 is dead since 2023.
- **Variable fonts** pack many weights/widths into one file along axes (`wght` , `wdth` , `ital` , `slnt` , `opsz`) — fewer downloads for the storefront.
- **GSUB picks glyphs, GPOS positions them;** rendering goes outline → scale → hint → rasterize → anti-alias. Hinting is a screen/low-DPI concern, not a print one.
- **Leading is vertical, tracking is uniform horizontal, kerning is pairwise.** Don't confuse them; work macro to micro.
- **For print:** 9–12 pt body, 50–75 chars/line, 120–150% leading, 100% K for small black text, embed fonts. PF360 has no auto RGB→CMYK or preflight — these are manual checks.
- **In the PF360 designer,** "Spacing" = Fabric `lineHeight` (unitless leading multiplier) + `charSpacing` (tracking in 1/1000 em); there is no kerning control — label accordingly.

12. PDF/X, Output Intent & Page Boxes — The Print-Ready Target

By now you understand color, ink, and presses. This section is about the **file** you actually hand to the printer. A file that looks perfect on your screen can still print wrong — fonts go missing, colors shift, shadows turn into white boxes, edges show white slivers. The solution is a special, locked-down kind of PDF built specifically for printing: **PDF/X**. We'll learn what it is, why transparency is the great dividing line between its flavors, what the "output intent" is, and how the five page boxes tell the press exactly where to cut.

9.1 Why PDF/X Exists — The "Blind Exchange" Idea

A normal PDF is built for *viewing* on a screen. A print PDF must be something stricter: a **blind, deterministic exchange file**. "Blind exchange" means the printer can open it and run it with *no questions asked* — no fonts to chase down, no color to guess at, no risky features to trip over.

PDF/X

> Stands for **"PDF for eXchange."** It is a *restricted subset* of the normal PDF format, standardized by ISO (the umbrella standard is **ISO 15930**). Each "flavor" — X-1a, X-3, X-4 — is a separate part of that standard.

Restriction

> PDF/X *forbids* things that make printing unpredictable (missing fonts, unmanaged RGB, JavaScript, encryption, audio/video) and *requires* things the printer needs (all fonts embedded, a declared output intent).

Key takeaway: The whole philosophy of PDF/X is one sentence: **"Anything that can vary at the printer is removed; anything the printer needs to know is embedded."**

Analogy: A normal PDF is like flat-pack furniture that ships with some screws missing and a note saying "instructions available online" (fonts to fetch, color to guess). PDF/X is the same kit with every screw bolted in and the manual taped inside the box — the printer just opens it and runs it.

9.2 The Transparency Model & Flattening — The Great Divide

The single biggest difference between the PDF/X flavors is how they handle **transparency**. Transparency means any artwork that lets what's underneath show through: drop shadows, glows, soft edges, blend modes, and reduced-opacity objects.

Live (native) transparency

> The PDF stores the *actual* transparency instructions — the blend modes, opacity values, drop shadows, and soft masks. The RIP (the press's processor) resolves them at output time, at the device's full resolution. This was introduced with PDF 1.4 (Acrobat 5 / InDesign 2, around 2001).

Flattening

> A pre-processing step that *destroys* live transparency. It carves overlapping transparent artwork into a mosaic of fully opaque pieces — some kept as vector, some turned into raster images — that *look* like the original blend but contain no transparency anymore.

Why flattening was ever needed

The original PostScript imaging model — and the older RIPs built on it — **has no concept of transparency at all**. Every object is opaque. A RIP that only understands PostScript Level 3 cannot interpret a blend mode, so any transparency must be "baked in" *before* it reaches that older equipment. **PDF/X-1a and X-3 require flattened files** precisely so they run on this older, opaque-only generation of machines.

How the flattener works

Atomic regions

> The discrete tiles the flattener slices artwork into wherever transparent objects overlap. Each tile is reduced to a single, fully opaque appearance (a vector fill or a raster patch).

Rasters/Vectors balance

> A slider plus a flattener resolution setting controls how much becomes pixels vs. vectors. Typical values: **line art & text at 1200 ppi**, **gradients & meshes at 300 ppi**.

Clip Complex Regions

> An option that forces region boundaries to land on object edges, reducing visible seams.

Flattening artifacts — the failure modes

- **Stitching / hairline artifacts** — faint white or black lines tracing the boundaries between atomic regions, caused by anti-aliasing where a rasterized tile meets a vector tile. Often visible on screen and *sometimes* in print.
- **Text rasterized or outlined** — small type sitting over a shadow can get rasterized at flattener resolution, turning slightly fuzzy or heavy.
- **Color shifts at overlaps** — spot colors near transparency can be converted to process builds, or RGB→CMYK can shift inconsistently across a region.
- **Broken overprint / knockout** — flattening changes overprint relationships, so objects print over or under each other incorrectly.

Example: A designer exported a brochure with soft drop shadows as **PDF/X-1a** to an old RIP. The flattener rasterized each shadow region at low resolution and left a faint **white rectangle** around every photo. Re-exporting as **PDF/X-4** to a modern RIP resolved the transparency live — the shadows printed clean.

The modern fix: PDF/X-4 keeps transparency LIVE — no flattening at all. A modern Adobe PDF Print Engine (APPE) RIP resolves transparency natively at device resolution. The result is smaller files, no stitching seams, sharper text, more predictable color, and "late binding" of color. This is *the* reason X-4 is the modern default.

Key takeaway: Flattening exists only to feed **old, transparency-blind RIPs**. It can introduce visible artifacts. Modern presses don't need it — keep transparency live with PDF/X-4.

9.3 The Three Standards — Differences and When to Use Each

Feature	PDF/X-1a	PDF/X-3	PDF/X-4
ISO part	15930-1 / -4 (2001–03)	15930-3 / -6 (2002–03)	**15930-7 (2008/2010)**
Base PDF version	1.3 / 1.4	1.3 / 1.4	**1.6**
Color allowed	**CMYK + spot + gray ONLY** (no RGB)	CMYK, spot, gray **+ calibrated RGB, CIELAB, ICC-tagged**	CMYK, gray, **RGB**, spot, ICC-managed
Color management	**None** (blind, already device CMYK)	Color-managed (ICC)	Fully color-managed (ICC), late binding
Transparency	**Forbidden — must flatten**	**Forbidden — must flatten**	**Live transparency permitted**
Layers (OCGs)	No	No	Yes (optional)
Fonts	All embedded (required)	All embedded (required)	All embedded (required)
Output intent	Required	Required	Required

In plain language:

- **X-1a** — "Everything is already CMYK and flat. No color management, no surprises." The 20-year-old safe-but-dumb standard: total predictability, zero flexibility. Use it *only* when a printer explicitly demands it or for a legacy RIP.
- **X-3** — "I can send RGB/Lab images and let an ICC profile convert them, but transparency is still flattened." Historically common in **Europe**; a transitional standard.

- **X-4** — "Live transparency *plus* ICC color management." **The modern recommended default** for virtually all commercial and digital print. Required whenever the design uses real transparency (shadows, glows, blend modes) and you want it resolved at the RIP.

(There's also **PDF/X-5** = X-4 plus *external* referenced profiles/graphics — niche — and **PDF/X-6**, the newest part based on PDF 2.0.) Industry consensus from the PDF Association: **stop using X-1a; default to X-4.**

> **Common mistake:** Defaulting to **PDF/X-1a in 2026**. It forces flattening, which causes the very shadow/transparency artifacts that X-4 avoids — and it throws away the chance for modern, late-binding color management.

9.4 Output Intent — The Embedded "Target Press Condition"

The **output intent** is a required PDF/X entry that declares *the exact printing condition the file was prepared for* — in plain words, "this file is built to look right on *this* paper / ink / press combination."

What it contains:

Subtype `GTS_PDFX`

> The only output-intent subtype valid for PDF/X.

OutputConditionIdentifier

> The *reference name* of a standard print condition from the ICC Characterization Data registry — e.g., `FOGRA39`, `FOGRA51`, `CGATS21-CRPC6` (GRACoL), `CGATS TR 001 SWOP`.

DestOutputProfile

> The **embedded ICC profile** describing that condition — e.g., *Coated FOGRA39 (ISO 12647-2:2004)*, *GRACoL2013 CRPC6*, *US Web Coated (SWOP) v2*.

OutputCondition / RegistryName / Info

> Human-readable text describing the condition.

The conditions it points at (with their standard total-ink-limit figures):

- **FOGRA39** — coated stock, older European ISO 12647-2:2004; total area coverage (TAC) **~330%**.
- **FOGRA51 / FOGRA52** — newer ISO 12647-2:2013 (PSO Coated v3 / uncoated v3).
- **GRACoL 2013 (CRPC6)** — US premium coated; TAC **~310%**.
- **SWOP (CRPC5 / CRPC3)** — US web/publication; TAC **~300%**.

Why it matters: the output intent is the file's color *contract*. The RIP uses it as the destination profile for any color conversion and as the assumed proofing target. **Without it, the file is not valid PDF/X.** Note that PDF/X allows *only* printer (output-device) profiles here — unlike PDF/A, it never permits monitor profiles.

Example: A US shop received a file carrying a **FOGRA39 (European coated)** output intent and ran it on a **GRACoL** press without re-profiling. The corporate blue came out noticeably cooler. The output intent is the contract — a mismatched press condition means wrong color.

Analogy: Output intent is the recipe's oven setting. The same cake batter (your colors) needs to know whether it's baking on glossy coated stock at one "temperature" (FOGRA39) or uncoated at another. The output intent tells the press exactly which oven it was baked for.

9.5 Page Geometry Boxes — Where the Blade Cuts

Every PDF page defines a set of nested rectangles called **page boxes**. The PDF spec *requires only the MediaBox*; the other four are optional in general PDF — but a real print PDF needs the TrimBox and BleedBox.

MediaBox

> The **only mandatory** box: the full physical sheet/canvas. In prepress it's deliberately *oversized* to hold bleed, crop marks, and slug/registration info. Everything else must fit inside it.

CropBox

> The region a viewer displays/prints; **defaults to the MediaBox** if absent. **Avoid it in prepress** — it can hide bleed and confuse imposition.

BleedBox

> The clip boundary for production output; extends *beyond* the TrimBox so ink runs past the cut line. Must be ≥ TrimBox.

TrimBox

> The **finished page size after cutting** (the A4 / Letter / business-card size in the customer's hand). **The single most important box for print** — imposition and finishing equipment read it to know exactly where to cut.

ArtBox

> The meaningful-content extent; today often used to mark the **safe / live area** — keep text and logos inside it.

The containment rules: `MediaBox ⊇ BleedBox ⊇ TrimBox ⊇ ArtBox`. All of CropBox, BleedBox, TrimBox, and ArtBox must fit inside the MediaBox.

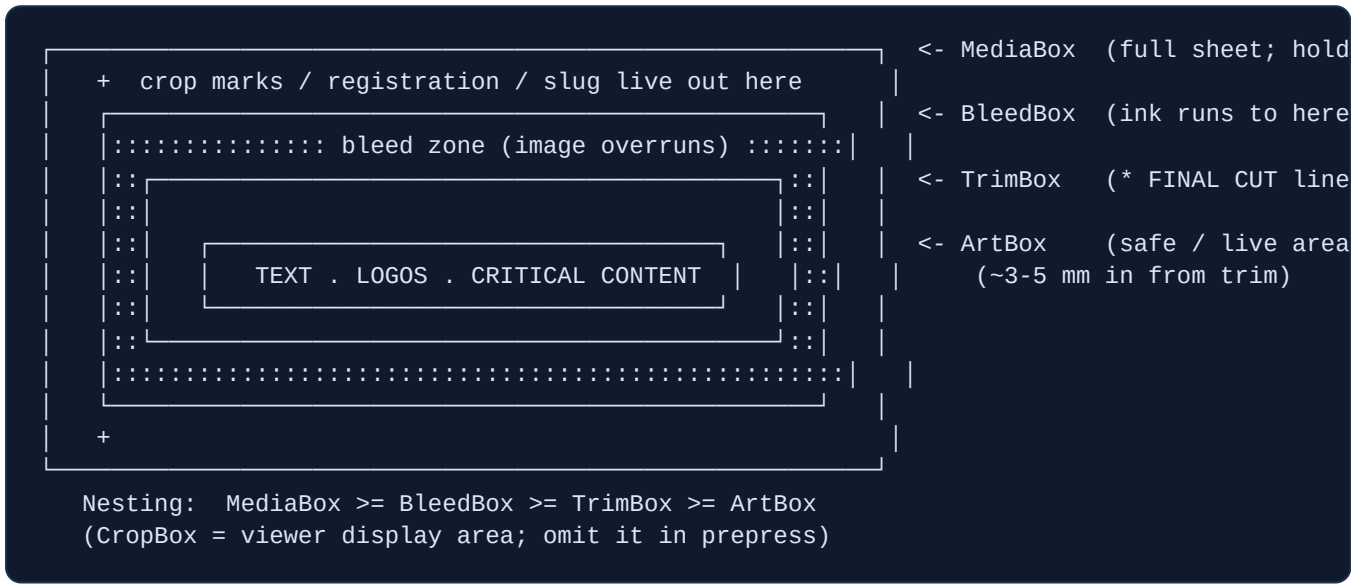
Bleed is not Trim. Trim is where the blade actually cuts; **bleed** is the extra image area pushed *past* the cut so a slightly-off blade never exposes white paper.

Canonical numbers

- Standard bleed = **3 mm** (Europe / ISO) up to **0.125 in (⅛ in ≈ 3.175 mm)** (US); typical range **3–5 mm ≈ 9–14 pt**. Large-format jobs may use 5 mm or more.
- Safety margin (keep critical content inside trim): typically **3–5 mm / ⅛ in** in from the trim line.
- PDF coordinates are in **points (1 pt = 1/72 in)**; so 3 mm ≈ **8.5 pt** and ⅛ in = **9 pt**.

What each standard requires

- **PDF/X-1a & X-3:** require **MediaBox + TrimBox + BleedBox** (TrimBox mandatory; BleedBox present when there is bleed).
- **PDF/X-4:** requires **MediaBox + (TrimBox *OR* ArtBox)** — and a page **must not have both** TrimBox and ArtBox. TrimBox is the normal choice.



Example: A 4×6 postcard was built with the background stopping *exactly* at trim — no BleedBox. The guillotine drifted about 1 mm, and thousands of cards shipped with a thin **white edge** on one side. The only fix was a reprint, with the background extended **3 mm past TrimBox into the BleedBox**.

Analogy: The page boxes are a tailor's pattern. TrimBox is the finished garment's seam line (where you cut); BleedBox is the extra fabric past the seam so the pattern never runs short; ArtBox/safe area is the "keep the embroidery away from the edge" zone; and MediaBox is the whole bolt of cloth on the table.

Best practice: Always define **MediaBox + TrimBox + BleedBox**. Set bleed to **3 mm (or ⅛ in)**, keep critical content **3–5 mm inside trim** (the ArtBox/safe area), and **do not set a CropBox** in prepress files.

9.6 Putting It Together — The Shop-Ready Checklist

Best practice: Default to **PDF/X-4** with the printer's correct **output intent** (ask the shop: FOGRA51 / PSO Coated v3 in Europe, GRACoL 2013 / CRPC6 in the US). Keep transparency live and let the modern RIP flatten only if it must.

Best practice: Embed all fonts and preflight before sending. Use Acrobat Preflight, Enfocus PitStop, or callas pdfToolbox to confirm the PDF/X flavor, the output intent, the total ink (TAC), and the box geometry. Never trust a blind "Save As PDF/X."

Common mistake: Exporting a plain PDF instead of PDF/X — fonts subset incorrectly, RGB images sneak through, and the printer's preflight bounces the job. The other classic is building to trim with **no bleed / no BleedBox** (or relying on CropBox instead of TrimBox), so imposition can't find the true page size and white edges appear after cutting.

Section summary:

- **PDF/X** is a locked-down, ISO-standard subset of PDF for "blind exchange": risky things removed (missing fonts, unmanaged RGB), needed things embedded (fonts + output intent).
- **Transparency is the dividing line:** X-1a and X-3 forbid live transparency and require flattening (which can cause stitching/text/color artifacts); **X-4 keeps it live** and is the modern default.
- **X-1a** = CMYK-only, no color management (legacy); **X-3** = ICC/RGB but flattened (transitional, European); **X-4** = live transparency + ICC color management (use this).
- **Output intent** is the embedded ICC profile naming the target press condition (FOGRA39 ~330% TAC, GRACoL2013/CRPC6 ~310%, SWOP ~300%) — without it the file is not valid PDF/X.
- **Page boxes** nest as MediaBox ⊇ BleedBox ⊇ TrimBox ⊇ ArtBox; TrimBox is the cut line (most important), bleed runs ~3 mm past it, and a print PDF must define MediaBox + TrimBox + BleedBox.

13. Preflight: Validating a File Before It Prints

Preflight is the single cheapest piece of insurance in the entire print workflow. It is an automated technical inspection of a print file — almost always a **PDF** (Portable Document Format, the universal "frozen" page format that presses use) — performed *before* the file goes to plate or press. Its one job: confirm the file meets every requirement for a clean print run, and if it doesn't, say *exactly* what is wrong and where.

The name is borrowed from aviation. Pilots run a pre-flight checklist on the ground because you cannot pull over and fix a problem at 30,000 feet. Print is the same: fixing a file on the desktop is free; fixing it after 5,000 sheets have printed wrong is not.

Analogy: Preflight is a pre-flight checklist for an aircraft. Pilots verify everything *before* takeoff because mid-air is the wrong place to discover a fault. You validate a file *before* plates are made, not after the run is on the floor.

10.1 What preflight is — and why it saves money

Preflight is a **gate, not a guess**. The file is run against a defined ruleset called a **preflight profile** (more on that below). It either **passes**, or it produces a **report** listing every failure and pointing to the offending object.

Why it saves money is simple arithmetic of *when* you catch the error:

- Caught at the desktop: costs **minutes** and is free to fix.
- Caught on press: costs **reprints, wasted paper and ink, blown deadlines, and an angry customer**.

A single missed RGB logo, a missing bleed, or one over-inked black can scrap an entire run.

Key takeaway: Preflight is a pass/fail gate that catches mechanical file errors when they are free to fix (the desktop) instead of when they are catastrophic to fix (on press).

10.2 What preflight actually checks

Before the list, two terms that every check leans on:

CMYK

> The four printing inks — Cyan, Magenta, Yellow, blacK — that presses physically lay down. Everything must end up here (plus any named spot inks).

RGB

> Red/Green/Blue — the *screen* color model. It cannot be printed directly; it must be converted to CMYK, and the conversion shifts colors.

1. Resolution / effective dpi

Resolution is how many dots (pixels) fit per inch — **dpi** (dots per inch). The standard for photos is **300 dpi at final print size** for sheetfed/offset; ~150–200 dpi is fine for large-format viewed at distance; line art wants **1200 dpi**.

The trap is **effective resolution**: preflight reads the dpi *at the placed/scaled size*, not the raw pixel count. Scaling an image *up* in the layout silently degrades it.

```
300 dpi image scaled to:
  100% -> 300 dpi effective   (perfect)
  200% -> 150 dpi effective   (half - risky)
  600% ->  50 dpi effective   (junk - will print as mush)
```

Example: A 600×400 px web JPEG dragged to fill an A4 cover lands at roughly 40 dpi effective. It looks fine on the laptop and prints as visible mush. The effective-resolution check flags it instantly.

2. Color space — stray RGB and unexpected spot colors

Presses print CMYK plus named **spot** inks. Any stray **RGB**, **Lab**, or untagged object shifts unpredictably — bright blues, greens, and oranges go dull or wrong. Preflight also flags **unexpected spot/Pantone colors**: each named spot is an extra plate and extra cost. A logo that should be CMYK but carries a stray "PANTONE 286 C" either adds a plate or converts badly.

3. Embedded fonts

A font is **embedded** when the file carries the actual letter shapes. If a font is not embedded, the RIP (the press's *Raster Image Processor* — the software that turns the PDF into dots) substitutes a different typeface, reflowing and breaking the layout. **All fonts must be embedded or outlined to curves.**

4. Bleed presence

Bleed is background artwork extended *past* the trim line so that when the cutting blade drifts (it drifts ~1–2 mm), no white slivers appear. Standard bleed is **0.125 in (1/8") = 3 mm** on every side (Europe/some printers ask 5 mm). The **safe / quiet zone** keeps text and critical elements at least **0.125 in** (¼" for high-stakes jobs) *inside* the trim. Preflight checks the art actually extends into the bleed and that a BleedBox is defined.

5. Ink limits — Total Area Coverage (TAC)

TAC (also called TIC) is the sum of C+M+Y+K percentages in the heaviest area; the maximum possible is **400%**. Too much ink won't dry or trap, causing smearing, set-off, picking, sticking sheets, and paper curl. Standard ceilings by process and stock:

Process / stock	Max TAC
Sheetfed offset, coated	320–340%
Heatset web (magazines)	300–320%
SWOP (US web publication)	300%
Uncoated / newsprint (coldset web)	240–260% (most restrictive)

Common mistake: Using "registration black" — 100/100/100/100 = 400% — for art or text. That value is only for crop and registration marks. On a coldset newsprint run (limit ~240%) it never dries, sets off onto the next sheet, and smears the whole run.

6. Overprint settings

Knockout (the default): the top object removes the ink beneath it. **Overprint**: the top object prints *on top of* what's below, mixing inks. Two failures preflight catches:

- **Overprint white** — white "ink" doesn't exist on most presses, so an overprinting white object simply **vanishes**.
- **Unintended overprint mixing** — a knockout object accidentally set to overprint mixes into the background (blue over yellow turns green).

Conversely, **small black text should overprint** so press misregistration doesn't leave white halos around the letters.

Example: A white company name left set to "overprint" looks fine on screen, then prints as a blank box on 10,000 brochures because there is no white ink to overprint with. The overprint-white check catches it for free.

7. Minimum line weight / hairlines

A **hairline** is a stroke thinner than about **0.25 pt**. These can drop out, print inconsistently, or break up. Preflight flags them and bumps them to a safe minimum (often 0.25–0.5 pt). It also flags thin rules and small *reversed* type (light text on dark) built from more than one ink, which smear or fill in unless registration is tight.

8. Transparency / flattening

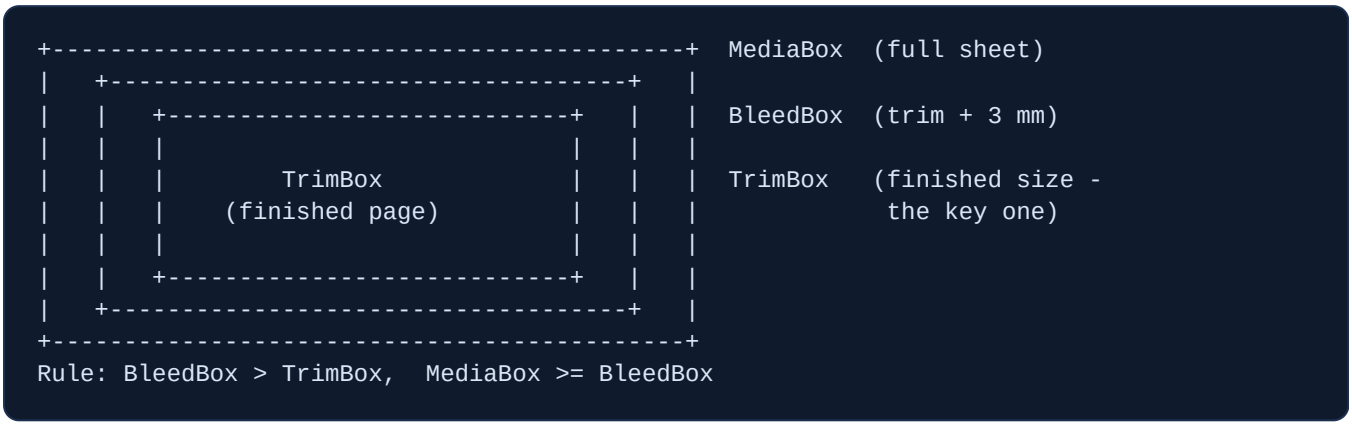
Transparency means live effects — drop shadows, blends, opacity. **Flattening** bakes those effects into plain opaque artwork the press can handle. PDF/X-1a **requires flattening**; PDF/X-4 keeps transparency **live** and lets the RIP handle it. Bad flattening leaves artifacts preflight tries to prevent: thin white "stitching" lines, color/text shifts, fattened text, and stray white rectangles inside artwork.

9. Image compression / encoding

Preflight flags destructive or over-aggressive **JPEG compression** (blocky artifacts), wrong encoding, and oversized images, recompressing cleanly where it can.

10. Page geometry — PDF page boxes

A PDF carries several nested "boxes" that define sizes. Getting them right is half of being print-ready:



MediaBox

> The full sheet size.

TrimBox

> The finished page size after cutting — the most important box; it defines the actual print size.

BleedBox

> Trim plus bleed (3–5 mm larger than TrimBox).

A missing or wrong TrimBox or BleedBox is a preflight failure.

Key takeaway: Preflight checks resolution (effective, not raw), color space, embedded fonts, bleed, ink limit (TAC), overprint, hairlines, transparency, compression, and page boxes — the repeatable, mechanical faults that ruin a run.

10.3 Preflight profiles & PDF/X compliance

A **preflight profile** is a saved bundle of rules — for example: "error if RGB present", "warn if image < 300 dpi", "error if TAC > 320%", "require all fonts embedded". Printers keep one profile per press/stock combination and run every incoming file through it.

PDF/X is the ISO "print-safe" subset of PDF; preflight verifies compliance with it. The main variants:

Variant	Color	Transparency	Notes
PDF/X-1a	CMYK + spot only, *no RGB*	Must be flattened	Fonts embedded, no JS/encryption/forms, output intent required. Safest, most universal.
PDF/X-3	Adds ICC-managed color; RGB allowed if tagged with an ICC profile	Flattened	RIP converts at output. Common in Europe.
PDF/X-4	ICC-managed color	Live (no flattening)	OpenType fonts. The modern, preferred standard today.

Every PDF/X file requires an **output intent**: an embedded or referenced **ICC profile** (an *International Color Consortium* color description) or named printing condition — such as **FOGRA39/FOGRA51** in Europe or **GRACoL/SWOP** in the US — that tells the RIP the intended printing condition so it interprets colors correctly instead of guessing.

10.4 How the tools work

- **Adobe Acrobat Pro preflight** (*Tools → Print Production → Preflight*) ships press-ready profiles, runs PDF/X validation, reports every issue with **clickable jump-points** to the offending object, and offers built-in **fixups**.
- **Enfocus PitStop Pro** — the prepress industry standard, an Acrobat plug-in — pairs a **Preflight Profile** (the check ruleset) with an **Action List** (a recorded sequence of edits/fixes). An "**Allow fixes**" toggle decides whether problems are auto-corrected or only logged. It produces a clear report with **visual highlights and clickable jumps**. **PitStop Server** automates the same via hot folders.
- Others: **callas pdfToolbox** (the engine behind many systems), **Markzware FlightCheck**, and many free online DPI/bleed/CMYK checkers.

Tools run in **two modes**: *report-only* (flag it, let a human decide) versus *auto-fixup* (correct it in place). Production shops automate with hot folders so files are checked and corrected without manual touch.

10.5 Fixups — automatic correction

Many problems can be repaired automatically:

- **RGB → CMYK** conversion via the destination ICC profile.
- **Downsample** over-resolution images; **flag** under-resolution (you cannot invent pixels — a 72 dpi photo can't be made sharp).
- **Flatten transparency** for PDF/X-1a targets.
- **Add/define bleed** by extending edge content; set TrimBox/BleedBox.
- **Embed/subset fonts**; outline thin text if needed.
- **Bump hairlines** to a minimum stroke weight.

- **Reduce ink** to the max-TAC limit (often using GCR/UCR to swap CMY under-color for K).
- **Fix overprint** (turn off overprint-white, set black text to overprint).
- **Add output intent / convert to PDF/X.**

Common mistake: Trusting auto-fixup blindly. Fixups have limits — a stray spot color converted automatically may shift, and a low-res image can't be rescued. Risky fixes need a human and customer sign-off.

10.6 Reporting issues to non-technical users (the UX last mile)

A store owner uploading artwork must **never** see "RGB object in TrimBox, effective res 96 ppi, TAC 367%." That is developer output and it reads as "broken." Every check must be translated into **plain language + a clear fix**:

Technical problem	What the customer should read
Low effective resolution	"This photo will print blurry. Use a sharper version (at least 300 dpi at this size)."
RGB content	"Your colors may shift when printed. We'll convert them for print — preview to confirm."
No bleed	"Add 1/8 inch of background past the edge so there's no white border after trimming." (offer auto-fix)
Text in quiet zone	"Move text in a little so it isn't cut off at the edge."
Missing font	"A font isn't included. Outline your text or embed the font."

Best practice: Show a **visual proof with the problem area highlighted**, a one-line plain explanation, the consequence ("will look blurry"), and a recovery action ("upload a higher-res image" / "Auto-fix bleed"). Offer safe auto-fixes; require sign-off for risky ones. Never reject silently; never dump a raw error code.

Analogy: Preflight is spell-check and grammar-check for a print file. It catches the mechanical, repeatable mistakes automatically (and can auto-fix many), but it can't judge whether the *content* is right — a human still has to proof the message.

10.7 A realistic preflight checklist

1. **File format:** export as PDF/X-1a (safe CMYK) or PDF/X-4 (transparency/ICC); output intent set (FOGRA/GRACoL/SWOP).
2. **Page boxes:** correct TrimBox (finished size) + BleedBox (trim + 3 mm).

3. **Bleed:** artwork extends 0.125 in / 3 mm past trim on all sides.
4. **Safe zone:** text/logos ≥ 0.125 in inside trim.
5. **Resolution:** raster ≥ 300 dpi effective at placed size (line art 1200 dpi); no upscaled images.
6. **Color:** CMYK/spot only (no stray RGB); spot colors intentional and named correctly.
7. **Fonts:** 100% embedded or outlined.
8. **Ink limit (TAC):** within stock ceiling — 240–260% uncoated/news, 300% SWOP, 320–340% sheetfed coated.
9. **Black recipes:** large blacks use rich black (~60/40/40/100 or shop recipe); *never* 400% registration black in art/text; small black text = 100% K only, set to overprint.
10. **Overprint:** no overprinting white; black text overprints; no accidental overprint mixing.
11. **Hairlines:** no strokes < 0.25 pt.
12. **Transparency:** flattened (X-1a) or correctly live (X-4); no flatten artifacts.
13. **Compression:** no destructive JPEG artifacts; images sensibly sized.
14. **Misc:** single page per page (no spreads unless requested), correct orientation, all links/images present, no security/encryption, and a final human content review (preflight can't catch typos).

Best practice: Preflight *at the source and at the printer*. Designers run a profile before sending; the shop re-runs its own press-specific profile on intake. The earliest catch is always the cheapest.

Common mistake: Trusting raw dpi over effective dpi. A "300 dpi" image scaled 300% in the layout is really 100 dpi — scaling silently degrades it, and only the effective-resolution check reveals the truth.

Section summary:

- Preflight is an automated pass/fail gate on the print PDF that catches mechanical errors when they're free to fix, not when they're catastrophic.
- It checks effective resolution, color space, embedded fonts, bleed, ink limit (TAC), overprint, hairlines, transparency, compression, and page boxes — against a saved profile, often verifying PDF/X compliance with a required output intent.
- Tools like Acrobat Pro and Enfocus PitStop report issues with clickable jumps and offer auto-fixups (RGB→CMYK, flatten, add bleed, embed fonts, reduce ink) — but low-res and spot-color decisions still need a human.
- Standard figures to remember: 300 dpi photos, 3 mm bleed, 0.125 in safe zone, 0.25 pt hairline floor, and TAC ceilings of 240–260% (uncoated) up to 320–340% (coated sheetfed). Never 400% registration black in art.
- For non-technical store owners, translate every failure into plain language with a visual proof, the consequence, and a recovery action — never a raw error code or a silent rejection.

14. Imposition & Binding: Arranging Pages on the Sheet

Open any printed booklet and the pages run neatly 1, 2, 3, 4… But the giant sheet that came off the press did *not* have its pages in that order. They were scattered, and some were printed upside-down. This section explains why — and how the printer plans that "scramble" so it folds back into perfect reading order. The craft of doing this is called **imposition**, and the way the finished pages are held together is called **binding**. Get these wrong and you waste paper, trim off page numbers, or hand the bindery a job it can't fold.

11.1 What imposition is, and why sheets look "wrong"

Imposition

> The prepress step of arranging many page-images on one large press sheet so that, after the sheet is printed, folded, trimmed, and bound, the pages read in the correct order. The layout on the sheet is deliberately **not** in reading order.

Flat

> The big unfolded printed sheet, before any folding.

Signature

> A flat after it is folded into a book section — one folded sheet that contains several pages.

Form

> The set of pages printed on one side of a plate (the front form and the back form).

Imposition is driven by the press. Big sheetfed and web presses print 8, 16, 32, or more pages on each side of one huge sheet. To make those pages end up readable after folding, the imposition software must place them in non-obvious positions — and rotate about half of them 180°. The two goals are always the same: **print and finish the job correctly**, and do it **with the fewest sheets and least waste**.

Analogy: Imposition is like a paper fortune-teller (cootie-catcher) you made as a kid. Unfolded, the numbers look randomly scattered and some are upside-down — yet once you fold it, every number lands in its right place. That's exactly why a flat press sheet looks scrambled.

Key takeaway: The page order on the press sheet is intentionally not 1-2-3. Imposition is the plan that turns a "scrambled" flat into a correctly ordered, folded, trimmed booklet.

11.2 Signatures and folding

Each fold doubles the number of page-faces, so a signature always holds a power-of-two number of leaves times two pages each:

- **1 fold → 4 pages**
- **2 folds → 8 pages**
- **3 folds → 16 pages**
- **4 folds → 32 pages**

A signature commonly holds 8, 16, or 32 pages — and **16 is the workhorse**. A whole book is several signatures collated together. The total page count must be a multiple of the signature size (or padded with blank/filler pages); leftover pages force a smaller, costlier partial signature. Fold accuracy is critical — if a fold drifts, the page sequence and color registration break.

Best practice: Plan your page count to fill whole signatures *early* — aim for multiples of 16 or 32 — and choose the binding before you design. The binding dictates gutter, spine allowance, and maximum page count.

11.3 N-up, step-and-repeat, and gang runs (three different things)

These three terms are constantly mixed up. They are not the same:

N-up

> Printing N page-images per sheet pass (2-up, 4-up, 8-up, 16-up). "4-up" just means four pages imposed at once. It's a pure efficiency term.

Step-and-repeat

> **One** job tiled repeatedly across the sheet at precise intervals — business cards, labels, stickers, tickets. Same artwork, many copies, filling the sheet.

Gang run (combination run)

> **Multiple different** jobs imposed on one shared sheet to split the press setup and plate cost and cut paper waste. This is the economic engine behind cheap online "gang" printers.

The key distinction: step-and-repeat = *one* job repeated; gang run = *many unrelated* jobs sharing a sheet.

Example: An online gang-run business-card vendor puts your 100 cards on a 28×40" sheet alongside a dozen strangers' jobs — one plate set, one press run, then a guillotine cuts them apart. That shared setup is why 500 full-color cards can cost just a few dollars. Meanwhile, a wedding-invite shop runs a single invite 8-up step-and-repeat on a 12×18" sheet, prints 200 sheets, and cuts — 1,600 invites from 200 passes.

11.4 Printing the back of the sheet (second-side methods)

Method	How the sheet flips	Plates needed	Trade-off
Sheetwise (work-and-back)	Front and back from two separate plates; sheet flipped and fed again	Two plate sets	Most flexible; highest plate cost
Work-and-turn	Both sides on one plate; sheet flipped left-to-right (around the vertical axis); same gripper edge	One plate	Half the plate cost; cut in half yields two finished copies
Work-and-tumble (work-and-flop)	One plate; sheet flipped top-to-bottom (around the horizontal axis); uses a *different* gripper edge on pass 2	One plate	Demands very square, accurately cut stock or registration drifts

11.5 Binding imposition: page order depends on the binding

How pages are ordered on the sheet depends entirely on how the finished book is held together.

Saddle-stitch

Folded sheets are **nested inside one another** and stapled through the spine fold. Each folded sheet = 4 pages, so the page count must be a **multiple of 4** (8, 12, 16, 20…). Practical range is about 8–64 pages (sometimes 80); beyond that the spine bulges and won't staple or lie well. The ordering pairs the outermost page with the innermost. For a 16-page booklet the printer-spread pairs are (16,1)(2,15)(14,3)(4,13)(12,5)(6,11)(10,7)(8,9) — and the two page numbers on any spread always **sum to total + 1** (here, 17).

Perfect-bound

Signatures are **stacked**, the spines are ground flat, and the block is glued into a wrap cover. Pages still work in multiples of 4 (ideally filling full signatures, e.g. multiples of 16). Each signature is its own self-contained booklet gathered in sequence — they are *not* nested — so there's no whole-book creep, and you get a flat spine you can print a title on.

Choose…	When you want…	Page count	Spine
Saddle-stitch	Thin, cheap, fast, lay-flat-ish booklets	Multiple of 4, ~8–64pp	Stapled fold (no printable spine)
Perfect-bound	Higher page counts, shelf spine, catalog/manual feel	Multiple of 4, ideally full signatures	Flat, glued, printable

Common mistake: Submitting a 14-page (or any non-multiple-of-4) file for a folded booklet. The bindery is forced to pad it with a blank page or reject the file. Always make folded-booklet page counts a multiple of 4.

11.6 Creep / shingling — the signature classic

Creep (push-out, binder's creep, shingling)

> In nested (saddle-stitch) books, each inner folded sheet sticks out slightly farther than the sheet wrapping it, because of accumulated paper thickness. After the face is trimmed flush, the inner pages end up **narrower** and their content sits closer to the trim edge — outer margins look uneven and content can get clipped.

Creep is negligible on thin 8–12pp jobs but serious on 60+ pages or heavy stock. **Compensation** is automatic: imposition software nudges each spread's content progressively *toward the spine*, more for the more-inner spreads, so margins look uniform after trim. As a rough guide, the number of nested spreads ≈ (total pages ÷ 4) − 1, and total creep ≈ that number × the paper's thickness (caliper) × 2 (it pushes out both sides of the fold). The outermost spread shifts the most.

Analogy: Creep is like wrapping a thick beach towel around a roll — every wrap has a longer circumference than the one beneath, so the outer edges fan out. Trim them all flush and the inner wraps end up shorter (narrower pages). That's why inner content must be shifted inward to compensate.

Example: A 64-page saddle-stitched magazine on 100lb gloss, with no creep compensation, loses several millimeters of outer margin on the center spread — a page number gets trimmed off. The RIP's auto-creep slides center content spine-ward so all 64 pages trim with matching margins.

Watch **crossovers** — an image spanning the gutter from one page to the facing page. Creep plus folding tolerance can misalign a crossover, so allow the image to overlap into the spine.

11.7 Reader's spreads vs printer's spreads

Reader's (designer's) spread

> Pages in natural reading order, facing pages side by side: 1, then 2-3, 4-5, 6-7… This is how you **design**, because it lets you build crossovers correctly across the gutter.

Printer's spread

> The imposed, non-consecutive arrangement that lands in order after print/fold/trim — e.g. a 16pp cover sheet carries 16+1 on one side and 2+15 on the other.

The workflow rule: **design in reader's spreads; convert to printer's spreads only at output** (InDesign's "Print Booklet", or dedicated imposition software / the RIP).

Best practice: Hand the printer **single pages** (or reader spreads) plus bleed and marks, and let *their* imposition software build the printer spreads. Doing your own printer spreads collides with their automated creep and imposition tools — a frequent, costly error.

11.8 Sheet-level geometry: gripper, gutters, bleed, and marks

Gripper margin

> The leading edge of the sheet the press jaws ("grippers") clamp to pull the stock through. **No printing is possible there.** Typically **3/8"–1/2" (≈9–13 mm)**, minimum about 1/4" (6 mm). All live image must stay off this edge.

Gutter

> Two meanings: (a) the inner/spine margin where pages meet at the binding, and (b) the blank lane *between* imposed pages on the sheet that leaves room for trim, bleed, and fold.

Bleed

> Image extended past the trim so trimming variance never reveals a white edge. Standard **3 mm (0.125") per edge**; some books want 5 mm. Add overlap into the spine for crossovers.

Trim/crop marks

> Where to cut.

Fold marks

> Where to fold.

Registration marks

> Bullseye/star targets printed by all four plates; the operator overlays the CMYK impressions to confirm color-to-color registration.

Color bars (control strip)

> Density/ink patches run along the gripper or tail edge, one zone per ink key, so the operator reads and adjusts ink density, dot gain, gray balance, and trapping.

Marks live in the waste/"slug" area outside the trim. A typical edge stack, inside to out, is: trim line → bleed (3 mm) → offset (3 mm) → mark zone (5 mm) → margin — budget about **11 mm of extra sheet space per side** for marks plus bleed.

11.9 A simple imposition / signature diagram

```
ONE PRESS SHEET — printed both sides, then 2 folds + trim → 8 pages

  FRONT of sheet                BACK of sheet
 +--------+--------+           +--------+--------+
 |   8    |   1    |           |   2    |   7    |
 |(upside |(right  |           |(right  |(upside |
 | down)  | side)  |           | side)  | down)  |
 +--------+--------+           +--------+--------+
 |   5    |   4    |           |   3    |   6    |
 |(right  |(upside |           |(upside |(right  |
 | side)  | down)  |           | down)  | side)  |
 +--------+--------+           +--------+--------+
  printer-spread pairs: (8,1) (2,7) (5,4) (3,6)
  each pair sums to 9  (total pages 8 + 1)
  half the pages are rotated 180 deg so they read
  upright after folding -- that's why a flat looks "wrong"

NESTING & CREEP (saddle-stitch, side view of folded sheets):

  spine                                face (trim edge)
   | ____________________________________ outer sheet
   ||  ___________________________________ next sheet (out less)
   |||  ____________________________________  inner sheet (out MOST)
   ||||
   ||||___________________________________ ) <- trim cuts all flush
                                          ^ inner pages end up narrower
  => imposition shifts inner content toward spine to compensate
```

11.10 Mistakes to avoid

Common mistake: Sending a PDF already laid out as printer's spreads (or as "perfect-bound 2-up"). It collides with the printer's own imposition and creep software and produces scrambled or double-imposed output. Send single pages or reader spreads.

Common mistake: No bleed, or parking text/page numbers in the binding gutter or on the gripper edge. Trimming and binding then clip live content — and thin saddle margins plus creep make it worse. Keep live matter clear of the gripper, add bleed, and give generous inner margins.

Section summary:

- **Imposition** arranges page-images on a big sheet (a "flat") so they read in order after folding, trimming, and binding — the scattered, partly upside-down layout is intentional. A folded flat is a **signature** (1 fold = 4pp, up to 32pp; 16 is the workhorse).
- **N-up** = N pages per pass; **step-and-repeat** = one job tiled; **gang run** = many different jobs sharing a sheet to split cost. Second-side methods: sheetwise (two plates), work-and-turn (flip left-right, one plate), work-and-tumble (flip top-bottom, needs square stock).
- Page order depends on binding: **saddle-stitch** nests sheets (multiples of 4, ~8–64pp, facing pages sum to total+1); **perfect-bound** stacks self-contained signatures (no whole-book creep, printable spine).
- **Creep/shingling** makes inner pages of nested booklets push out and trim narrower; imposition software auto-shifts inner content toward the spine. Worse with thick stock and high page counts.
- Design in **reader's spreads** with 3 mm bleed and generous inner margins, keep live image off the **gripper margin** (≈9–13 mm), and let the printer's RIP build printer spreads and add trim/fold/registration marks and color bars.

15. Finishing & Document Geometry: Bleed, Trim & Safe Area

When a designer sends a file to a print shop, the screen shows a clean rectangle exactly the size of the finished product. But that screen rectangle is a comforting lie. The real machines that cut, fold, and decorate paper are **mechanical and imperfect** — they miss by a fraction of a millimeter every single run. This section teaches you the geometry that absorbs that imperfection so your printed piece looks crisp instead of broken, and the finishing operations that turn a flat printed sheet into a folded brochure, an embossed business card, or a foil-stamped invitation.

Let me define the words first, because everything else builds on them.

Finishing (post-press)

> Everything done to the paper *after* the ink is on it: cutting, folding, laminating, foiling, die-cutting, and so on. "Press" puts ink down; "post-press" shapes and decorates the result.

Trim

> To cut the printed sheet down to its final size with a blade.

Guillotine

> The heavy industrial paper cutter — a long blade that slices through a tall stack of sheets at once.

Register / off-register

> "In register" means everything lands exactly where intended. "Off-register" means it shifted slightly. Cutting is never perfectly in register.

12.1 The three nested rectangles

Every correctly built print file has **three concentric rectangles**, drawn from the outside in. They exist for one reason: **cutting is never perfect**.

Bleed line (outermost)

> The area where artwork **extends beyond the final cut edge**, so background colors and images run all the way off the page. Standard = **3 mm (0.125 in, or 1/8 in)** past the trim on every side. Some shops accept 2 mm; large-format and packaging often want 5–10 mm.

Trim line (middle)

> The **final finished size** — where the guillotine is actually set to cut. This is your document/page size (for example A4 = 210 × 297 mm). Crop marks point here.

Safe area (innermost)

> Also called the safe zone or safe margin: a buffer **inside** the trim where all critical content — text, logos, page numbers, key graphics — must live. Standard = **3 mm minimum, 3–5 mm recommended** inside the trim. Push toward 5 mm near a binding or fold.

Analogy: Bleed is like painting *past* the edge of a wall before you trim the wallpaper. You slap paint a few centimeters beyond where you'll cut, so even a crooked cut never reveals bare wall. You always cut "inside the paint," never "at the boundary of the paint."

12.2 Why all three exist: the cut is a drunk blade

The whole system exists because **guillotines have tolerances** — a built-in margin of physical error.

Element	Standard value	Notes
Bleed	**3 mm = 0.125 in = 1/8 in** per side	Most common worldwide; some shops 2 mm
Safe margin	**3 mm min, 3–5 mm recommended**	Push to 5 mm near binding/fold
Total bleed added	trim + 2× bleed	A4 file = **216 × 303 mm** (210+6 × 297+6)
US Letter with bleed	8.5 × 11 → **8.75 × 11.25 in**	+0.125 in each side
Modern guillotine tolerance	**±0.3 mm**	Best machines — physics, not quality
Older guillotines	**±0.5 mm or more**	
Typical real-world shift	**0.8–1.6 mm (1/32–1/16 in)**	Trim can land ~1.5 mm either way batch-to-batch
Spot-finish standoff	**1.5–3 mm** from edge/fold	Keeps foil/UV off the crack zone
Min clearance from fold/cut	**0.125 in (3.2 mm)**	For content
Artwork past dieline	**0.25 in (6.4 mm)**	Packaging overextension

Why bleed exists, stated precisely: if a full-color page is cut even 0.5–1.5 mm off-register and there is *no* bleed, the blade lands slightly outside the artwork and exposes unprinted white paper → a thin white "sliver" line along the edge. Bleed gives the blade roughly 3 mm of "wrong place to land and still look right."

Why the safe margin exists: that same ±1.5 mm shift can also cut *into* the page. Anything closer than ~3 mm to the trim risks being sliced — a logo with its edge shaved off, a page number cut in half.

Analogy: The safe margin is the airline rule "don't seat passengers in the aisle." The trim can swerve ±1.5 mm like a slightly drunk blade, so keep anything important well back from the door it might slam.

Key takeaway: Background art must extend **3 mm past** the trim (bleed); critical content must stay **3 mm inside** the trim (safe area). The trim itself can land up to ~1.5 mm off in either direction — both buffers absorb that error invisibly.

12.3 Finishing operations and how each changes your file

These operations happen after printing. Many of them need an extra **spot-color layer** or a **dieline** so the machine knows where to act. Crucially, these layers are **instructions, not printed ink** — the shop reads them to control a machine, then removes or converts them; they never appear as printed color on the final piece.

Cutting (guillotine / three-knife trimmer)

> A straight blade cuts stacks to trim size. This is what drives all the bleed/safe geometry above. A three-knife trimmer cuts the three open edges of a bound booklet at once.

Folding

> Half-fold, tri-fold (letter fold), Z-fold, gate fold. Setup: lay panels out at the correct widths and keep content off the fold lines. On a tri-fold, the **inside-folding panel must be ~2 mm narrower** so it tucks in cleanly.

Scoring / creasing

> A blunt rule presses a channel into the stock so it folds without cracking. **Mandatory above ~250–300 gsm** and on any laminated or digital (toner) stock, or the coating/toner cracks along the fold. Setup: mark crease lines on a separate non-printing layer.

Laminating

> A thin plastic film bonded to the surface. **Gloss** = shiny, makes colors pop. **Matte** = subdued, low-glare, slightly mutes color, premium feel. **Soft-touch** (velvet/silk) = velvety tactile finish that deepens color richness — the luxury default. Laminated stock cracks at folds, so **crease before folding**.

Foil stamping (hot foil)

> A heated die presses metallic or pigment foil (gold, silver, copper, holographic) onto the sheet. Setup: a separate layer (e.g. "FOIL"), a **100% solid spot color**, **vector only**, text converted to outlines. No gradients — foil is on/off. Keep it 1.5–3 mm from edges and folds.

Die-cutting

> A custom steel die cuts non-rectangular shapes, windows, or tabs (a folder tab, a box window). Requires a **dieline**: a vector path on its own layer in a named spot color (often "Dieline" or "CutContour"), **never flattened, never printed**. Extend artwork ~0.25 in past the dieline.

Embossing / debossing

> A die and counter-die under pressure **raise** (emboss) or **press in** (deboss) an area — no ink, the effect is purely 3D. Needs thick stock (250–300 gsm+); avoid fine detail and thin lines. Mark the region on a separate spot layer.

Perforation

> A line of tiny cuts/holes so a piece tears off cleanly (tickets, coupons, reply cards). Mark the perf line on its own layer and keep critical content clear of it.

Spot UV (spot gloss varnish)

> A glossy UV-cured varnish on **selected** areas for contrast (a gloss logo on a matte card). Setup: a separate **spot-color layer** (commonly "SPOT-UV"), **vector**, solid shapes, text outlined. Use on bold shapes, not fine type. Keep it 1.5–3 mm from edges and folds so it doesn't crack when creased or cut. Grayscale values can control thickness for raised UV.

Analogy: A dieline or spot layer is a stencil or cookie-cutter outline. It's the *instruction* telling the machine where to cut, foil, or varnish — it never becomes part of the printed picture itself.

Universal finishing-file rules

- Work in **CMYK** color mode.
- Dieline and spot layers stay **vector and unflattened**.
- Each effect on its **own clearly named layer / spot color** — ask the shop for its exact swatch and layer names.
- Spot layers are **instructions, not printed ink**.
- Export PDF with crop marks and document bleed turned on.

12.4 Real-world print-shop examples

Example — the white-edge disaster: A designer sets a full-blue business card to exactly 90 × 50 mm with no bleed. During cutting the stack shifts 1 mm, so every card gets a 1 mm white hairline on one edge. Reprint. Adding 3 mm bleed (96 × 56 mm artwork) would have absorbed the shift invisibly.

Example — guillotined page number: A booklet has page numbers 2 mm from the trim. The trim drifts +1.5 mm inward and the bottom of every page number is shaved off across the whole run. Keeping them 3–5 mm inside the trim would have saved them.

Example — cracked foil on a folded invite: Gold foil runs straight across the fold of a heavy invitation with no creasing applied. The foil flakes and cracks at the spine. Fix: crease first, and terminate the foil 2–3 mm short of the fold.

A note on creep (thick saddle-stitched booklets)

Saddle stitch = binding with staples through a folded spine. In a thick saddle-stitched booklet the inner pages "push out" along the spine; the three-knife trim then shaves more off those inner sheets, so inner-page outer margins end up narrower. Imposition software applies **creep compensation** — shifting content inward toward the spine on inner pages to counter this. Perfect binding (a glued flat spine) has **no creep**.

12.5 Common mistakes and best practices

Common mistake: No bleed — the background stops at the trim line (or a photo edge lands exactly on trim with nothing behind it). Result: white slivers after cutting.

Common mistake: Critical text or logos in the safe-margin danger zone (less than 3 mm from the trim). The ±1.5 mm cut shift shaves them off.

Common mistake: Flattening the dieline/spot layer into the artwork, or building spot UV / foil / dieline as printed CMYK instead of a named spot color. The finishing machine can't read it, and the cut or foil line prints as ordinary ink.

Best practice: Set the document to *trim* size, then turn ON the app's bleed setting (3 mm all sides), pull every background element out to the bleed edge, and export the PDF with crop marks and "use document bleed."

Best practice: Put each finishing effect on its own named spot-color layer — vector, text outlined — kept 1.5–3 mm clear of all edges and folds. Confirm the shop's exact swatch and layer names before sending.

Best practice: Crease before folding anything over ~250–300 gsm or anything laminated, and apply creep compensation in imposition for thick saddle-stitched work.

Section summary:

- **Three nested rectangles:** bleed (3 mm out), trim (final size), safe area (3–5 mm in) — they exist because guillotines have a real ±0.3–1.5 mm tolerance.
- **Bleed** = artwork running off the page so a slightly-off cut never reveals white paper; **safe area** = keep important content back so it never gets shaved.
- **Finishing operations** (cut, fold, score/crease, laminate, foil, die-cut, emboss/deboss, perforate, spot UV) happen after printing and often need an extra dieline or spot-color layer.
- **Spot/dieline layers are instructions, not ink** — keep them vector, unflattened, named, and 1.5–3 mm clear of edges and folds; never flatten or print them as CMYK.
- Crease before folding heavy or laminated stock to stop cracking, and use creep compensation for thick saddle-stitched booklets.

16. Print Methods & Substrates: How Ink Meets Paper

So far you have learned how color is described, measured, and managed. Now we reach the moment where all of that theory becomes something physical: **ink (or toner) actually landing on a material**. This section answers the question every print-shop owner faces dozens of times a week: *"Which machine should print this job, on what stock, with what ink?"*

Here is the single most important idea before we start. Choosing a print method is mostly an **economics-of-volume plus substrate decision**. Let me define those two plain-English ideas:

Substrate

> The material you print *onto* — paper, card, a T-shirt, vinyl, a cardboard box. "Substrate" is just the industry word for "the thing receiving the ink."

Run length

> How many copies you are printing in one job — 50 business cards is a "short run," 50,000 catalogs is a "long run."

Run-length sweet spot

> The quantity range where one method's **cost per piece** beats every rival. Each method has its own sweet spot.

Key takeaway: There is no single "best" print method. There is only the right tool for a specific **quantity**, **material**, and **quality bar**. Get those three right and the method almost picks itself.

13.1 The Major Print Methods

A. Offset Lithography ("litho" / "offset")

How it works: The image is burned onto a thin metal **plate**. The plate is inked, the ink is transferred ("offset") onto a rubber **blanket cylinder**, and the blanket presses the image onto the paper. The image *never* touches plate-to-paper directly — it always goes through the rubber blanket.

The physics is **oil-and-water repulsion**. Image areas accept oily ink; the non-image areas are kept wet with a water-based "fountain solution" (called dampening) and therefore reject the oily ink. Oil and water don't mix, so the image and background stay cleanly separated.

Cost shape: The headline is **high setup cost** (plates, films or computer-to-plate, "makeready" calibration, ink wash-up between jobs) but **very low cost per unit**. The more you print, the cheaper each piece gets.

- **Sweet spot:** long runs, roughly **2,000+ pieces** (it can win from ~500–1,000 once setup is spread out).

- **Turnaround:** slow to start — typically **2–5 business days** just to prep and plate.
- **Quality:** the gold standard for color fidelity, fine type, large flat solid areas, and exact Pantone spot matching. Sheet sizes are far larger than digital.
- **Variants:** *sheetfed* (cards, brochures, packaging) and *web offset* (a continuous roll for magazines, newspapers, catalogs; "heatset" dries with heat, "coldset/non-heatset" air-dries).

Analogy: Offset is like baking a wedding cake from scratch — slow and costly to set up, but dirt-cheap per slice once you're serving hundreds. Digital is a cupcake vending machine — instant, no setup, but pricier per unit. That fixed-vs-variable cost shape *is* the whole run-length story.

B. Digital — Toner (electrophotographic / xerographic)

How it works (electrophotography, also called xerography): a light-sensitive **drum** is given an electric charge; a laser or LED array "writes" the image by selectively discharging spots on the drum; charged **toner** (a fine powder) sticks only to the image areas; the toner transfers to the paper and is **fused** — melted on with heat and pressure.

- **Dry toner** uses powder with small particles, needs **higher fusing temperatures**, and can leave a slight surface sheen.
- **Liquid toner** (e.g. HP Indigo "ElectroInk") uses charged ink particles only **1–2 microns** across, suspended in a liquid that evaporates with heat. The result is higher resolution, a uniform gloss, and a very thin, sharp image layer that looks **offset-like**.

The defining advantage: no plates means no setup cost, and because the image is rebuilt from the file for every sheet, **every print can be different** (this is called variable data — think 1,000 postcards each with a different name).

- **Sweet spot:** short runs, **under ~2,000 pieces**; print-on-demand and even single copies.
- **Turnaround:** prints straight from the file — the fastest method.

C. Digital — Inkjet (production / wide-format)

How it works: tiny nozzles in a printhead fire microscopic **ink droplets** directly at the substrate, building the image as a dot pattern — no plate, no drum. Machines are either roll-fed (production) or flatbed.

Production inkjet is very high-speed roll-fed printing. It has caught up to coated-paper quality, but historically struggles with **drying when ink coverage is heavy**, so it needs large drying systems.

Best practice: Rule of thumb for the two digital families — **toner** is sharp on small formats with consistent gloss (great for short office and marketing runs); **production inkjet** is faster at scale (better for very long variable runs like direct mail, transactional statements, and books).

D. Screen Printing (silkscreen)

How it works: ink is pushed with a **squeegee** through a stencil sitting on a fine **mesh** screen, onto the substrate. You need **one screen per color**.

Mesh count means threads per inch of screen. Lower count lets more ink through (bold, opaque); higher count lets less through (fine detail):

- **110–160** mesh: bold, opaque prints — thick ink deposits, white ink on dark shirts.
- **200–305** mesh: fine lines, halftones, and detailed artwork.

Plastisol ink (PVC resin plus plasticizer) is the apparel standard. It won't dry at room temperature; it must be **heat-cured at about 320–350°F (160–177°C)**, broadly the 300–330°F range. Store it below 90°F or it can pre-cure right in the bucket.

- **Setup-heavy:** you burn a screen per color, so unit cost falls as quantity rises and rises as color count rises.
- **Sweet spot:** medium-to-large runs; **~24+ garments** is the classic break-even versus DTG.
- **Use cases:** T-shirts and apparel, posters, signage, promotional goods, and printing on odd materials like glass, plastic, metal, and fabric.

E. DTG — Direct-to-Garment

How it works: a specialized inkjet sprays water-based ink **directly into garment fibers**. **Pretreatment** is required (especially on dark fabric) so the ink adheres, and **dark garments also need a white underbase layer** printed first — both add cost and slow the job.

Cost shape: mostly flat per unit (it's mainly ink) — **no screens, so no volume-discount curve**. A small order doesn't get punished and a large one doesn't get rewarded.

- **Sweet spot:** small orders, **under ~24 pieces**; one-offs and print-on-demand.
- **Cost example:** about **$1** for a white shirt versus about **$3** for a dark shirt (the dark one needs pretreat plus white ink).
- **Quality:** photographic full color and fine gradients — but less opaque and punchy than screen-printed plastisol, with a different feel and durability.

F. Wide / Large / Grand-Format

How it works: large inkjet machines (roll-fed or flatbed) for oversized output. The big differentiator is the **ink** — UV-curable, eco-solvent, or latex (covered in §13.3).

Substrates: vinyl, canvas, fabric, wallcovering, backlit PET film, foam board, corrugated plastic, acrylic, wood, and aluminum composite.

Use cases: banners, posters, wall murals, window and floor graphics, vehicle wraps, trade-show displays, and retail point-of-purchase signage — indoor and outdoor.

G. Flexography ("flexo")

How it works: rotary printing using a **flexible photopolymer relief plate** — a raised, rubber-stamp-like image. The ink path is: ink → **anilox roller** → **doctor blade** → plate cylinder → substrate (backed by an impression cylinder).

Anilox roller

> A ceramic or chrome cylinder covered in precisely engraved microscopic cells that **meter an exact volume of ink** onto the plate. Its cell geometry (size, shape, count) governs ink volume and color consistency — it is the heart of flexo quality.

Doctor blade

> A thin blade that wipes excess ink off the anilox roller so only the metered amount in the cells remains.

- **Very high speed: up to ~2,000 ft/min.** Plate and setup costs are high, so the economics favor **very long runs**.
- **Big edge:** it prints **non-porous and flexible materials** (plastic film, foil, brown kraft, corrugated board, pouches) with water-based inks — things offset can't handle well.
- **Use cases:** packaging — corrugated boxes, labels, flexible food pouches, poly mailers, tape, wallpaper.

13.2 Substrates — Paper & Stock

Paper Weight: GSM vs. lb (Basis Weight)

This is where beginners get burned the most, so go slowly.

GSM (grams per square meter)

> The metric measure of paper weight. It is **universal and consistent across every paper type**, which makes it the reliable spec — especially for international orders.

lb / basis weight (US system)

> The weight of **500 sheets (one ream) at that paper's "basic" uncut size**. The catch: the basic size **differs by paper category**, so a "lb" number is meaningless without naming its category.

The basic sizes per category:

- **Bond / Writing:** basic size 17" × 22"
- **Text / Book:** basic size 25" × 38"
- **Cover:** basic size 20" × 26"

Rough lb-to-gsm conversions:

- **Bond:** gsm ≈ lb × 3.76 → **20 lb bond ≈ 75 gsm**
- **Text/Book:** gsm ≈ lb × 1.48 → **80 lb text ≈ ~104–120 gsm**
- **Cover:** gsm ≈ lb × 2.71 → **80 lb cover ≈ ~216–218 gsm**

Common mistake: Ordering "80#" paper with no category. **80 lb text (~104 gsm)** is light brochure paper, while **80 lb cover (~218 gsm)** is heavy card stock — the *same number* for roughly **twice the weight**. Always specify gsm *and* category.

One more axis: **points (pt / caliper)** measure **thickness** in thousandths of an inch (e.g. 14 pt card). Thickness is separate from weight — gsm tells you how heavy, points tell you how thick.

Coatings & Finishes — and Their Effect on Color and Dot Gain

A **coating** is a thin clay or mineral layer applied to the paper that seals the surface so ink **sits on top** instead of soaking in. First, a quick reminder of one term:

Dot gain

> The tendency of printed halftone dots to come out **larger** than specified (because ink spreads as it touches paper), which makes the print look darker than intended.

Surface	What ink does	Color result	Dot gain
Coated	Sits on top, little absorption	Crisp, sharp dots; deeper, more vivid, glossier color	**LOW**
Uncoated	Soaks into porous fibers and spreads	Muted, softer, slightly darker; writable surface	**HIGH**

The finish scale from most to least reflective: **Gloss** → **Silk/Satin/Semi-gloss** → **Dull** → **Matte** → **Uncoated**. Coating thickness drives it: heavy = gloss, medium = silk/satin, light = matte.

Specialty stocks include recycled, textured (linen, laid, felt), synthetic/waterproof (Yupo, Teslin), kraft, metallic/pearlescent, carbonless (NCR), backlit film, canvas, magnetic, and security stocks.

Analogy: Coated paper is a glossy ceramic tile — ink beads on top, staying sharp and vivid. Uncoated paper is a paper towel — ink wicks in, spreads, and goes soft and muted. That single difference is exactly why dot gain and color saturation behave so differently on the two.

13.3 Inks & Toners

Process color (CMYK / "4-color")

> Cyan, Magenta, Yellow, and Black printed as overlapping halftone dots that blend into the full spectrum. Cheap (only 4 plates) and ideal for **photographs and continuous-tone artwork**.

Spot color (Pantone / PMS)

> A single **pre-mixed** ink matched to an exact standard (each PMS color has a number/name). Used for **brand logos, exact-match solids, and colors outside the CMYK gamut** — pastels,

> fluorescents, and metallics. It costs more (an extra plate plus a press wash-up per spot color). A common combo is **CMYK + one spot** for a brand color CMYK can't reach.

UV-curable ink

> Cures **instantly** under UV light (often cool-running **UV-LED**, so it can print heat-sensitive vinyl and PET). Forms a tough, durable film and sticks to nearly any surface — a wide-format and rigid-media workhorse.

Eco-solvent ink

> Etches into vinyl and plastics for **outdoor durability**, with lower VOC and emissions than true solvent ink. A signage staple.

Latex ink

> Water-based (about **70% water**), heat-cured, odorless and VOC-free — safe for **indoor** printing. Flexible and fade-resistant.

Plastisol

> The standard screen-print apparel ink: heat-cured, opaque, and durable.

Toner

> Either dry powder (higher fuse temperature) or liquid (1–2 micron, offset-like) for electrophotographic digital presses.

Canonical Prepress Numbers — Total Ink in the Shadows

Total Area Coverage (TAC / total ink limit)

> The maximum combined CMYK dot percentage allowed in the darkest shadow. Go over it and the ink won't dry or "trap" properly — you get smearing and "set-off" (wet ink transferring to the back of the next sheet).

- **Sheetfed offset, coated:** ~300–340%
- **Web offset / SWOP, coated:** 300%
- **GRACoL uncoated:** 280%
- **Uncoated newsprint / non-heatset web:** ~240–260%

And remember **dot gain**: it is **lower on coated** and **higher on uncoated/textured** stock, and you compensate for it by using the correct ICC output profile for the exact stock and process.

Common mistake: Building a deep black at 400% ink (CMYK all at 100) and sending it to coated offset capped at 300% TAC. The shadows come out wet, smearing, and setting off. Instead use a **rich black within the TAC limit** — for example 60/40/40/100.

13.4 Method Comparison Table

Method	How image forms	Setup cost	Run-length sweet spot	Best substrates	Typical use
Offset litho	Plate → blanket → paper (oil/water)	High (plates)	**~2,000+** (long)	Coated/uncoated paper, card	Books, catalogs, brochures, packaging
Digital toner	Charged drum + fused toner	**None**	**<~2,000** (short)	Paper, light card	Print-on-demand, short marketing, variable data
Production inkjet	Droplets from nozzles	None/low	Long variable runs	Paper rolls	Direct mail, transactional, books
Screen print	Ink through mesh stencil	High (1 screen/color)	**~24+ garments** (med–large)	Apparel, posters, odd materials	T-shirts, signage, promo goods
DTG	Inkjet into garment fibers	None	**<~24** (small)	Cotton garments	POD apparel, one-offs, photo prints
Wide-format	Large inkjet (UV/eco-sol/latex)	Low	Any (1+)	Vinyl, fabric, rigid boards	Banners, wraps, signage, murals
Flexography	Flexible relief plate + anilox	High (plates)	**Very long**	Film, foil, kraft, corrugated	Packaging, labels, pouches

13.5 Putting It Together: Real Quoting Decisions

Example — the 1,500 vs. 5,000 flyer quote: Digital quotes **$0.48/piece with no setup**; offset quotes **$0.28/piece plus a $400 setup**. Break-even lands at about **2,000 pieces**. Below it, digital wins; above it, offset's lower per-piece cost swamps the setup fee. This is the most common quoting decision a print shop makes.

Example — the 24-shirt line in an apparel shop: 12 custom shirts go to **DTG** (no screens to burn). 200 shirts of one design go to **screen printing** (per-shirt cost collapses once the screens are paid off). The shop quotes differently on either side of the ~24-piece line.

Example — the Coca-Cola red / Starbucks green job: the brand color can't be hit reliably in CMYK, so the press runs **CMYK + a Pantone spot** — one extra plate and wash-up, but the brand color is dead-on every single run.

```
ROUTE A JOB BY VOLUME (paper)        ROUTE APPAREL BY VOLUME
---------------------------          ----------------------
  qty < ~2,000  --> DIGITAL            qty < ~24  --> DTG
  qty > ~2,000  --> OFFSET             qty > ~24  --> SCREEN
  very long pkg --> FLEXO              dark shirt --> +white
                                                      underbase
```

Best practice: Quote by quantity first — find the break-even and route the job to the cheaper method. Then spec the stock in **gsm + finish** and supply the matching **ICC output profile** so both color and dot gain are predicted *before* plates are cut. Use spot colors for brand and exact-match solids, process for photos, and combine CMYK + spot when a brand color falls outside gamut.

Common mistake: Proofing on a coated profile and then printing on uncoated stock. The uncoated dot gain plugs up the midtones and the job arrives dark and muddy. Always proof and profile against the stock you will actually print on.

Section summary:

- **Method = economics + substrate.** Offset and flexo have high setup but cheap per-unit costs (long runs); digital toner/inkjet and DTG have no setup but flat per-unit costs (short runs); each has a run-length sweet spot.
- **Know the break-evens:** ~2,000 pieces for offset vs. digital on paper, and ~24 garments for screen vs. DTG on apparel — these two numbers drive most quoting decisions.
- **Paper weight: always specify gsm + category.** "80#" alone is a trap — 80 lb text (~104 gsm) is light brochure stock while 80 lb cover (~218 gsm) is heavy card.
- **Coating controls color and dot gain:** coated = vivid, sharp, low dot gain; uncoated = muted, soft, high dot gain. Use the correct ICC profile for the real stock.
- **Ink choice matters:** process (CMYK) for photos, spot (Pantone) for brand/exact solids, and respect TAC limits (≈300% coated offset) by building rich blacks like 60/40/40/100.

17. The RIP, Press Operation & Color Measurement

You have designed a file, chosen colors, and exported a PDF. But a printing press cannot read a PDF, and it cannot lay down "50% gray ink." Something has to translate your beautiful page into the only language a press understands: **ink-here / no-ink-here dots**. That translator is the **RIP**. This section follows your file from PDF all the way to a printed sheet that matches an agreed color, and shows you the tools that prove it matched.

14.1 The RIP (Raster Image Processor) — the engine that makes press-ready dots

A **RIP (Raster Image Processor)** is software (sometimes dedicated hardware plus software) that reads a page-description file — PDF, PostScript, or the print-safe flavor PDF/X — and converts *everything* on the page (text, vector shapes, photos) into one high-resolution **bitmap of dots** that the output device (platesetter, imagesetter, digital press, inkjet) can physically print. The verb "RIPping" means this whole pipeline: interpret → render → color-manage → screen → output.

Analogy: The RIP is a translator *and* typesetter for the press. Your PDF is a manuscript written in a language the press cannot read. The RIP translates it into the only thing the press understands — dots of ink in fixed positions — and arranges those dots into a clean pattern.

The four jobs a RIP does, in order

1. **Interpretation** — reads the page; resolves fonts, vector paths, embedded photos, transparency, overprint, and trapping instructions.
2. **Rendering / rasterization** — flattens the page into a continuous-tone bitmap at the device's **addressable resolution** (e.g. 2400 dpi on a platesetter).
3. **Color management** — **this is where color management actually executes.** The RIP applies ICC profiles to convert your colors into what the press can produce.
4. **Screening (halftoning)** — converts the continuous-tone bitmap into the **1-bit on/off dot pattern** the press needs, because a press can only put ink down or not.

Key takeaway: Color decisions are *made* in design (when you pick profiles), but they are *executed* in the RIP. The RIP is the single place where source colors actually become press colors and where flat tones become printable dots.

Common mistake: Confusing the device's **addressable resolution (dpi, e.g. 2400)** with the **screen ruling (lpi, e.g. 150)**. They are different. Many tiny device dots (2400 dpi) build up each individual halftone dot (150 lpi). One is how finely the laser can place marks; the other is how coarse the visible dot grid is.

Raster vs vector inside the RIP

Vector (text, logos, line art)

> Stays mathematically sharp until the very last moment, then is rasterized at full device resolution — so edges are crisp at any size. This is why text should stay vector, not be flattened to a low-resolution image.

Raster (photos)

> Resolution-fixed; the RIP resamples and screens it. Rule of thumb: image **ppi ≈ 2 × screen lpi** (a "quality factor" of 1.5–2.0). For a 150-lpi job, supply ~300 ppi; for a 200-lpi job, ~400 ppi.

Common mistake: Flattening vector text into a 150-ppi image so the edges go soft, or expecting 300-ppi sharpness at a 200-lpi ruling. Keep text and line art as vector, and match image ppi to roughly twice the screen ruling.

Screening (halftoning) — how the RIP fakes shades of gray

A press cannot print "40% ink." So tone is *simulated* using a pattern of dots — bigger/closer dots look darker, smaller/sparser dots look lighter. The RIP decides the screening type:

Type	How it works	Strengths	Weaknesses
AM (Amplitude Modulated) / conventional	Dots sit on a fixed grid at a fixed frequency (lpi); **dot SIZE varies** with tone	Standard, predictable, easy to control	Needs different screen angles per color to avoid moiré
FM (Frequency Modulated) / stochastic	Tiny, mostly fixed-size dots; **placement/frequency varies**	Largely moiré-free (no fixed angle), sharper detail, good for >4 colors	Harder to control dot gain; more sensitive to plate/press noise
Hybrid / XM	AM in midtones, FM in highlights and shadows	Combines control with smooth extremes	More complex to set up

Screen frequency (LPI) — the canonical values

- Newspaper (uncoated/newsprint): **85 lpi** (sometimes 65–100)
- Commercial coated / magazines: **150 lpi** (the de-facto general standard)
- High-end art/photo books: **175–200+ lpi**
- Screen printing / flexo: much coarser, often 45–65 lpi

Screen angles (AM) — and the rosette

If all four colors printed their dots at the same angle, they would clash into an ugly wavy pattern called **moiré**. So the RIP rotates each color's screen. The canonical CMYK set is:

Cyan = 15°, Magenta = 75°, Yellow = 0° (90°), Black = 45°. The three most visible colors (C, M, K) are kept **30° apart** (45 / 75 / 105). Yellow — the least visible — is squeezed to just **15°** from cyan. Correct angles produce the desirable flower-like **rosette** pattern; wrong angles produce moiré. (Yellow is sometimes run at ~108% of the others' frequency to further suppress moiré.)

```
DESIRABLE: rosette          UNDESIRABLE: moiré
 . o . o . o .               ~~~~~~~~~~~~~~
 o . O . o . O   <- petals    ~  wavy plaid  ~
 . o . o . o .               ~  interference ~
 (clean flower pattern)       ~~~~~~~~~~~~~~
```

Example: Put a printer's loupe on any glossy magazine photo and you will see the tiny C/M/Y/K dots forming a rosette. If a screen angle is wrong — or a vendor accidentally re-screened an already-screened image — that rosette breaks into a wavy moiré plaid. It is a dead giveaway of a RIP/screening error.

14.2 Press operation (offset lithography)

Plates and CTP

Modern prepress is **CTP (Computer-to-Plate)**: the RIP's 1-bit bitmap is imaged by a laser **directly onto an aluminum plate**, skipping film (older workflows used CTF, computer-to-film). CTP is more precise, faster, and loses less dot than film. **Thermal CTP** is most common (a laser changes the image areas); UV/violet CTP also exists. After developing, image areas become **ink-receptive (oleophilic)** and non-image areas become **water-receptive (hydrophilic)**. There is one plate per color (CMYK plus any spot colors).

The core **lithography principle**: image and non-image areas sit on the *same plane*. Printing works because **oil (ink) and water repel each other** — not because of raised or recessed areas.

Makeready

Makeready is all the setup before good sheets run: mounting plates, loading the right ink in each fountain, setting ink keys, achieving ink/water balance, getting registration, and bringing density to target. Makeready is the costly, time-consuming part, and spoilage sheets are normal.

Example: A 4-color sheetfed job typically burns ~150–300 spoilage sheets getting ink/water balance, registration, and density to target before the first sellable sheet. That fixed setup cost is exactly why a 250-piece run costs nearly as much as 1,000, and why "gang runs" save money.

Ink/water balance — the heart of offset

The dampening system feeds **fountain solution** (water plus additives: gum arabic, isopropyl alcohol or a substitute, an acid/buffer to control pH, and fungicide) to keep non-image areas ink-free. Operators

ride a narrow window:

- **Too much water:** solid ink density drops, color goes weak, dots soften/fuzz, ink emulsifies, drying slows.
- **Too little water:** non-image areas start taking ink → **scumming / toning / tinting** (a dirty background).

Registration and the color bar

Registration means getting all the color plates to align exactly, so the rosette forms and edges stay crisp; misregister shows up as colored fringing or blur. It is checked with crosshair **registration marks**. The **color bar (control strip)** is a strip of patches printed in the trim/gripper edge: **solid patches** (density per ink), **tints** (dot gain/TVI), **gray-balance patches**, slur/doubling/trap targets, and registration marks. The operator — or an automated scanner — reads these to steer the press.

Closed-loop color control: inline or offline spectro scanners (on KBA/Heidelberg/Komori presses) read the color bar automatically every few sheets and **auto-adjust the ink keys** to hold target density/color — cutting operator guesswork and waste.

14.3 Proofing workflow — contract proof vs soft proof

	Contract proof (hard proof)	Soft proof (monitor proof)
What it is	A physical, color-accurate print, usually from a calibrated inkjet proofer driven through the press ICC profile	An on-screen simulation via "Proof Colors" using the press profile
Standard	Certified to **ISO 12647-7**	Monitor per **ISO 12646**; viewing per **ISO 3664**
Role	**Legally binding** "this is the color we agreed," signed by the client; the press is matched to it	Cheap, instant, collaborative review
Requirements	Verified against a control strip (e.g. **Fogra Media Wedge**) with pass/fail ΔE limits	Valid only on a hardware-calibrated wide-gamut monitor at **D50 (5000K)**, **≥160 cd/m²**, in a controlled D50 environment

Key takeaway: The contract proof is the agreement. Once the client signs it, the press operator's job is to match that proof — and color measurement is how everyone proves the match really happened. Certified soft-proof setups can even serve as a contract proof.

14.4 Color measurement — densitometer vs spectrophotometer

Densitometer

> Measures **optical density** — how much of a filtered band of light the ink film absorbs (`D = log10(1/reflectance)`). It tells you **ink film thickness / how much ink** is down per channel. Fast, simple, cheap; ideal for an operator holding ink levels steady on a long run. **It does NOT measure color** — it is a process measure, not a perceptual one. Reports density, TVI/dot gain, print contrast, ink trap, slur/doubling.

Spectrophotometer

> Measures **spectral reflectance across the whole visible spectrum (~380–730 nm)**, then computes **colorimetric values: CIE L*a*b*** and **ΔE** color differences. It tells you the **actual color a human sees**, independent of which ink produced it — essential for color matching, spot-color verification, proof/press agreement, and brand QC. A **spectrodensitometer** does both.

Key takeaway: Density = *how much ink* (a control knob for the operator). Colorimetry / L*a*b* = *what color it actually is* (the customer's truth). Right density does NOT guarantee right color.

Analogy: Density is a thermometer — one number in range tells you how much ink is down. The spectrophotometer with ΔE is the doctor's full diagnosis — it tells you whether the patient (the actual color a human sees) is genuinely healthy. *"A densitometer can fool you into thinking you're printing the right color; a spectrophotometer won't."*

Measurement conditions (ISO 13655 — the M-series)

Modern papers contain **OBAs (optical brighteners)** that fluoresce under UV light, which skews readings. So the standard defines lighting conditions:

- **M0** — undefined UV (legacy); not recommended when paper fluoresces or when sharing data between sites.
- **M1** — D50 daylight with *defined* UV; the **modern preferred standard**, correctly handling OBA-brightened papers. Current specs (GRACoL/FOGRA51) reference M1.
- **M2** — UV-cut (excludes UV/fluorescence). (M3 = polarized.)

14.5 ΔE (Delta E) — "how close are two colors?"

ΔE is the numerical distance between two colors — bigger means more different.

Metric	What it is	When to use
ΔE76 (CIE76)	Simple Euclidean distance in L*a*b*	Easy, but perceptually non-uniform — overstates differences in blues/saturated colors
ΔE94	Adds chroma/hue weighting	Better for saturated colors
ΔE2000 (CIEDE2000)	Corrects lightness, chroma, hue, and the blue region	**Current gold standard** — use for specs and tolerances

Canonical tolerance benchmarks

- **ΔE ≈ 1.0 = JND** (Just Noticeable Difference) — below 1, essentially no observer can tell them apart.
- **ΔE ≤ 1:** high-precision / luxury / branding / proofing.
- **ΔE ≤ 2–3:** standard acceptable range for professional commercial print. Brand spot colors on prime labels: **ΔE2000 < 2.0** is common.
- **ΔE 3–5:** noticeable difference.
- **ΔE > 5:** clear mismatch / reject.

Example: A brand red prints at "correct density," yet a spectrophotometer reads **ΔE2000 = 4** against the brand standard, because the paper's OBAs (measured under M0 instead of M1) shifted the white point. The densitometer said "fine"; the spectrophotometer caught the reject. Right density, wrong color.

Common mistake: Trusting density alone for color. Holding solid density on-target while the actual L*a*b* color drifts (wrong paper, ink batch, or measurement condition). Density controls amount; only colorimetry controls color.

14.6 Process-control standards — ISO 12647-2 and G7

ISO 12647-2 (offset) sets the targets: solid-ink L*a*b* aims, paper aims, and **TVI (Tone Value Increase / dot gain)** curves. Canonical TVI on #1 coated at 150 lpi is **≈16% at the 50% midtone** (CMY; black slightly higher), ~19–22% at the 40% patch. Typical offset dot gain runs **15–30%** depending on substrate.

TAC / Total Area Coverage (Total Ink Coverage) is the summed % of C+M+Y+K in the darkest shadow. Limits prevent set-off, slow drying, and trapping problems: **~300% on coated**, **~240–260% on uncoated/newsprint**. The RIP/separation enforces it via ink limiting and GCR/UCR.

G7 (Idealliance) is a device-independent **gray-balance + tonality calibration method** that works on offset, flexo, gravure, and digital. Instead of chasing per-ink dot-gain curves, G7 calibrates to two things:

- **NPDC (Neutral Print Density Curve)** — a defined relationship of neutral density across the tonal scale.
- **Gray balance** — the specific C/M/Y combinations that print as visually neutral gray.

The output is a set of **1-D correction curves** applied in the RIP so any device "shares the same gray appearance." Certification is **G7 Master / Press Control System**. Closed-loop scanning of the color bar ties G7/ISO targets to automatic ink-key correction for consistent, repeatable runs.

Common mistake: No (or garbage) color management at the RIP — sending untagged RGB, a wrong or missing ICC profile, ignoring overprint/spot handling, or setting no TAC limit. The result: shadows fill in, spot colors convert wrong, and the proof and press disagree.

14.7 The whole pipeline, tied together

```
Design (RGB/CMYK + spots)
  -> PDF/X export (right profile, fonts embedded, TAC set)
  -> RIP:
       interpret -> render to device bitmap
       -> ICC color mgmt (source -> PCS -> output,
          rendering intent, spot + ink-limit)
       -> screen to 1-bit dots (AM/FM, lpi, angles)
  -> CTP images aluminum plates
  -> press makeready (ink/water, registration, density)
  -> run; color bar read by densitometer/spectro
     (closed-loop auto-correction)
  -> match to ISO 12647-7 contract proof
  -> verify with delta-E 2000 to spec
```

Best practice: Calibrate, then characterize, then control. Linearize the device → build/apply correct ICC profiles in the RIP → run G7/ISO 12647 targets → verify with ΔE2000 against an ISO 12647-7 contract proof, measuring with M1. Let the RIP own both color management and screening (one consistent place), supply clean PDF/X with embedded profile and ink limit, and keep text/line art as vector. Use the right tool per job: densitometer/closed-loop for on-press consistency; spectrophotometer (L*a*b*/ΔE) for color accuracy and brand sign-off — always under controlled D50/ISO 3664 viewing.

Section summary:

- The **RIP** interprets the PDF, renders it to a device bitmap, applies **ICC color management** (this is where color management actually executes), and **screens** it to 1-bit press dots — vector stays sharp to the last moment, raster needs ppi ≈ 2× lpi.
- **Offset press operation** runs on CTP plates and the oil/water repulsion principle; makeready (ink/water balance, registration, density) is the costly setup, steered by the **color bar** and often automated via closed-loop control.
- The **contract proof** (ISO 12647-7) is the binding color agreement; the **soft proof** is cheap and instant but valid only on a calibrated D50 monitor.
- A **densitometer** measures how much ink (density); a **spectrophotometer** measures the actual color (L*a*b*/ΔE) — right density never guarantees right color, and M1 measurement handles modern brightened papers.
- **ΔE2000** is the gold-standard color-difference metric (≈1 = just noticeable, ≤2–3 acceptable, >5 reject); **G7** and ISO 12647 plus closed-loop scanning keep gray balance and tone on target so press, proof, and brand all agree.

Glossary of Terms

This glossary collects every important term from the guide in one place, in plain English. Terms are listed alphabetically. Where a term is best understood next to a partner term (for example, **raster** and **vector**), the definition points you to the related entry.

Additive color

> Making colors by **adding light** together. Start from black (no light) and add red, green, and blue light; all three at full strength make white. This is how screens, projectors, and your eye work. (Contrast with **subtractive color**.)

Alpha (alpha channel)

> An extra channel stored with an image that records how **transparent or opaque** each pixel is. An alpha of 0 means fully see-through, full alpha means fully solid. It lets a cut-out logo sit over a background with no white box around it.

Alpha channel

> An extra channel stored alongside the color channels that records how opaque or transparent each pixel is, from fully see-through to fully solid. A logo on a transparent background owes that transparency to its alpha channel.

AM screening (Amplitude Modulated)

> A halftoning method where halftone dots sit on a fixed grid at regular spacing, and the **dot size grows or shrinks** to show lighter or darker tone. The traditional, predictable kind of screening. (Contrast with **FM screening**.)

Anti-aliasing

> Smoothing the jagged "staircase" edges of text and shapes by adding partly-shaded pixels along the boundary, so a curve looks smooth instead of blocky.

ArtBox

> An optional PDF page box that marks the **meaningful artwork area** of a page, ignoring surrounding white space. Rarely used in everyday print; placement tools sometimes read it.

Baseline

> The invisible line that letters sit on. Most characters rest on it; parts like the tail of a "y" hang below it (the descender).

Bézier curve

> The math behind smooth vector curves. A curve defined by anchor points plus **control handles** that pull the line into shape. Fonts and vector drawings (the "S" in a logo) are built from Bézier curves, so they stay crisp at any size.

Bicubic resampling

> A high-quality way to resize an image: each new pixel is calculated by blending a neighborhood of surrounding pixels with a smooth curve. It looks better than simpler methods, which is why `sharp` in the PDF service uses high-quality resampling by default.

Bit depth

> How many **bits of information describe each color channel** of a pixel. 8-bit gives 256 levels per channel (about 16.7 million RGB colors); higher depth (16-bit) gives smoother gradients with less banding. More bits = finer tonal steps.

Black point compensation (BPC)

> An adjustment during color conversion that **maps the darkest black of the source to the darkest black the destination can print**, so shadow detail isn't crushed into a flat dark blob. Usually left on for photographic work.

Bleed

> Extra artwork that **extends past the trim edge** (commonly 3 mm / 0.125 in) so that when the paper is cut, color runs right to the edge with no white slivers, even if the cut drifts slightly.

BleedBox

> The PDF page box that defines the **area including bleed** — the region the press should render out to before trimming. Sits just outside the TrimBox.

Blend mode

> A rule for how the colors of an upper layer **combine with the layer below** (Multiply, Screen, Overlay, etc.). "Multiply," for example, darkens by multiplying the two colors together — handy for shadows.

Choke

> A type of trap where the lighter background is pulled slightly inward over a darker foreground object, hiding gaps caused by misaligned print plates. The opposite of a spread.

CIELAB (Lab)

> A **device-independent color space** built to match human vision, using three numbers: L* (lightness), a* (green↔red), b* (blue↔yellow). Because it describes color absolutely, it's used as the common reference language for color management and for measuring color differences (see **Delta-E**).

CMM (Color Management Module)

> The **software engine** that does the actual color math — reading ICC profiles and converting colors from one device's space to another through the connection space. The "translator" in the color pipeline.

CMYK

> The four printing inks: **Cyan, Magenta, Yellow, and Key (black)**. A **subtractive** system — inks absorb (subtract) light from white paper. The standard color model for full-color ("process") printing.

Coated paper

> Paper with a smooth clay-like surface coating (gloss, silk, or matte). The coating **holds ink on top of the sheet** rather than letting it soak in, giving sharper detail and more vivid color. (Contrast with **uncoated**.)

Color depth (bit depth)

> How many bits are used to record each pixel's color, which sets how many distinct shades are possible. 8 bits per channel gives 256 levels each (about 16.7 million colors); 16-bit gives far more, useful for smooth gradients without banding.

Color management

> The whole system of profiles and conversions that keeps a color looking consistent as it moves from camera to screen to printer, instead of shifting at every step.

Color space

> The specific range and arrangement of colors a device or standard can represent, such as sRGB or US Web Coated (SWOP) for offset printing. Two devices can both be "RGB" yet have different color spaces.

Colorimetric (rendering intent)

> A family of rendering intents that aim for **color accuracy**. **Relative colorimetric** shifts white to the paper white and keeps in-gamut colors exact, clipping out-of-gamut ones. **Absolute colorimetric** reproduces colors exactly including the original paper white — used for proofing/contract matching.

Colorimetric intent

> A rendering intent that keeps in-gamut colors exactly accurate. *Relative* colorimetric remaps the white point and clips out-of-gamut colors to the nearest reproducible ones (best for most print work); *absolute* reproduces the paper white too (used for proofing one paper on another).

Content stream

> The part of a PDF that holds the **actual drawing instructions** for a page — "put this text here, draw this line, place this image." A sequence of operators the viewer executes to paint the page.

Creep (push-out)

> In folded booklets, the inner pages stick out slightly further than the outer pages because of the **thickness of the folded paper stacking up**. Imposition compensates by shifting content inward on inner pages. (See **shingling**.)

Crop marks (trim marks)

> Short thin lines printed just outside the page corners that show the cutter exactly where to slice the paper to the final size.

CropBox

> The PDF page box that defines **what a viewer displays and clips to**. It doesn't affect the final print target the way TrimBox does, but controls the visible region on screen.

Delta-E (ΔE)

> A single number measuring the **difference between two colors** in Lab space. Roughly: ΔE of 1 is a just-noticeable difference; under ~2 is generally an acceptable match for commercial print. The standard yardstick for "how close is the color?"

Densitometer

> An instrument that measures **how much light an ink layer absorbs** (its density) for each process color. Used on press to check that ink is being laid down at the right strength. (Compare **spectrophotometer**, which measures actual color.)

Die-cut

> Cutting paper into a **custom (non-rectangular) shape** using a steel die, like a stamp. Used for rounded corners, shaped tags, pocket folders, and window cut-outs.

Digital print

> Printing directly from a digital file with **no fixed printing plates** (toner or inkjet). Cheap for short runs, allows variable data (every copy different), but per-page cost stays flat as quantity rises. (Contrast with **offset**.)

Dot gain

> The tendency of halftone dots to **print larger than intended** as ink spreads into the paper, making images look darker than the file. Profiles and the RIP compensate for it; uncoated stock has more dot gain than coated.

DPI (dots per inch)

> How many **ink/toner dots a printer or RIP places per inch** of paper. A device/output measure. Often confused with PPI; see **PPI** and **LPI**.

Embossing

> A finishing process that presses a **raised (or, for deboss, recessed) design into the paper** using a die, with no ink — felt with the fingers. Adds a tactile, premium feel.

Flattening (transparency flattening)

> Converting overlapping transparent layers, blend modes, and soft shadows into plain opaque shapes (or a single rasterized image) that older print workflows can handle, since classic CMYK printing has no concept of transparency.

FM screening (Frequency Modulated / stochastic)

> A halftoning method using **tiny dots of fixed size scattered more or less densely** to show tone (more dots = darker). Avoids screen-angle moiré and can hold fine detail. (Contrast with **AM screening**.)

Foil (foil stamping)

> Pressing a **thin metallic or colored foil onto paper** with a heated die, leaving a shiny gold/silver (or colored) mark. A decorative finishing effect.

Font embedding

> Storing the **actual font data inside the PDF** so the file prints with the exact typeface even on a machine that doesn't have that font installed. Required for reliable print.

Font subsetting

> Embedding **only the characters actually used** in the document rather than the whole font, to keep file size down. Standard for print PDFs.

Gamut

> The **full range of colors a device can produce** (or a color space can describe). A camera or screen has a wide gamut; CMYK print has a smaller one — which is why some bright screen colors can't be printed. (See **gamut mapping**.)

Gamut mapping

> The process of **fitting colors from a larger gamut into a smaller one** when converting (for example, screen RGB to CMYK). The chosen **rendering intent** decides how out-of-gamut colors are handled.

GCR (Gray Component Replacement)

> A black-generation technique that **replaces the gray part built from cyan+magenta+yellow with black ink** across the whole tonal range. Saves colored ink, improves gray stability and registration. (Compare **UCR**.)

Glyph

> The drawn shape of a single character in a font — the visible form of the letter "a." One character can map to several glyphs (for example, alternate styles or ligatures).

GSM (grams per square meter)

> The standard **weight/thickness measure of paper**: grams per square meter. Higher GSM = heavier, stiffer stock (copy paper ~80 gsm, business cards ~350 gsm).

Halftone

> The trick of simulating **continuous shades using patterns of tiny dots** of solid ink. From a normal viewing distance the dots blur together into smooth tone. The foundation of nearly all ink-on-paper imaging.

HSL / HSV

> Two **human-friendly ways to describe RGB color**: by Hue (which color), Saturation (how vivid), and Lightness or Value (how bright). Easier for people to adjust than raw R/G/B numbers, but still device-dependent.

ICC profile

> A standardized file (from the International Color Consortium) that **describes how a specific device reproduces color** — how its numbers map to real, measurable color. Profiles let the CMM translate color faithfully between devices.

Imposition

> Arranging the **pages of a document onto a large press sheet** in the right positions and order so that, after printing, folding, and trimming, they read in sequence. (See **signature**.)

Kerning

> Adjusting the space between two specific letters so the spacing looks even — for example tucking a "T" and "o" closer together. Different from tracking, which spaces a whole run uniformly.

Knockout

> When a top object **removes (knocks out) the ink beneath it** so colors don't mix — the default behavior. The opposite of **overprint**.

Lab (CIELAB)

> A device-independent color model that describes color the way the human eye perceives it, using Lightness plus two color axes (a, b). Because it doesn't depend on any device, it is used as the neutral reference for conversions (the PCS).

Leading

> The vertical space between lines of text, measured baseline to baseline. Named after the strips of lead once placed between rows of metal type.

LPI (lines per inch)

> The **frequency of the halftone screen** — how many rows of halftone dots per inch. Higher LPI = finer detail but needs better paper and press. Distinct from DPI (device dots) and PPI (image pixels).

Makeready

> All the **setup work before a print run** proper begins: mounting plates, setting registration, balancing ink and water, and running test sheets until color and position are right. Wasted paper here is "makeready spoilage."

MediaBox

> The PDF page box defining the **full physical size of the page sheet**. It's the largest, mandatory box; all other boxes (TrimBox, BleedBox, etc.) sit inside it.

Moiré

> An **unwanted interference pattern** of ripples or grids that appears when two regular patterns overlap badly — often from halftone screens at wrong angles, or from scanning a printed image. Correct screen angles produce a clean **rosette** instead.

Offset (offset lithography)

> The dominant high-volume print method: ink is transferred from a **plate to a rubber blanket and then onto the paper**. Excellent quality and low per-unit cost at large quantities, but needs plates and makeready. (Contrast with **digital print**.)

OpenType

> A modern, cross-platform font format that can hold thousands of glyphs plus smart features like ligatures and alternates. It supersedes older formats and works the same on Windows and Mac.

Outlines (converting text to outlines)

> Turning live text into plain vector shapes so it no longer needs the font. It guarantees the look but makes the text uneditable and unsearchable, so it's a last resort, not a substitute for embedding.

Output intent

> An ICC profile **embedded in a PDF/X file** that declares the intended printing condition (the target paper/press, e.g. a specific coated-stock standard). It tells everyone downstream exactly what the CMYK numbers mean.

Overprint

> Setting an object to **print on top of the ink beneath it instead of knocking it out**, so the inks mix. Used deliberately (e.g. black text over a background) and a frequent source of surprises if set by accident. (Opposite of **knockout**.)

Pantone (PMS)

> The Pantone Matching System — a catalog of premixed spot-color inks, each with a number (like `PMS 185`). Brands specify them so a logo color stays identical across every printer and material.

Pantone / PMS (Pantone Matching System)

> A standardized library of **pre-mixed spot-color inks** identified by number, so a brand color looks the same everywhere regardless of press. (See **spot color**.)

PCS (Profile Connection Space)

> The **neutral middle ground** (CIELAB or CIE XYZ) that color management converts *into* from the source and then *out of* to the destination. Every conversion passes through the PCS.

PDF content stream

> See *Content stream* — the sequence of drawing operators that paints a PDF page.

PDF object

> A numbered building block inside a PDF — a page, a font, an image, a dictionary of settings. The whole file is a **collection of objects that reference each other**, located via the cross-reference table (**xref**).

PDF/X

> A family of restricted PDF standards built specifically for reliable printing. They forbid risky things (like missing fonts or unspecified color) and require an output intent, so the file prints predictably anywhere.

PDF/X-1a

> A strict print-ready PDF standard that requires **all color be CMYK or spot** (no RGB), all fonts embedded, and an output intent set. The safest, most locked-down choice; no transparency or RGB

> allowed.

PDF/X-3

> A print-ready standard like X-1a but **also allows color-managed RGB/Lab** content with embedded profiles, leaving final conversion to the RIP. More flexible than X-1a.

PDF/X-4

> A modern print-ready standard that **allows live transparency and layers** (no forced flattening), plus color-managed content. The current preferred target for most workflows.

Perceptual (rendering intent)

> A rendering intent that **smoothly compresses the whole source gamut to fit the destination**, shifting all colors a little to preserve the relationships between them. Best for photographs with many out-of-gamut colors.

Perceptual intent

> A rendering intent that gently compresses the whole image's colors to fit the smaller print gamut, preserving the relationships between colors. Best for photographs, where smooth gradients matter more than exact values.

Perfect binding

> A binding method where pages are **glued at the spine into a square-edged cover** (like a paperback book or thick catalog). Suited to higher page counts. (Compare **saddle-stitch**.)

PPI (pixels per inch)

> How many **image pixels fall into each inch** when a raster image is placed at a given size. The key resolution measure for photos in a layout; ~300 PPI at final size is the usual target for quality print. Often loosely called "DPI."

Preflight

> Automatically **checking a file against print requirements** before it goes to press — looking for low-resolution images, RGB color, missing fonts, missing bleed, overprints, and so on. Named after a pilot's pre-flight checklist.

Preflight fixup

> An **automatic correction** applied during preflight — for example converting RGB to CMYK, embedding fonts, or adding bleed — to bring a file into spec.

Premultiplied alpha

> A way of storing transparent images where each pixel's color values are already multiplied by its opacity. It speeds up compositing but must be "un-multiplied" correctly, or semi-transparent edges pick up dark or bright fringes.

Process color

> Color built from the four standard CMYK inks, as opposed to a premixed spot ink. Photographs and full-color artwork are reproduced with process color.

Raster (bitmap)

> An image made of a **grid of pixels**, each with its own color. Photographs are raster. They look great at their native size but go blocky or soft when enlarged. (Contrast with **vector**.)

Rasterization

> Converting vector shapes or text into a grid of pixels for display or output. It happens whenever a resolution-independent design is turned into something a screen or print head can actually paint.

Registration

> The **precise alignment of the separate ink colors** on top of one another. When colors are misaligned ("out of registration") you see colored fringes. (See **trapping**, which guards against small misregistration.)

Registration black (registration color)

> A special "color" set to 100% of every plate (C, M, Y, and K) used only for printer's marks so they print on every plate. Never use it for actual design elements — it dumps far too much ink.

Registration marks

> Small crosshair-in-a-circle targets placed outside the page that let the press align each color plate exactly on top of the others.

Rendering intent

> The chosen strategy for handling colors that exist on the source device but not on the destination — whether to shift everything to keep relationships natural (perceptual) or keep what fits exact and clip the rest (colorimetric).

Resampling

> Changing the **pixel count of a raster image** — downsampling (fewer pixels) for smaller files, or upsampling (more pixels), which can't add real detail and tends to soften the image.

RGB

> The **additive Red-Green-Blue color model** used by cameras, screens, and scanners. Wide gamut but device-dependent and not directly printable — it must be converted to CMYK or handled by color management for print.

Rich black

> A deeper black made by **adding some cyan, magenta, and/or yellow under the black ink** (e.g. C40 M30 Y30 K100), rather than using K100 alone, for large solid areas. Must respect the **TAC** limit. (Plain K100 is fine for small text.)

RIP (Raster Image Processor)

> The software/hardware that **turns a PDF into the dot pattern the print device actually lays down** — interpreting the file, doing color conversion, applying halftone screening, and producing the final raster for each ink. The brain between file and press.

Rosette

> The small, regular **flower-like pattern formed when the four CMYK halftone screens are set at their correct angles**. A clean rosette means good screening; a bad one becomes **moiré**.

Saddle-stitch

> A binding method where folded sheets are **stapled through the spine fold** (like a thin magazine or brochure). Cheap and fast; best for lower page counts. (Compare **perfect binding**; relates to **creep**.)

Safe area (safe zone / margin)

> An inner margin **inside the trim edge** where important content (text, logos) should stay, so nothing critical gets cut off if the trim shifts. The counterpart to **bleed** on the outside.

Safe zone (safe area)

> An inner margin (commonly `3–5 mm` from the trim) where you keep important text and logos, so nothing critical gets clipped if the cut drifts slightly inward.

Screen angle

> The **angle at which each color's halftone grid is rotated** (commonly C 15°, M 75°, Y 0°, K 45°). Correct angles produce a clean rosette; wrong ones cause **moiré**.

Screen print (screen printing)

> Pushing ink through a **fine mesh stencil, one color at a time**, onto a surface (T-shirts, posters, signage). Great for bold solid colors and many substrates; setup per color makes it best for larger runs of few colors.

Screening

> The overall process of converting continuous tones into a printable pattern of dots — including the choice of halftone shape, frequency (LPI), and angle for each ink.

Separation color space

> A PDF color space that defines a **single named colorant** — typically a spot color (a Pantone ink) or a varnish — so it can ride through the workflow as its own plate. (See **spot color**.)

Shingling

> Another name for the inward content shift applied to compensate for **creep** in saddle-stitched booklets. (See **creep**.)

Signature

> A **single large press sheet printed with multiple pages** that, once folded, becomes a section of the finished book in correct page order. Books are assembled from several signatures. (See **imposition**.)

Slug

> An area outside the bleed used for production notes, job names, or instructions to the printer. It is trimmed off and never appears on the finished piece.

Soft proof

> A **simulation on a calibrated screen of how a file will look when printed** on a specific paper/press, using its profile. Lets you preview gamut clipping and paper-white before committing ink.

Spectrophotometer

> An instrument that measures the **full spectrum of light a sample reflects** and reports its actual color (e.g. in Lab). Used to build profiles, verify Pantone matches, and check color with **Delta-E**. (Compare **densitometer**.)

Spot color

> A **single pre-mixed ink printed on its own plate**, rather than simulated from CMYK dots. Used for exact brand colors, metallics, or fluorescents. (See **Pantone**, **separation color space**.)

Spot UV

> A finishing effect applying a **glossy UV-cured coating to selected areas only**, creating shine and contrast against a matte background (e.g. a glossy logo on a matte card).

Spread

> A type of trap where a lighter foreground object is expanded slightly outward into the darker background, overlapping the edge to hide registration gaps. The opposite of a choke.

Subsetting

> Embedding only the specific glyphs a document actually uses instead of the entire font, which keeps the PDF small while still printing correctly.

Subtractive color

> Making colors by **removing (absorbing) light with inks or dyes**. Each ink subtracts certain wavelengths; layering cyan, magenta, and yellow removes more light and trends toward dark. How ink-on-paper printing works. (Contrast with **additive color**.)

TAC (Total Area Coverage / total ink coverage)

> The sum of all four CMYK percentages in the heaviest part of a file (so the maximum is 400%). Presses cap it — often around `240–300%` — because too much ink won't dry, smears, and sticks pages together.

TAC / TIC (Total Area Coverage / Total Ink Coverage)

> The **sum of all four ink percentages** in the darkest spot of a file (e.g. C+M+Y+K). Presses cap this (commonly ~300% coated, lower for uncoated/newsprint) to prevent ink not drying, set-off, and smearing.

Tracking

> Adjusting the spacing evenly across a whole word, line, or block of text — for example loosening letterspacing on a small-caps heading. Unlike kerning, it isn't pair-specific.

Transparency flattening

> Converting **live transparency effects (soft shadows, blends, opacity) into solid opaque artwork** that older devices/standards can print. Required for PDF/X-1a; avoided in PDF/X-4, which keeps transparency live.

Trapping

> Deliberately **overlapping adjacent colors by a hair** so that small misregistration on press doesn't leave a white gap between them. Usually handled automatically by the RIP. (See **registration**.)

Trim

> The **final cut line — the finished size of the piece** after the press sheet is cut down. Bleed sits outside it; the safe area sits inside it. (See **TrimBox**.)

TrimBox

> The PDF page box that defines the **finished page size after trimming**. The single most important box for print: it tells the workflow exactly where to cut.

TrueType

> A long-established outline font format (originally from Apple and Microsoft) that describes glyphs with quadratic curves. Reliable and widely supported, though newer designs often ship as OpenType.

UCR (Under Color Removal)

> A black-generation technique that **removes overlapping C, M, and Y only in the neutral shadow areas** and replaces them with black, reducing ink in the darkest tones. Narrower than **GCR**, which works across the whole range.

Uncoated paper

> Paper with **no surface coating**, so it's more absorbent and tactile (letterhead, many business cards). Ink soaks in, giving a softer look, more **dot gain**, and a lower ink limit. (Contrast with **coated**.)

Vector

> Artwork described by **math — points, lines, and Bézier curves** — rather than pixels. It scales to any size with no quality loss, ideal for logos, type, and line art. (Contrast with **raster**.)

Xref (cross-reference table)

> The **index at the end of a PDF that lists where every object lives** in the file, so a reader can jump straight to any object. A corrupted xref is a common cause of "this PDF won't open."

Section summary:

- Color terms split into models (RGB/CMYK/Lab/HSL), management plumbing (ICC, PCS, CMM, rendering intents), and ink-on-page behavior (spot, overprint, GCR/UCR, TAC, dot gain).
- Image-quality terms hinge on raster vs vector, and on keeping DPI, PPI, and LPI distinct — they measure device dots, image pixels, and screen frequency respectively.
- The print-ready chain runs file → preflight → PDF/X with the right page boxes and output intent → RIP → press, with measurement (Delta-E, densitometer, spectrophotometer) closing the loop.
- Geometry and finishing terms (bleed, trim, safe area, imposition, signature, creep, binding, foil/emboss/spot UV) cover how flat artwork becomes a physical, cut, folded, bound piece.

Frequently Asked Questions

This section answers the practical questions a beginner running real print work bumps into again and again. Each answer is plain-English and short enough to act on. A few quick definitions first, so the answers read smoothly.

RGB

> Red-Green-Blue — the way screens make color by mixing **light**. Lots of light = bright, glowing color.

CMYK

> Cyan-Magenta-Yellow-Key(black) — the way presses make color by mixing **ink** on paper. More ink = darker.

DPI / PPI

> Dots (or pixels) per inch — how much fine detail an image has packed into each printed inch.

Preflight

> An automated "pre-flight check" of your PDF that flags problems (low-res images, missing fonts, wrong color, no bleed) before printing.

RIP

> Raster Image Processor — the software/hardware that turns your PDF into the exact dot pattern the printer lays down.

Why do my colors look duller when printed than on screen?

Your screen makes color with **emitted light** (RGB), which can glow far brighter and more saturated than any ink can. Print makes color with **ink absorbing light** (CMYK) on non-glowing paper. The set of colors a process can reproduce is called its **gamut**, and CMYK's gamut is smaller than your screen's — especially for vivid blues, greens, and oranges. When a bright RGB color falls outside the CMYK gamut, it gets "pulled in" to the nearest printable color, which looks duller. This is normal physics, not a mistake.

> **Best practice:** Soft-proof in your design app (View > Proof Colors with the right CMYK profile) so you see the printable version on screen *before* you commit, and avoid relying on neon RGB values for anything important.

What DPI do I really need?

The standard answer for most printing is **300 DPI at the final printed size**. That gives enough detail that individual dots disappear at normal reading distance. The two traps: (1) "300 DPI" only means something *at the size you actually print it* — a 300-DPI image blown up 2× becomes effectively 150 DPI and turns soft; (2) large-format work viewed from far away (banners, billboards) needs far less, often **100–150 DPI** or even less, because your eye can't resolve it from across the room.

Common mistake: Pulling a small image off a website (usually 72 DPI) and stretching it to fill a flyer. It looks fine on screen and blurry in print. Always check the *effective* DPI after scaling, not the original number.

RGB or CMYK — which should I design in, and when do I convert?

Design and edit in a wide color space (RGB photos, or a working RGB space), because editing tools and effects behave best there and you keep maximum color information. Convert to **CMYK at the end**, using the specific profile your printer asks for, and review the result. Converting too early locks you into the small gamut and can make later edits look wrong; converting too late (handing raw RGB to a printer who auto-converts) means a machine guesses your colors instead of you choosing them.

Best practice: Ask your print shop which CMYK profile they want (e.g., a sheet-fed coated profile vs. an uncoated one) and convert to *that exact one*. "CMYK" is not one thing.

What is bleed and why 3 mm?

Bleed is artwork that extends *past* the final trim edge so that when the cutter slices the stack of sheets, there's no thin white sliver if the blade lands a hair off. Cutting hundreds of sheets at once is never perfectly precise, so you give the blade room to be wrong. **3 mm** (about 1/8 inch) is the widely accepted standard amount; some jobs use 5 mm. Keep important content (text, logos) inside a **safe margin** of roughly 3 mm *inside* the trim, so nothing critical gets clipped.

Which PDF/X should I export?

PDF/X is a family of "print-safe" PDF standards that force fonts to be embedded, color to be defined, and risky features stripped out. The two you'll meet most: **PDF/X-1a** (everything flattened to CMYK + spot colors, very predictable, the safe default if unsure) and **PDF/X-4** (allows live transparency and color management, better for modern workflows). When in doubt, **ask the printer which they accept** and pick the matching export preset.

Best practice: Use your design app's built-in PDF/X preset rather than custom-tweaking export settings by hand — the preset already enforces the right rules.

Why did my fonts change at the printer?

If the font isn't **embedded** in the PDF (packaged inside the file) and the printer's computer doesn't have it installed, their software substitutes a different font. Spacing, line breaks, and shapes all shift. Exporting as PDF/X embeds fonts automatically. For logos and headline type, you can also **outline (convert to curves)** the text so it becomes a shape that can never be substituted — but keep an editable copy first, because outlined text can't be edited or spell-checked.

Common mistake: Outlining *all* body text "to be safe." It can slightly thicken small text and removes any chance to fix a typo. Embed fonts for body copy; outline only logos/special display type if needed.

What is rich black, and when should I use it?

Plain black is 100% K (black ink only) and can look weak or grayish over large areas. **Rich black** adds some cyan/magenta/yellow under the black (a common recipe is C40 M30 Y30 K100) to make big black areas look deep and solid. Use rich black for **large black backgrounds and big shapes**. Use **plain 100% K** for small body text — rich black under tiny letters can show colored fringes if registration (ink alignment) drifts.

Why are there registration marks and color bars on the sheet?

Those marks live in the margin outside your artwork and are tools for the press operator. **Registration marks** (crosshair targets) let them confirm all four ink plates line up exactly — if cyan is shifted, the crosshairs won't overlap. **Color bars** are strips of known ink patches the operator measures to keep ink density and color consistent across the whole run. They're trimmed off the final piece.

What causes moiré, and how do I avoid it?

Moiré is an ugly interference pattern — wavy grids or rosettes that weren't in your original. It happens when two regular dot/line patterns overlap at a bad angle: e.g., printing a photo that was *already* printed and scanned (so it has its own halftone dot grid) on top of the press's halftone grid. Avoid it by using original continuous-tone images, not scans of printed material, and by letting the RIP set proper screen angles.

Best practice: If you must reproduce something already printed, descreen it (a scanner/Photoshop option that blurs out the old dot grid) before placing it.

My file failed preflight — what now?

Read the report; it names the exact issue. Most failures fall into a few buckets: **low-resolution images** (replace with higher-res or scale down the placement), **RGB or spot colors where CMYK was expected** (convert), **missing fonts** (embed/outline), **no bleed** (extend artwork and re-export), or **hairline strokes** too thin to print. Fix in the *source file* and re-export — don't try to patch the PDF blindly. Warnings can sometimes be ignored; **errors** usually cannot.

Spot color vs CMYK — when is Pantone worth it?

A **spot color** (e.g., a Pantone) is a single pre-mixed ink, like buying paint in the exact shade. CMYK *simulates* colors by mixing tiny dots of four inks. Spot color is worth it when you need a **color CMYK can't hit** (bright orange, metallic, a precise brand color), **exact brand consistency** across many print runs, or special effects (neon, metallic, varnish). The trade-off: each spot color is an extra plate and extra cost, so for full-color photos you stick with CMYK.

Aspect	CMYK (process)	Spot (Pantone)
How it makes color	Mixes 4 ink dots	One pre-mixed ink
Best for	Photos, full-color art	Logos, exact brand color, specials
Consistency run-to-run	Good, can drift	Excellent
Cost	Standard	Extra plate per color
Can do metallic/neon?	No	Yes

Why does heavy ink coverage cause problems?

Every ink you stack adds wetness. Pile too much on and the paper can't absorb or dry it — you get **set-off** (wet ink smearing onto the next sheet), slow drying, and curling. Printers cap this with **Total Ink Coverage / TAC** (also called TIL), the sum of all four ink percentages in any spot. Typical limits are about **300% for coated paper** and lower (~240–260%) for uncoated. So a "super black" of C100 M100 Y100 K100 (= 400%) is a problem; a proper rich black stays under the limit.

> **Common mistake:** Building deep colors or black by maxing all four channels. Always respect the printer's TAC limit; a good CMYK profile keeps you inside it automatically.

What is a RIP?

The **RIP (Raster Image Processor)** is the translator between your design and the physical printer. Your PDF describes shapes, text, and images mathematically; the printer can only lay down a grid of tiny dots. The RIP *rasterizes* — converts everything into the exact dot pattern, applies the halftone screening, handles color conversion, and manages how each ink plate prints. Think of it as the press's brain: same PDF through two different RIPs can look slightly different.

> **Analogy:** The PDF is sheet music; the RIP is the musician who actually plays it on a specific instrument. The notes are fixed, but the rendering depends on who (and what) performs it.

How close in color is "close enough" (Delta-E)?

Delta-E (ΔE) is a single number measuring the difference between two colors — 0 means identical, bigger means more different. Rough rule of thumb: **ΔE under 1** is invisible to most people, **1–2** is

noticeable only to a trained eye on close inspection, **2–3.5** is the realistic target for good production color, and **above ~5** is an obvious mismatch a client will reject. Brand colors are usually held to a tight tolerance (often ΔE ≤ 2).

What's the difference between trim, bleed, and safe area again?

Trim

> The final cut line — the actual size of the finished piece.

Bleed

> Artwork extending ~3 mm past trim, so cuts never leave white edges.

Safe area

> A margin ~3 mm *inside* trim — keep text and logos here so nothing important gets cut.

Should I send my native design file or a PDF?

Almost always a **print-ready PDF/X**. It locks fonts, images, and color into one self-contained file that prints the same everywhere, with nothing to go missing. Native files (the original design-app document) link out to separate fonts and images that are easy to lose or change. Send native files only if the printer specifically asks (e.g., they need to make edits) — and then **package** the file so all linked assets travel with it.

Best practice: Whatever you send, also send a flattened JPG/PDF "this is how it should look" reference so the printer can sanity-check the output against your intent.

Why does my thin line or hairline disappear in print?

"Hairline" was an old setting meaning "as thin as possible," which on a high-resolution press can be thinner than a single printable line — so it nearly vanishes or breaks up. Set a **minimum stroke weight of about 0.25 pt (roughly 0.1 mm)** for solid colors, and thicker for reversed (white-on-color) lines, which need extra weight because surrounding ink can choke them.

Section summary:

- **Color physics is the root of most surprises:** screens glow (RGB), ink absorbs (CMYK), so printed color is duller and smaller in gamut — soft-proof and convert with the printer's exact profile, late, not early.
- **Mechanical tolerances drive the rules:** bleed (~3 mm), safe margins, registration marks, and minimum line weights all exist because cutting and ink alignment are never perfect.
- **Export print-ready PDF/X** to embed fonts and lock color; let **preflight** catch low-res images, RGB, and missing bleed before they reach the press.
- **Respect ink limits:** use rich black for big areas and plain K for small text, and stay under the paper's TAC limit (~300% coated) to avoid smearing and drying problems.
- **Know your targets:** ~300 DPI at final size, ΔE around 2–3.5 for solid production color, and Pantone spot inks only when CMYK genuinely can't deliver the color or consistency you need.

Revision Cheat Sheet

Everything that matters for an exam or a press job, packed tight. Skim the headers, grab the numbers, go.

1. Color Spaces at a Glance

Space	Channels	Add / Subtract	Where it lives	Use it for
RGB	Red, Green, Blue	Additive (light)	Screens, cameras, scanners	On-screen design, web, photo capture
CMYK	Cyan, Magenta, Yellow, Key/Black	Subtractive (ink)	Printing presses	Final print output
Lab (CIELAB)	L* (light), a* (g–r), b* (b–y)	Device-independent	Color management hub	Conversions, measuring ΔE
Spot (e.g. Pantone)	1 pre-mixed ink	Subtractive	Special inks	Exact brand colors, metallics, neons
Grayscale	1 (black only)	Subtractive	B&W print	Text-only / 1-color jobs

Additive vs subtractive (one-liner): Additive (RGB) adds colored *light* — all on = white. **Subtractive (CMYK)** adds *ink* that subtracts light from white paper — all on = (muddy) dark.

2. Rendering Intents — Quick Chooser

Intent	What it does	Pick it for
Perceptual	Shifts *all* colors to keep relationships natural; sacrifices accuracy of in-gamut colors	Photos with out-of-gamut colors
Relative Colorimetric	Keeps in-gamut colors exact; clips out-of-gamut to nearest edge; remaps white point	Most print work (logos + photos), default choice
Saturation	Maximizes vividness over accuracy	Charts, graphics, business graphics
Absolute Colorimetric	Like relative but keeps source white point (simulates paper)	Proofing / matching another paper stock

Best practice: Default to **Relative Colorimetric + Black Point Compensation** for general print; switch to **Perceptual** only when an image has obvious out-of-gamut saturated areas.

3. Halftone Screening — Angles, LPI vs DPI

Standard CMYK screen angles (keeps rosette pattern, avoids moiré):

Channel	Cyan	Magenta	Yellow	Black (K)
Angle	15°	75°	0°	45°

- **Yellow at 0°** because it's the lightest — moiré shows least there.
- **Black at 45°** — the most visible ink sits at the angle least noticed by the eye.
- C and M sit 30° apart from K (15°, 75°).

LPI (lines per inch)

> Halftone screen frequency — how many rows of dots per inch. Higher = finer detail. Newspaper ~85, magazine/sheet-fed ~150, high-end ~175–200.

DPI (dots per inch)

> Output/image resolution — how many addressable dots the device or file has.

The 2× rule: Image resolution should be about **DPI ≈ 2 × LPI**. For a **150 lpi** screen, supply **300 dpi** images. (Range 1.5×–2×; 2× is the safe standard.)

4. Ink Limits, Rich Black, Dot Gain

Item	Sheet-fed offset	Web offset / newsprint
TAC (Total Area Coverage / ink limit)	≈ 300–340%	≈ 240–280% (newsprint ~240%)
Typical dot gain (midtones)	≈ 12–18%	≈ 20–30%

TAC / TIL

> Total Area Coverage = sum of all four ink % in the darkest spot. Over the limit → ink won't dry, smears, sets off.

Dot gain

> Dots print bigger than intended (ink spreads into paper) → image darkens. Profiles compensate.

Rich black recipe: A common screen/coated value is **C60 M40 Y40 K100** (≈240% total). For large solids, **C40 M30 Y30 K100** stays under web limits. **Plain text = K100 only** (never rich black on small type — registration shows colored fringes).

Common mistake: Using "registration black" (C100 M100 Y100 K100 = 400%) for body text. It blows past every ink limit and won't dry. Registration color is ONLY for crop marks.

5. Resolution Rules

- **Photos / raster:** 300 dpi at final print size.
- **Line art / bitmap:** 1200 dpi.
- **Type & logos:** use **vector**, not raster — resolution-independent, razor-sharp at any size.
- **Large-format / billboards:** lower dpi OK (100–150 dpi) because viewing distance is far.

Effective dpi after scaling: Effective dpi = native dpi ÷ scale factor. A 300 dpi image placed at **200%** drops to **150 dpi**. Enlarging lowers effective resolution — check it after placement.

6. PDF/X Cheat

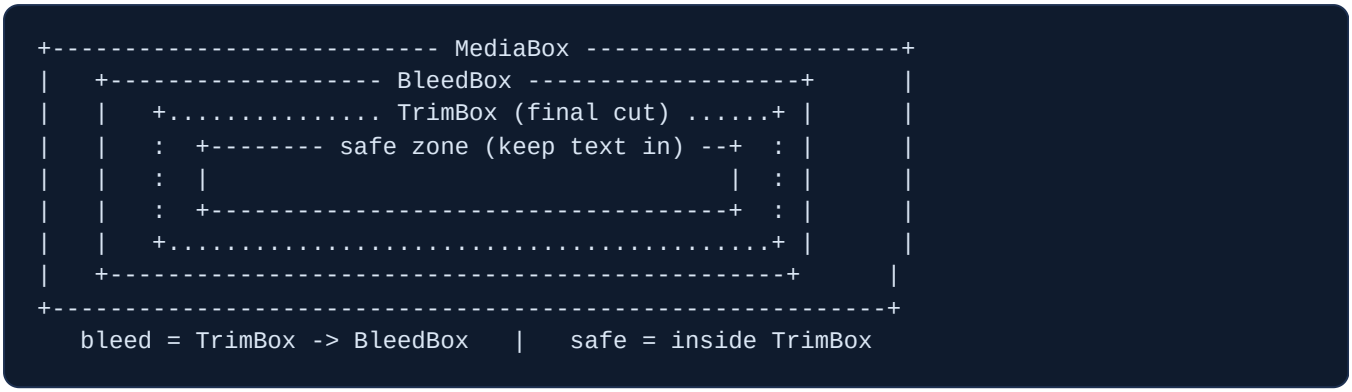
Standard	Color allowed	One-line summary
PDF/X-1a	CMYK + spot only (no RGB)	Everything flattened, fonts embedded, all-CMYK — safest for legacy offset workflows.
PDF/X-3	CMYK, spot *and* color-managed RGB/Lab	Allows tagged RGB with an output intent; printer converts at RIP.
PDF/X-4	Same as X-3 + live transparency & layers	Modern default — keeps transparency live, no forced flattening; best for digital/RIP.

Output intent

> The embedded ICC profile that says "this file is built for THIS press/paper" (e.g. *GRACoL 2013*, *FOGRA39/51*, *SWOP*). Required in every PDF/X file.

7. PDF Page Boxes

Box	What it defines
MediaBox	The full sheet — outermost boundary (paper size).
CropBox	Visible/displayed region in a viewer.
BleedBox	Trim + bleed margin (where ink runs past the cut).
TrimBox	Final size after cutting — the most important box for print.
ArtBox	Meaningful content extent (rarely used).



Bleed / trim / safe typical values: Bleed = **3 mm** (≈ **0.125 in / 1/8")** beyond trim. Safe margin = keep critical text **3 mm** inside the trim. Always set the **TrimBox** correctly — the RIP cuts to it.

8. Preflight Quick Checklist

1. Color mode is **CMYK** (or spot) — no stray RGB, no untagged.
2. **Fonts fully embedded** (or outlined).
3. Images ≥ **300 dpi** effective at placed size.
4. **Bleed present** (3 mm) and TrimBox set.
5. **TAC under limit** (≤300% sheet-fed / ≤240% web).
6. **Black text = K-only**, overprint set correctly.
7. Spot colors named correctly / converted as intended (no rogue swatches).
8. **Overprint & knockout** reviewed (white objects NOT set to overprint).
9. Output intent / ICC profile embedded (PDF/X).
10. Correct page size, count, and orientation; no hairlines < 0.25 pt.

Common mistake: White text accidentally set to **overprint** → it vanishes on press (prints "on top of" color as nothing). Always preflight overprint preview before sending.

9. Print Methods Quick Chooser (by run length)

Method	Best run length	Sweet spot
Digital (toner/inkjet)	1 – ~500	Short runs, variable data, fast turnaround, proofs
Offset (litho)	~500 – 100,000+	High-quality, high-volume flat stock (books, brochures)
Screen printing	Dozens – thousands	Apparel, thick/vivid spot ink, specialty substrates
DTG (direct-to-garment)	1 – small batches	Full-color photo prints on garments, no minimums
Flexography	Very long (10,000s+)	Packaging, labels, film, corrugated rolls
Large-format inkjet	1 – few	Banners, posters, signage, vehicle wraps

Crossover rule of thumb: Below a few hundred copies → **digital** wins on cost; above ~500–1,000 → **offset** wins because plate cost spreads thin.

10. Delta-E (ΔE) Tolerance — Rule of Thumb

ΔE value	What it means
< 1	Imperceptible — even trained eyes can't tell.
1 – 2	Perceptible only on close side-by-side inspection.
2 – 3	Acceptable for most commercial print / contract proofs.
3 – 5	Noticeable; tolerable for non-critical work.
> 5	Clearly different color — usually a reject.

Delta-E (ΔE)

> The measured distance between two colors in Lab space. Smaller = closer match. Measure with a spectrophotometer against the reference. (ΔE2000 is the modern, eye-weighted formula.)

11. The Whole Pipeline in One Line

```
design (RGB/Lab)
   -> color-manage (assign/convert via ICC, set intent)
   -> export PDF/X (X-1a/X-3/X-4 + output intent)
   -> preflight (color, fonts, dpi, bleed, TAC, overprint)
   -> RIP / screen (rasterize, halftone @ angles, LPI)
   -> plate / press (offset/digital/etc.)
   -> measure (spectrophotometer -> ΔE vs reference)
```

Section summary:

- **RGB = additive light, CMYK = subtractive ink, Lab = the neutral hub** for conversion and ΔE measurement.
- Remember the magic numbers: screen angles **C15 / M75 / Y0 / K45**, **DPI ≈ 2× LPI** (300 dpi for 150 lpi), bleed **3 mm / 0.125"**, TAC **~300% sheet-fed / ~240% web**.
- **PDF/X-1a** = all-CMYK/flattened, **X-3** = adds managed RGB, **X-4** = adds live transparency — all need an **output intent**; **TrimBox** is the box that matters.
- Pick the method by run length: **digital** short, **offset** long, plus screen/DTG/flexo/large-format for their specialties.
- Judge color with **ΔE**: under 1 invisible, 2–3 acceptable, over 5 reject — and always preflight before the press eats your mistake.